

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ AHB LITE-UART IP VÀ XÂY
DỰNG MÔI TRƯỜNG KIỂM CHỨNG UVM
NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **TRẦN ĐỨC VINH**

MSSV: 22161339

NGUYỄN VĂN ĐỨC TRẮNG

MSSV: 22161325

Hướng dẫn: **PGS.TS. PHAN VĂN CA**

TP. HỒ CHÍ MINH – 06/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
THIẾT KẾ AHB LITE-UART IP VÀ XÂY
DỰNG MÔI TRƯỜNG KIỂM CHỨNG UVM

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **TRẦN ĐỨC VINH**

MSSV: 22161339

NGUYỄN VĂN ĐỨC TRẮNG

MSSV: 22161325

Hướng dẫn: **PGS.TS. PHAN VĂN CA**

TP. HỒ CHÍ MINH – 06/2026

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Trần Đức Vinh

MSSV: 22161339

Họ và tên: Nguyễn Văn Đức Trắng

MSSV: 22161325

2. Thông tin đề tài

Tên đề tài: Thiết kế AHB Lite - UART IP và xây dựng môi trường kiểm chứng UVM

Mục đích đề tài: Thiết kế và xây dựng lõi AHB Lite - UART IP hỗ trợ đầy đủ các chức năng truyền nhận dữ liệu nối tiếp, đồng thời xây dựng môi trường kiểm chứng theo phương pháp UVM nhằm đánh giá và xác minh chức năng của lõi IP thiết kế.

3. Nội dung thực hiện

- Nghiên cứu giao thức AHB Lite, APB và UART; tìm hiểu phương pháp kiểm chứng UVM.
- Thiết kế lõi AHB Lite - UART IP bằng ngôn ngữ mô tả phần cứng Verilog/SystemVerilog, bao gồm cầu nối AHB-APB, bộ tạo tốc độ Baud, FIFO truyền/nhận và cơ chế ngắt.
- Xây dựng môi trường kiểm chứng UVM gồm AHB Agent, UART Agent, Scoreboard và mô hình độ bao phủ chức năng.
- Thực hiện mô phỏng, kiểm chứng chức năng và đánh giá độ bao phủ của lõi IP trên phần mềm QuestaSim.

4. Lời cam đoan của sinh viên

Chúng tôi cam đoan đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn. Các kết quả, số liệu và nội dung trình bày trong đồ án là trung thực, được trích dẫn đầy đủ theo quy định và chưa từng được sử dụng để bảo vệ bất kỳ học vị nào trước đây. Chúng tôi hoàn toàn chịu trách nhiệm về nội dung cam đoan này.

TP.HCM, ngày 22 tháng 6 năm 2026

Sinh viên thực hiện:

Trần Đức Vinh

Nguyễn Văn Đức Trắng

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Tính kế AHB Lite - UART IP và xây dựng mô-

hình kiểm chứng MVM

Sinh viên: + Trần Đức Vinh

MSSV: 22.161.339

+ Nguyễn Văn Đức Tráng

MSSV: 22.161.325

Hướng dẫn: P.G.S.T.S. Phan Văn Ca

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề phù hợp, đề xuất giải quyết hợp lý.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện phù hợp và vấn đề đặt ra.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Đề tài có mô phỏng kiểm chứng kết quả.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Hình thức trình bày tốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Sinh viên thể hiện khả năng làm việc thiết kế hệ thống.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Số tài liệu trích dẫn.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đề nghị tra cứu và.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tốt.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Bảo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Đề nghị cho phép bảo vệ và.

Tp. HCM, ngày 24 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Phan Văn Ca

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trước hết, chúng tôi xin gửi lời cảm ơn chân thành và sâu sắc đến thầy PGS.TS. Phan Văn Ca, người đã tận tình hướng dẫn, hỗ trợ và định hướng chuyên môn trong suốt quá trình thực hiện đề tài. Sự tận tâm và những kiến thức quý báu mà thầy truyền đạt là nguồn động lực lớn giúp chúng tôi hoàn thành đồ án một cách hiệu quả và bài bản.

Chúng tôi cũng xin chân thành cảm ơn quý thầy cô trong Khoa Điện Điện Tử, Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh đã tận tình giảng dạy và truyền đạt kiến thức trong suốt những năm học vừa qua, tạo nền tảng vững chắc để chúng tôi có thể nghiên cứu và thực hiện đề tài này.

Cuối cùng, chúng tôi xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, ủng hộ và tạo điều kiện thuận lợi để chúng tôi có thể tập trung hoàn thành tốt nhiệm vụ được giao.

Mặc dù đã cố gắng hết sức trong quá trình thực hiện, song do giới hạn về thời gian và kinh nghiệm, đồ án không tránh khỏi những thiếu sót nhất định. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và hội đồng phản biện để hoàn thiện hơn trong các nghiên cứu tiếp theo.

TÓM TẮT

Đề tài “Thiết kế AHB Lite–UART IP và xây dựng môi trường kiểm chứng UVM” tập trung thiết kế một lõi IP UART có khả năng giao tiếp với bus AHB Lite thông qua cầu nối AHB–APB. Hệ thống được xây dựng bằng ngôn ngữ Verilog/SystemVerilog, bao gồm các khối chức năng chính như AHB Lite-APB Bridge, APB Register Interface, Baud Generator, FIFO truyền/nhận, UART Transmitter, UART Receiver và Interrupt Controller. Lõi IP hỗ trợ các tốc độ baud có thể lập trình, cấu hình độ dài dữ liệu từ 5 đến 8 bit, các chế độ parity (Không, chẵn, lẻ), lựa chọn 1 hoặc 2 stop bit và cơ chế ngắt phục vụ truyền nhận dữ liệu.

Song song với quá trình thiết kế, một môi trường kiểm chứng dựa trên phương pháp luận UVM được xây dựng nhằm tự động hóa quá trình xác minh chức năng của hệ thống. Môi trường kiểm chứng bao gồm AHB Agent, UART Agent, Scoreboard, mô hình thanh ghi UVM RAL và mô hình độ bao phủ chức năng. Nhiều testcase kiểm thử trực tiếp và kiểm thử ngẫu nhiên có ràng buộc đã được triển khai để đánh giá các chức năng của hệ thống trong nhiều điều kiện hoạt động khác nhau.

Kết quả mô phỏng trên phần mềm QuestaSim cho thấy lõi AHB Lite–UART IP hoạt động đúng theo đặc tả thiết kế. Các chức năng truy cập thanh ghi, cấu hình UART, truyền nhận dữ liệu, FIFO, cơ chế ngắt và xử lý lỗi đều được kiểm chứng thành công. Kết quả độ bao phủ chức năng đạt 100% và độ bao phủ mã nguồn đạt trên 82%, góp phần khẳng định tính đúng đắn và độ tin cậy của thiết kế cũng như môi trường kiểm chứng được xây dựng.

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	ix
DANH MỤC BẢNG.....	xiv
DANH SÁCH CÁC TỪ VIẾT TẮT	xvi
CHƯƠNG 1. GIỚI THIỆU	1
1.1 GIỚI THIỆU	1
1.2 MỤC TIÊU ĐỀ TÀI.....	2
1.3 GIỚI HẠN ĐỀ TÀI.....	2
1.4 PHƯƠNG PHÁP NGHIÊN CỨU.....	3
1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	3
1.6 BỐ CỤC ĐỒ ÁN	3
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	5
2.1 TỔNG QUAN VỀ HỆ THỐNG SOC VÀ AMBA	5
2.1.1 Giới thiệu System-on-Chip	5
2.1.2 Kiến trúc AMBA.....	5
2.2 TỔNG QUAN VỀ GIAO THỨC AHB-LITE	7
2.2.1 Giới thiệu giao thức AHB-Lite	7
2.2.2 Nguyên lý hoạt động của AHB-Lite	8
2.2.3 Các tín hiệu của giao thức AHB-Lite.....	9
2.2.4 Các loại giao dịch trên AHB-Lite	13
2.3 TỔNG QUAN VỀ GIAO THỨC APB	19
2.3.1 Giới thiệu giao thức APB.....	19
2.3.2 Các tín hiệu của giao thức APB	21
2.3.3 Các trạng thái hoạt động của giao thức APB	22

2.3.4	Quá trình ghi dữ liệu	23
2.3.5	Quá trình đọc dữ liệu	24
2.4	TỔNG QUAN VỀ GIAO THỨC UART	26
2.4.1	Giới thiệu về giao thức UART	26
2.4.2	Cấu trúc khung truyền UART	27
2.4.3	Bộ tạo tốc độ Baud và cơ chế Oversampling.....	28
2.4.4	Nguyên lý truyền dữ liệu của UART	29
2.4.5	Nguyên lý nhận dữ liệu của UART.....	30
2.5	PHƯƠNG PHÁP KIỂM CHỨNG UVM.....	30
2.5.1	Giới thiệu UVM	30
2.5.2	Kiến trúc phân lớp của UVM.....	31
2.5.3	Các thành phần chính của UVM.....	33
2.5.4	Lớp trừu tượng hóa thanh ghi	37
2.6	KIỂM CHỨNG DỰA TRÊN ĐỘ BAO PHỦ.....	39
2.6.1	Độ bao phủ mã nguồn	39
2.6.2	Độ bao phủ chức năng	40
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG		41
3.1	YÊU CẦU CỦA HỆ THỐNG.....	41
3.1.1	Yêu cầu người dùng	41
3.1.2	Yêu cầu chức năng	41
3.1.3	Yêu cầu kỹ thuật và kiến trúc.....	42
3.1.4	Yêu cầu hiệu năng.....	42
3.2	THIẾT KẾ HỆ THỐNG.....	43
3.2.1	Sơ đồ khối tổng quát	43

3.2.2 Thiết kế khối AHB Lite to APB Bridge	44
3.2.3 Thiết kế khối APB Register Interface	51
3.2.4 Thiết kế khối Baud Generator	61
3.2.5 Thiết kế khối FIFO.....	63
3.2.6 Thiết kế khối UART Transmitter	67
3.2.7 Thiết kế khối UART Receiver.....	73
3.2.8 Thiết kế khối Interrupt Controller.....	79
3.3 THIẾT KẾ MÔI TRƯỜNG KIỂM CHỨNG UVM	82
3.3.1 Kiến trúc tổng thể.....	82
3.3.2 Thiết kế AHB Agent.....	84
3.3.3 Thiết kế UART Agent	85
3.3.4 Thiết kế Scoreboard	87
3.3.5 Thiết kế mô hình độ bao phủ chức năng.....	90
3.3.6 Xây dựng kế hoạch kiểm chứng	94
CHƯƠNG 4 KẾT QUẢ THỰC HIỆN.....	96
4.1 Kết quả kiểm thử chức năng toàn hệ thống	96
4.1.1 Kiểm thử truy cập thanh ghi UART.....	96
4.1.2 Kiểm thử tốc độ baud.....	97
4.1.3 Kiểm thử cấu hình khung truyền UART.....	99
4.1.4 Kiểm thử FIFO và cơ chế ngắt.....	102
4.1.5 Kiểm thử xử lý lỗi.....	104
4.1.6 Kiểm thử ngẫu nhiên có ràng buộc	106
4.2 Kết quả độ bao phủ	107
4.2.1 Độ bao phủ mã nguồn	107

4.2.2 Độ bao phủ chức năng	107
5. Kết luận và phương hướng phát triển	109
5.1. Kết luận	109
5.2. Hướng phát triển	109
TÀI LIỆU THAM KHẢO	111

DANH MỤC HÌNH ẢNH

Hình 2.1: Hệ thống AMBA điển hình.....	6
Hình 2.2: Hệ thống AHB-Lite đơn giản.....	7
Hình 2.3: Quá trình ghi dữ liệu, không có trạng thái đợi	13
Hình 2.4: Quá trình đọc dữ liệu, không có trạng thái đợi.....	14
Hình 2.5: Quá trình ghi dữ liệu, có trạng thái đợi.....	14
Hình 2.6: Quá trình đọc dữ liệu, có trạng thái đợi.....	15
Hình 2.7: Đa giao dịch	15
Hình 2.8: Phản hồi lỗi	19
Hình 2.9: Sơ đồ khối kiến trúc kết nối APB Master và APB Slaves	20
Hình 2.10: Các trạng thái hoạt động của giao thức APB.....	22
Hình 2.11: Quá trình ghi không có trạng thái chờ	23
Hình 2.12: Quá trình ghi có trạng thái chờ	24
Hình 2.13: Quá trình đọc không có trạng thái chờ	25
Hình 2.14: Quá trình đọc có trạng thái chờ	25
Hình 2.15: Kết nối của hai bộ UART	26
Hình 2.16: Khung dữ liệu UART có 8 bit dữ liệu	27
Hình 2.17: Kiến trúc phân lớp của UVM	32
Hình 2.18: Kiến trúc tổng quát của một môi trường kiểm chứng UVM	33
Hình 2.19: Vai trò của interface trong môi trường kiểm chứng UVM	34

Hình 2.20: Cơ chế điều phối giao dịch giữa sequence, sequencer và driver	35
Hình 2.21: Quá trình chuyển đổi giao dịch thành tín hiệu điều khiển DUT của driver	35
Hình 2.22: Quá trình thu thập, chuyển đổi giao dịch của monitor ...	36
Hình 2.23: Quá trình so sánh kết quả trong scoreboard	36
Hình 2.24: Minh họa tổ chức thanh ghi của các ngoại vi trong hệ thống SoC	37
Hình 2.25: Cấu trúc tổng quát của mô hình thanh ghi.....	38
Hình 2.26: Cơ chế hoạt động của adapter và predictor	38
Hình 2.27: Phân loại các độ đo bao phủ trong kiểm chứng.....	39
Hình 3.1: Sơ đồ khối của AHB Lite – UART IP	43
Hình 3.2: Sơ đồ tổng quát khối AHB Lite to APB Bridge.....	44
Hình 3.3: Sơ đồ máy trạng thái của khối điều khiển	46
Hình 3.4: Sơ đồ mạch khối chốt địa chỉ và tín hiệu điều khiển AHB Lite	47
Hình 3.5: Sơ đồ mạch khối chốt dữ liệu đọc APB.....	48
Hình 3.6: Giảm đồ thời gian giao dịch ghi	49
Hình 3.7: Giảm đồ thời gian giao dịch đọc.....	50
Hình 3.8: Sơ đồ khối tổng quát APB Register Interface.....	51
Hình 3.9: Sơ đồ giải mã địa chỉ và truy cập thanh ghi	53
Hình 3.10: Sơ đồ mạch tạo tín hiệu sẵn sàng APB	54
Hình 3.11: Sơ đồ mạch tạo tín hiệu báo lỗi truy cập APB.....	54
Hình 3.12: Sơ đồ mạch tạo tín hiệu ghi dữ liệu vào TX FIFO	60

Hình 3.13: Sơ đồ mạch tạo tín hiệu đọc dữ liệu từ RX FIFO.....	60
Hình 3.15: Sơ đồ mạch khối phát hiện thay đổi hệ số chia	62
Hình 3.16: Sơ đồ mạch khối tạo xung baud.....	62
Hình 3.17: Xung bclk trong một chu kì bit dữ liệu.....	63
Hình 3.18: Sơ đồ tổng quát khối FIFO	64
Hình 3.19: Sơ đồ mạch khối điều khiển con trỏ đọc và ghi FIFO....	65
Hình 3.20: Cơ chế vòng đệm của FIFO.....	65
Hình 3.21: Sơ đồ mạch Khối xác định trạng thái FIFO.....	66
Hình 3.22: Sơ đồ mạch khối bộ nhớ FIFO	67
Hình 3.23: Sơ đồ tổng quát của khối UART Transmitter	68
Hình 3.24: Sơ đồ máy trạng thái của khối UART Transmitter	69
Hình 3.25: Sơ đồ mạch lựa chọn chế độ oversampling	70
Hình 3.26: Sơ đồ mạch của thanh ghi dịch dữ liệu.....	71
Hình 3.27: Sơ đồ mạch tạo parity bit	72
Hình 3.28: Sơ đồ mạch đọc TX FIFO và lưu dữ liệu truyền	72
Hình 3.29: Sơ đồ tổng quát của khối UART Receiver	73
Hình 3.30: Sơ đồ máy trạng thái của khối UART Receiver	75
Hình 3.31: Sơ đồ mạch lựa chọn chế độ oversampling	76
Hình 3.32: Sơ đồ mạch tạo tín hiệu lấy mẫu giữ bit.....	77
Hình 3.33: Sơ đồ mạch của thanh ghi lưu dữ liệu tạm thời.....	77
Hình 3.34: Sơ đồ mạch tính toán và kiểm tra parity.....	78
Hình 3.35: Sơ đồ mạch tạo tín hiệu ghi vào RX FIFO	78
Hình 3.36: Sơ đồ tổng quát khối Interrupt Controller	79

Hình 3.37: Sơ đồ mạch của khối Interrupt Controller	81
Hình 3.38: Kiến trúc tổng thể môi trường kiểm chứng AHB Lite - UART IP.....	82
Hình 3.39: Cơ chế phân phối cấu hình UART trong UVM.....	83
Hình 3.40: Cơ chế bắt tay giữa Sequence, Sequencer và Driver trong AHB Agent.....	84
Hình 3.41: Cơ chế bắt tay giữa Sequence, Sequencer và Driver trong UART Agent.....	86
Hình 3.42: Kiến trúc kiến trúc của UART Scoreboard.....	87
Hình 3.43: Lưu đồ giải thuật kiểm tra khung truyền UART.....	89
Hình 3.44: Cấu trúc tổng thể của UART_GROUP.....	91
Hình 3.45: Cơ chế ghi nhận điểm bao phủ dựa trên snapshot cấu hình thanh ghi UART	93
Hình 4.1: Waveform testcase default_value_test.....	96
Hình 4.2: Waveform rw_value_test – thanh ghi LCR (RW)	97
Hình 4.3: Waveform baud115200_16x_test	99
Hình 4.4: Waveform baud2400_16x_test	99
Hình 4.5: Waveform cấu hình 5 bit dữ liệu, parity chẵn và 1 stop bit	100
Hình 4.6: Waveform cấu hình 6 bit dữ liệu, parity lẻ và 1 stop bit	101
Hình 4.7: Waveform cấu hình 7 bit dữ liệu, không parity và 1 stop bit.	101
Hình 4.8: Waveform cấu hình 8 bit dữ liệu, không parity và 2 stop bit.	102

Hình 4.5: Waveform testcase tx_interrupt_test.....	103
Hình 4.6: Waveform testcase rx_interrupt_test	104
Hình 4.7: Waveform testcase rsvd_test.....	105
Hình 4.8: Waveform testcase parity_error_test	105
Hình 4.9: Báo cáo kiểm thử hồi quy của kiểm thử ngẫu nhiên ràng buộc	106
Hình 4.10: Kết quả độ bao phủ mã nguồn	107
Hình 4.11: Kết quả độ bao phủ chức năng.....	108

DANH MỤC BẢNG

Bảng 2.1: Mô tả các tín hiệu toàn cục.....	9
Bảng 2.2: Mô tả các tín hiệu từ thiết bị chủ.....	10
Bảng 2.3: Mô tả các tín hiệu từ thiết bị tớ	11
Bảng 2.4: Mô tả các tín hiệu từ bộ giải mã.....	12
Bảng 2.5: Mô tả các tín hiệu từ bộ ghép kênh	12
Bảng 2.6: Mã hóa tín hiệu HTRANS [1:0].....	16
Bảng 2.7: Mã hóa tín hiệu HSIZE [2:0].....	17
Bảng 2.8: Mã hóa tín hiệu HBURST [2:0]	17
Bảng 2.9: Mô tả các tín hiệu của giao thức APB.....	21
Bảng 2.10: Giá trị hệ số chia ứng với các tốc độ baud khi UART sử dụng xung clock đầu vào 100 MHz và chế độ Oversampling 16x...	28
Bảng 2.11: Giá trị hệ số chia ứng với các tốc độ baud khi UART sử dụng xung clock đầu vào 100 MHz và chế độ Oversampling 13x...	29
Bảng 3.1: Mô tả tín hiệu khối AHB Lite to APB Bridge.....	45
Bảng 3.2: Mô tả tín hiệu khối APB Register Interface	52
Bảng 3.3: Tổng hợp các thanh ghi trong khối APB Register Interface	55
Bảng 3.4: Mode Definition Register.....	56
Bảng 3.5: Divisor Latches Low	56
Bảng 3.6: Divisor Latches High.....	57
Bảng 3.7: Line Control Register	57
Bảng 3.8: Interrupt Enable Register	58

Bảng 3.9: FIFO Status Register	59
Bảng 3.10: Transmitter Buffer Register	59
Bảng 3.11: Receiver Buffer Register	60
Bảng 3.12: Mô tả tín hiệu khối Baud Generator.....	61
Bảng 3.13: Mô tả tín hiệu khối FIFO.....	64
Bảng 3.14: Mô tả tín hiệu khối UART Transmitter	68
Bảng 3.15: Mô tả tín hiệu khối UART Receiver	74
Bảng 3.16: Mô tả tín hiệu khối Interrupt Controller.....	80
Bảng 3.17: Tổng hợp toàn bộ các điểm bao phủ và điểm bao phủ giao chéo trong mô hình.	92
Bảng 3.18: Tổng hợp các nhóm kiểm tra trong kế hoạch kiểm chứng	94
Bảng 4.1: Kết quả kiểm tra tốc độ baud	98
Bảng 4.2: Kết quả kiểm thử cấu hình khung truyền UART	102

DANH SÁCH CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Giải nghĩa
AHB	Advanced High-performance Bus	Bus hiệu năng cao tiên tiến
AHB Lite	Advanced High-performance Bus Lite	Bus hiệu năng cao tiên tiến phiên bản đơn giản
AMBA	Advanced Microcontroller Bus Architecture	Kiến trúc bus vi điều khiển tiên tiến
APB	Advanced Peripheral Bus	Bus ngoại vi tiên tiến
AXI	Advanced eXtensible Interface	Giao diện mở rộng tiên tiến
CPU	Central Processing Unit	Bộ xử lý trung tâm
DLL	Divisor Latch Low	Thanh ghi chốt hệ số chia thấp
DLH	Divisor Latch High	Thanh ghi chốt hệ số chia cao
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp
DUT	Design Under Test	Thiết kế cần kiểm chứng
FIFO	First In First Out	Hàng đợi vào trước ra trước
FPGA	Field-Programmable Gate Array	Mảng cổng logic khả lập trình
IER	Interrupt Enable Register	Thanh ghi cho phép ngắt
IP	Intellectual Property	Lỗi sở hữu trí tuệ
LCR	Line Control Register	Thanh ghi điều khiển đường truyền
MDR	Mode Definition Register	Thanh ghi định nghĩa chế độ

Từ viết tắt	Tên đầy đủ	Giải nghĩa
RTL	Register Transfer Level	Mức truyền thanh ghi
RX	Receiver	Bộ nhận dữ liệu
SoC	System on Chip	Hệ thống trên chip
TLM	Transaction Level Modeling	Mô hình hóa mức giao dịch
TX	Transmitter	Bộ truyền dữ liệu
UART	Universal Asynchronous Receiver-Transmitter	Bộ thu phát bất đồng bộ đa năng
UVM	Universal Verification Methodology	Phương pháp kiểm chứng toàn diện
UVM RAL	UVM Register Abstraction Layer	Lớp trừu tượng hóa thanh ghi UVM

CHƯƠNG 1. GIỚI THIỆU

1.1 GIỚI THIỆU

Trong kỷ nguyên công nghệ vi mạch và bán dẫn hiện nay, xu hướng tích hợp toàn bộ một hệ thống phức tạp lên một chip đơn duy nhất (SoC - System on Chip) đã trở thành giải pháp cốt lõi nhằm tối ưu hóa hiệu năng, giảm kích thước vật lý và tiết kiệm năng lượng cho các thiết bị điện tử [1]. Trong một kiến trúc SoC, việc tổ chức giao tiếp dữ liệu hiệu quả giữa bộ vi xử lý trung tâm tốc độ cao và các khối ngoại vi tốc độ thấp là một trong những bài toán thiết kế quan trọng nhất [1]. Nếu kết nối trực tiếp các thành phần này mà không thông qua cơ chế phản hồi và chuyển đổi giao thức phù hợp, hệ thống sẽ gặp hiện tượng nghẽn mạch, gây giảm đáng kể hiệu suất tổng thể [1].

Để giải quyết thách thức phân tầng băng thông này, kiến trúc bus AMBA (Advanced Microcontroller Bus Architecture) do tập đoàn ARM phát triển đã trở thành tiêu chuẩn công nghiệp được ứng dụng rộng rãi nhất [1, 2]. Trong hệ thống AMBA, bus AHB Lite đóng vai trò là trục xương sống song song, tốc độ cao, hỗ trợ các tính năng tiên tiến như giao dịch đường ống và độ rộng dữ liệu lớn, chuyên dùng kết nối các thành phần chủ đạo như CPU với các thành phần phụ thuộc [1, 2]. Tuy nhiên, để SoC tương tác với thế giới bên ngoài cho các tác vụ như truyền dữ liệu, nạp chương trình hay cấu hình hệ thống, hệ thống luôn cần một giao tiếp nối tiếp đơn giản và tin cậy. Giao thức truyền nhận bất đồng bộ UART (Universal Asynchronous Receiver-Transmitter) chính là giải pháp kinh điển đáp ứng hoàn hảo yêu cầu này nhờ ưu điểm cấu trúc phần cứng tối giản, chỉ sử dụng hai đường truyền dữ liệu độc lập (TX và RX), giúp tiết kiệm tài nguyên mạch và công suất tiêu thụ [2, 3].

Do có sự khác biệt rất lớn về mặt tốc độ, độ rộng dữ liệu và phương thức truyền thông giữa trục bus hệ thống AHB Lite mang đặc tính song song, đồng bộ, tốc độ cao và khối ngoại vi UART hoạt động theo phương thức nối tiếp, bất đồng bộ ở tốc độ thấp, việc nghiên cứu và thiết kế một lõi AHB Lite – UART IP là vô cùng thiết yếu trong cấu trúc SoC [3, 4].

Bên cạnh bài toán thiết kế phần cứng, công tác kiểm chứng chức năng đang ngày càng trở thành thách thức cốt lõi, thường chiếm phần lớn trong tổng thời gian và nguồn lực của một chu trình thiết kế chip Front-end [4]. Các phương pháp viết testbench truyền thống hướng kịch bản cố định không còn đủ khả năng bao quát hết các điều kiện biên và các lỗi tiềm ẩn khi quy mô thiết kế tăng lên [4]. Vì lý do đó, việc ứng dụng Phương pháp kiểm chứng toàn diện UVM (Universal Verification Methodology) dựa trên nền tảng ngôn ngữ SystemVerilog – với các tính năng vượt trội như tạo kịch bản ngẫu nhiên có ràng buộc và quản lý bằng độ bao phủ là giải pháp tối ưu nhất hiện nay nhằm chuẩn hóa chất lượng thiết kế, đảm bảo thiết kế đạt độ tin cậy tuyệt đối trước khi tiến hành chế tạo [4, 5].

Xuất phát từ những nhu cầu thực tiễn của ngành công nghiệp vi mạch số, đề tài **“THIẾT KẾ AHB LITE-UART IP VÀ XÂY DỰNG MÔI TRƯỜNG KIỂM CHỨNG UVM”** đã được lựa chọn nghiên cứu và thực hiện.

1.2 MỤC TIÊU ĐỀ TÀI

Mục tiêu của đề tài tập trung vào thiết kế lõi AHB Lite-UART IP và xây dựng môi trường kiểm chứng UVM bao gồm 2 nhiệm vụ chính sau:

- Thiết kế lõi AHB Lite – UART IP với chức năng hoàn chỉnh.
- Kiểm tra và đánh giá chức năng của lõi thiết kế thông qua mô phỏng bằng môi trường kiểm chứng UVM.

1.3 GIỚI HẠN ĐỀ TÀI

Đề tài được thực hiện bằng cách xây dựng lõi AHB Lite-UART IP và môi trường kiểm chứng UVM bằng ngôn ngữ Verilog/SystemVerilog. Các file trong đề tài được biên dịch và quản lý trên phần mềm QuestaSim. Lõi IP được mô phỏng kiểm tra chức năng bằng môi trường UVM thông qua phần mềm QuestaSim.

Hệ thống hỗ trợ giao thức AHB Lite với kiểu truyền dữ liệu đơn (Single Transfer) và kích thước dữ liệu Word (32 bit). Các tính năng mở rộng nâng cao như truyền theo cụm (Burst Transfer), giao dịch khóa bus (Locked Transfer) và cơ chế bảo vệ truy cập (Protection Control) không được hỗ trợ.

Thiết kế tích hợp bộ tạo tốc độ baud có thể lập trình được, hỗ trợ ngắt phần cứng, và cấu trúc hàng đợi đệm FIFO truyền nhận song công toàn phần hoàn toàn tách biệt nhau với dung lượng 16 byte cho cả FIFO truyền và FIFO nhận. Hệ thống hỗ trợ độ dài dữ liệu 5, 6, 7 hoặc 8 bit; hỗ trợ các chế độ kiểm tra chẵn lẻ (parity bit) bao gồm chẵn (even), lẻ (odd) hoặc không sử dụng bit parity; đồng thời cho phép lựa chọn sử dụng 1 hoặc 2 stop bit tùy theo yêu cầu của ứng dụng.

Đồ án tập trung hoàn toàn vào giai đoạn Front-end bao gồm mô hình hóa RTL và kiểm chứng chức năng luận lý; không bao gồm tổng hợp mạch vật lý hay thực thi kiểm thử trên kit FPGA thực tế.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài sử dụng các phương pháp nghiên cứu như sau:

- Phương pháp phân tích và tổng hợp.
- Phương pháp phân loại và hệ thống.
- Phương pháp mô phỏng.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài này là lõi AHB Lite-UART IP và môi trường kiểm tra UVM.

Phạm vi nghiên cứu về các giao thức AHB, giao thức APB, giao thức UART, các khối trong cầu chuyển đổi AHB Lite sang APB, các khối UART và các thành phần trong môi trường kiểm tra UVM.

1.6 BỐ CỤC ĐỒ ÁN

Bố cục sẽ được nhóm chia thành 5 chương để cho độc giả có thể hình dung một cách cụ thể những gì chúng tôi sẽ làm trong từng chương:

- Chương 1. GIỚI THIỆU: Trong chương này, chúng tôi chủ yếu giới thiệu một cách khái quát về đề tài và làm rõ, phân tích các vấn đề mà đề tài này gặp phải, đưa ra các giới hạn đề tài riêng và đặt ra các mục tiêu cụ thể để có thể hoàn thiện đề tài một cách hiệu quả.

- Chương 2. CƠ SỞ LÝ THUYẾT: Trong chương 2, chúng tôi làm rõ các khái niệm cơ bản của đề tài, những lý thuyết về giao thức AHB-Lite, giao thức APB, giao thức UART và môi trường kiểm tra UVM.
- Chương 3. THIẾT KẾ HỆ THỐNG: Trong chương 3, nhóm tập trung phác thảo, vẽ lại hệ thống và phân tích các khối cần thiết của lõi AHB Lite-UART IP cùng môi trường kiểm tra UVM.
- Chương 4. KẾT QUẢ: Trong chương này, chúng tôi sẽ trình bày chi tiết các trường hợp kiểm tra được lựa chọn, chạy mô phỏng và phân tích từng trường hợp nhằm giúp người đọc hiểu về quá trình hoạt động của thiết kế.
- Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN: Trong chương này, chúng tôi sẽ tổng kết lại toàn bộ quá trình nghiên cứu và phát triển hệ thống. Đồng thời nhóm cũng sẽ nhận xét về các yêu cầu đạt được, đưa ra các hạn chế và phương hướng phát triển của đề tài.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ HỆ THỐNG SOC VÀ AMBA

2.1.1 Giới thiệu System-on-Chip

Hệ thống trên chip (System on Chip - SoC) là vi mạch tích hợp toàn diện các thành phần cốt lõi của một hệ thống điện tử trên một nền silicon duy nhất. Sự ra đời của SoC là xu hướng chuyển dịch tất yếu nhằm đáp ứng yêu cầu thu nhỏ kích thước vật lý, tối ưu hóa diện tích và hiệu suất so với các thiết kế bo mạch sử dụng vi mạch rời rạc truyền thống [1].

Một hệ thống SoC điển hình bao gồm bộ vi xử lý (như CPU, DSP), bộ nhớ, bộ điều khiển DMA, các khối giao tiếp ngoại vi (như USB, UART, I2C, PCIe) và mạng lưới bus liên kết nội bộ. Việc tích hợp cao độ này mang lại nhiều ưu điểm vượt trội: giảm đáng kể công suất tiêu thụ, hạ chi phí sản xuất trên quy mô lớn, thu nhỏ kích thước thiết bị và tăng cường độ tin cậy của toàn hệ thống nhờ giảm thiểu các kết nối vật lý bên ngoài. Nhờ những lợi thế đó, SoC đóng vai trò là nền tảng cốt lõi trong các hệ thống nhúng, thiết bị thông minh và bộ vi điều khiển hiện đại [1].

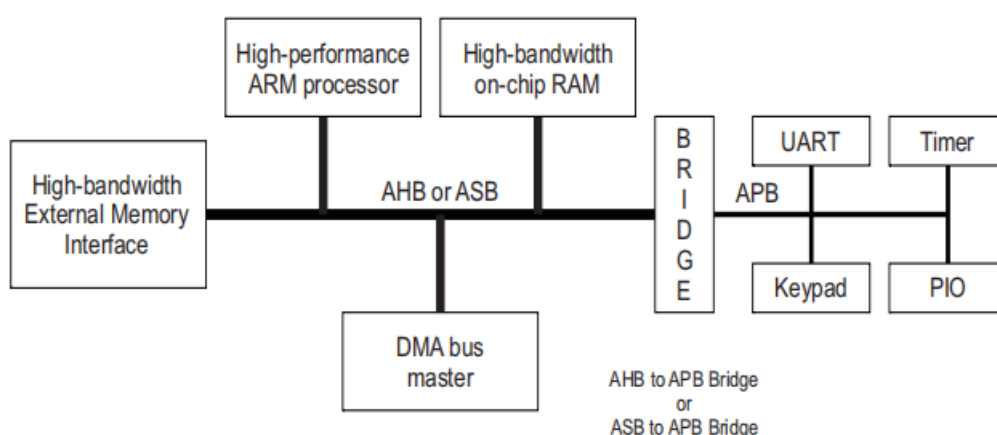
Để rút ngắn thời gian phát triển các hệ thống SoC phức tạp, quy trình thiết kế hiện đại ưu tiên sử dụng các lõi sở hữu trí tuệ (Intellectual Property - IP Core). Đây là các khối mạch logic chức năng đã được tối ưu hóa và xác minh sẵn, giúp đẩy nhanh tiến độ đưa sản phẩm ra thị trường và tăng khả năng tái sử dụng [2]. Tuy nhiên, sự đa dạng của các khối lõi IP từ nhiều nhà cung cấp đòi hỏi một phương thức kết nối chuẩn hóa và hiệu quả để đảm bảo chúng có thể giao tiếp đồng bộ và trơn tru trong cùng một hệ thống SoC [1].

2.1.2 Kiến trúc AMBA

Kiến trúc bus vi điều khiển tiên tiến (AMBA) do ARM phát triển là tiêu chuẩn giao tiếp mở trên chip, giúp kết nối và quản lý các khối IP trong hệ thống SoC. Việc chuẩn hóa này đảm bảo tính độc lập công nghệ, cho phép tái sử dụng IP

đễ dàng trên nhiều quy trình bán dẫn khác nhau, từ đó rút ngắn đáng kể thời gian thiết kế các hệ thống phức tạp [2, 14].

Để đáp ứng đa dạng yêu cầu về băng thông, AMBA cung cấp nhiều giao thức thường xuyên được ứng dụng như giao thức AXI mang lại hiệu năng cực cao với nhiều kênh truyền độc lập, giao thức AHB là bus hệ thống tốc độ cao, hỗ trợ đa chủ, truyền cụm và cơ chế đường ống [2, 12, 14]. Nhằm tối ưu diện tích, giao thức AHB Lite và giao thức AXI Lite được dùng cho các hệ thống đơn chủ bằng cách lược bỏ mạch phân xử. Ngược lại, giao thức APB có giao diện đơn giản, không dùng đường ống, nhằm tối ưu diện tích cho các thiết bị ngoại vi [13, 14].



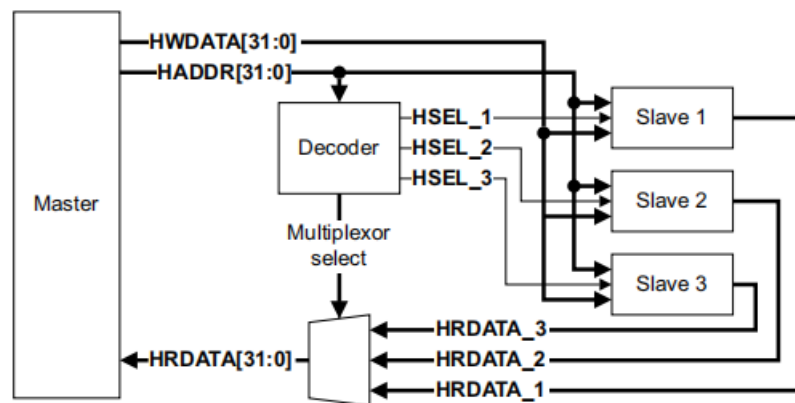
Hình 2.1: Hệ thống AMBA điển hình

Để tối ưu hóa hiệu suất, hệ thống SoC thường áp dụng mô hình phân cấp bus như được minh họa trong Hình 2.1. Cụ thể, các giao thức tốc độ cao (như AHB, AXI) đóng vai trò là trục xương sống kết nối trực tiếp bộ vi xử lý (CPU), bộ nhớ và khối DMA. Ngược lại, các ngoại vi tốc độ thấp (như UART, Timer, Keypad) được phân bổ sang phân cấp bus thứ cấp APB nhằm tránh lãng phí băng thông của bus hệ thống [1, 14]. Quá trình giao tiếp và chuyển đổi tín hiệu giữa hai phân cấp này được thực hiện liên mạch thông qua một khối cầu nối, giúp hệ thống cân bằng giữa hiệu năng và năng lượng tiêu thụ [1].

2.2 TỔNG QUAN VỀ GIAO THỨC AHB-LITE

2.2.1 Giới thiệu giao thức AHB-Lite

Giao thức AHB-Lite (Advanced High-performance Bus - Lite) là một tiêu chuẩn giao tiếp nội bộ tốc độ cao thuộc họ kiến trúc vi điều khiển tiên tiến AMBA do tập đoàn ARM phát triển và ban hành [12]. Trong hệ sinh thái AMBA, AHB-Lite đóng vai trò là một trục bus xương sống, cung cấp giao diện kết nối hiệu suất cao và băng thông lớn để đáp ứng các yêu cầu khắt khe của những hệ thống vi mạch tổng hợp hiện đại [12]. Tiêu chuẩn này được thiết kế để tối ưu hóa việc liên kết giữa các thành phần cốt lõi đòi hỏi tốc độ truy xuất dữ liệu cực nhanh, chẳng hạn như bộ vi xử lý trung tâm, các khối bộ nhớ nội bộ, và những thiết bị ngoại vi có băng thông rộng [12]. Điểm khác biệt và cũng là ưu điểm nổi bật nhất của AHB-Lite so với chuẩn AHB đầy đủ nằm ở thiết kế hướng tới các hệ thống chỉ sử dụng một thiết bị chủ duy nhất [12]. Bằng cách loại bỏ hoàn toàn sự phức tạp của các mạch phân xử đa chủ và các giao dịch phân tách, AHB-Lite mang lại một giải pháp giao tiếp tinh gọn, giúp giảm thiểu đáng kể diện tích silicon và mức tiêu thụ điện năng trong khi vẫn duy trì được hiệu suất truyền tải dữ liệu ở mức tối đa.



Hình 2.2: Hệ thống AHB-Lite đơn giản

Để vận hành một cách đồng bộ và trơn tru, kiến trúc tổng thể của một hệ thống AHB-Lite tiêu chuẩn được cấu thành từ bốn thành phần chức năng chính: Thiết bị chủ (Master), Thiết bị tớ (Slave), Bộ giải mã (Decoder) và Bộ ghép kênh (Multiplexor) [12]. Trong kiến trúc này, Thiết bị chủ là thành phần chủ động duy nhất, giữ vai trò khởi tạo và quản lý mọi giao dịch bằng cách phát ra các nhóm tín

hiệu địa chỉ, điều khiển và dữ liệu ghi mà không cần thông qua mạch phân xử phức tạp, giúp hệ thống vận hành với độ trễ thấp nhất [12]. Để khép kín chu trình giao tiếp, hệ thống sử dụng các Thiết bị tổ hoạt động ở trạng thái hoàn toàn thụ động [12]. Nhiệm vụ của thiết bị tổ là tiếp nhận các yêu cầu truy cập từ thiết bị chủ để thực hiện thao tác đọc/ghi vào không gian bộ nhớ nội bộ, đồng thời sử dụng nhóm tín hiệu phản hồi để báo cáo trạng thái hoàn thành hoặc thông báo lỗi giao dịch [12].

Sự tương tác giữa thiết bị chủ và hàng loạt thiết bị tổ không diễn ra trực tiếp mà được điều phối chặt chẽ thông qua cơ chế định tuyến thông minh của bộ giải mã và bộ ghép kênh. Bộ giải mã đóng vai trò là khối logic trung tâm thực hiện phân tích giá trị địa chỉ từ thiết bị chủ để kích hoạt tín hiệu chọn cho đúng một thiết bị tổ mục tiêu, qua đó loại trừ hoàn toàn rủi ro xung đột điều khiển trên bus [12]. Sau khi thiết bị tổ mục tiêu hoàn tất việc xử lý, Bộ ghép kênh sẽ đóng vai trò thay thế cho kiến trúc bus ba trạng thái truyền thống [12]. Nó sử dụng chính tín hiệu chọn từ bộ giải mã để thu thập và định tuyến duy nhất luồng dữ liệu đọc cùng các tín hiệu trạng thái từ thiết bị tổ đó quay ngược trở lại cho thiết bị chủ [12]. Sự phối hợp nhịp nhàng và liên tục giữa bốn thành phần này tạo nên một nền tảng truyền tải khép kín, an toàn và đạt thông lượng tối đa cho hệ thống SoC.

2.2.2 Nguyên lý hoạt động của AHB-Lite

Hệ thống AHB-Lite vận hành theo mô hình thiết bị chủ - thiết bị tổ tập trung, với duy nhất một thiết bị chủ nắm quyền khởi xướng giao dịch thông qua việc phát ra các tín hiệu địa chỉ và điều khiển [12]. Ngược lại, thiết bị tổ hoạt động thụ động, tiếp nhận yêu cầu để thực thi thao tác đọc/ghi vào bộ nhớ và gửi các tín hiệu phản hồi trạng thái ngược lại cho thiết bị chủ [12].

Để tối đa hóa thông lượng dữ liệu ở tần số xung nhịp cao, AHB-Lite sử dụng cơ chế đường ống (pipeline) chia mỗi giao dịch thành hai giai đoạn đan xen:

- Pha địa chỉ (Address Phase): Kéo dài đúng một chu kỳ xung nhịp, thiết bị chủ xuất thông tin địa chỉ và cấu hình giao dịch [12].
- Pha dữ liệu (Data Phase): Kéo dài một hoặc nhiều chu kỳ xung nhịp, dùng để luân chuyển dòng dữ liệu thực tế [12].

Đặc điểm cốt lõi của cơ chế đường ống này là pha địa chỉ của giao dịch hiện tại luôn được thực thi đồng thời với pha dữ liệu của giao dịch ngay trước đó, giúp hệ thống loại bỏ hoàn toàn các khoảng thời gian chết trên bus [12].

Sự đồng bộ tốc độ xử lý giữa thiết bị chủ và thiết bị tớ được quản lý linh hoạt qua tín hiệu HREADY [12]. Khi chưa sẵn sàng đáp ứng dữ liệu, thiết bị tớ sẽ kéo tín hiệu HREADY xuống mức logic thấp nhằm chèn thêm các trạng thái chờ vào pha dữ liệu [12]. Động thái này buộc thiết bị chủ phải duy trì ổn định các tín hiệu trên bus cho đến khi thiết bị tớ xử lý xong và đưa HREADY lên mức cao trở lại để chính thức kết thúc giao dịch [12].

2.2.3 Các tín hiệu của giao thức AHB-Lite

Các tín hiệu trong giao thức AHB Lite đảm nhiệm chức năng truyền địa chỉ, dữ liệu, điều khiển và phản hồi giữa các thành phần trong hệ thống. Để thuận tiện cho việc mô tả, các tín hiệu được phân thành năm nhóm chính gồm: tín hiệu toàn cục, tín hiệu từ thiết bị chủ, tín hiệu từ thiết bị tớ, tín hiệu từ bộ giải mã và tín hiệu từ bộ ghép kênh.

2.2.3.1 Các tín hiệu toàn cục

Các tín hiệu toàn cục bao gồm tín hiệu xung nhịp và tín hiệu đặt lại, dùng để đồng bộ hoạt động và khởi tạo trạng thái của hệ thống bus.

Bảng 2.1: Mô tả các tín hiệu toàn cục

Tên tín hiệu	Nguồn	Mô tả
HCLK	Nguồn tạo xung	Xung nhịp hệ thống, cung cấp xung nhịp đồng bộ để định thời cho mọi hoạt động truyền tải. Tất cả các thao tác lấy mẫu tín hiệu đều được tham chiếu với cạnh lên của HCLK.
HRESETn	Bộ điều khiển Reset	Tín hiệu thiết lập lại, đưa toàn hệ thống và bus về trạng thái khởi tạo ban đầu. Đây là tín hiệu duy nhất trong giao thức AHB Lite hoạt động ở mức logic thấp.

2.2.3.2 Các tín hiệu từ thiết bị chủ

Nhóm tín hiệu này do thiết bị chủ khởi tạo nhằm mục đích phát ra yêu cầu truy cập, cung cấp thông tin điều khiển và đẩy dữ liệu ghi lên hệ thống.

Bảng 2.2: Mô tả các tín hiệu từ thiết bị chủ

Tên tín hiệu	Đích đến	Mô tả
HADDR [31:0]	Thiết bị tớ và Bộ giải mã	Bus địa chỉ, bus địa chỉ hệ thống có độ rộng 32-bit.
HBURST [2:0]	Thiết bị tớ	Kiểu truyền cụm, xác định giao dịch là truyền đơn lẻ hay là một phần của chuỗi truyền theo cụm. Hỗ trợ các cụm độ dài cố định 4, 8 hay 16 nhịp. Cụm truyền có thể là cụm truyền tăng dần địa chỉ hoặc cụm truyền xoay vòng địa chỉ hoặc cụm tăng dần không xác định chiều dài.
HMASTLOCK	Thiết bị tớ	Khóa giao dịch, khi ở mức cao sẽ báo hiệu cụm giao dịch hiện tại không thể bị chia cắt. Có cùng định thời với bus địa chỉ và tín hiệu điều khiển.
HPROT [3:0]	Thiết bị tớ	Điều khiển bảo vệ, cung cấp thông tin phụ trợ về mức độ bảo vệ của truy cập: lấy lệnh hay dữ liệu, truy cập ở chế độ đặc quyền hay người dùng, có thể lưu đệm hay lưu cache hay không.
HSIZE [2:0]	Thiết bị tớ	Kích thước giao dịch, chỉ định kích thước của gói dữ liệu trong một nhịp truyền (Byte, Halfword, Word...). Tín hiệu này bắt buộc

		phải giữ nguyên không đổi trong toàn bộ quá trình truyền theo cụm.
HTRANS [1:0]	Thiết bị tớ	Loại giao dịch, cho biết trạng thái của giao dịch hiện hành, bao gồm: IDLE, BUSY, NONSEQ, SEQ.
HWDATA [31:0]	Thiết bị tớ	Bus dữ liệu ghi, truyền dữ liệu từ thiết bị chủ đến các thiết bị tớ trong quá trình thực hiện thao tác ghi.
HWRITE	Thiết bị tớ	Hướng truyền dữ liệu, mức cao biểu thị thao tác ghi, mức thấp biểu thị thao tác đọc. Tín hiệu này phải giữ nguyên không thay đổi trong suốt một chuỗi truyền theo cụm.

2.2.3.3 Các tín hiệu từ thiết bị tớ

Nhóm tín hiệu này do các Slave độc lập phát ra để cung cấp dữ liệu đọc và phản hồi trạng thái hoàn tất, báo lỗi của giao dịch.

Bảng 2.3: Mô tả các tín hiệu từ thiết bị tớ

Tên tín hiệu	Đích đến	Mô tả
HRDATA [31:0]	Bộ ghép kênh	Bus dữ liệu đọc, trong các giao dịch đọc, bus này truyền dữ liệu từ thiết bị tớ đang được chọn về bộ ghép kênh.
HREADYOUT	Bộ ghép kênh	Báo hoàn thành giao dịch, khi ở mức cao, tín hiệu báo hiệu giao dịch hiện tại trên bus đã kết thúc. Thiết bị tớ có thể kéo tín hiệu này xuống mức thấp để chèn trạng thái chờ nhằm kéo dài chu kỳ dữ liệu.
HRESP	Bộ ghép kênh	Phản hồi trạng thái, cung cấp trạng thái thành công hay thất bại của giao dịch. Mức thấp báo

		hiệu giao dịch thành công (OKAY). Mức cao báo hiệu có lỗi xảy ra (ERROR).
--	--	---

2.2.3.4 Các tín hiệu từ bộ giải mã

Bộ giải mã tập trung chịu trách nhiệm cung cấp tín hiệu kích hoạt cho thiết bị tớ dựa trên địa chỉ phát ra từ Master.

Bảng 2.4: Mô tả các tín hiệu từ bộ giải mã

Tên tín hiệu	Đích đến	Mô tả
HSELx	Thiết bị tớ	Tín hiệu chọn thiết bị tớ, là kết quả giải mã tổ hợp từ bus địa chỉ HADDR. Báo cho thiết bị tớ biết nó đang là mục tiêu của giao dịch. Khi được chọn, thiết bị tớ phải giám sát trạng thái HREADY để đảm bảo giao dịch trước đó đã hoàn tất rồi mới đưa ra đáp ứng.

2.2.3.5 Các tín hiệu từ bộ ghép kênh

Bộ ghép kênh thu thập thông tin trả về từ tất cả các thiết bị tớ và định tuyến duy nhất luồng tín hiệu của Slave đang được chọn để gửi về lại cho Master.

Bảng 2.5: Mô tả các tín hiệu từ bộ ghép kênh

Tên tín hiệu	Đích đến	Mô tả
HRDATA [31:0]	Thiết bị chủ	Bus dữ liệu đọc, luồng dữ liệu đọc đã được bộ ghép kênh lựa chọn từ thiết bị tớ đang hoạt động để gửi về cho thiết bị chủ.
HREADY	Thiết bị chủ và Thiết bị tớ	Tín hiệu đồng bộ sẵn sàng, khi ở mức cao, tín hiệu này thông báo cho thiết bị chủ và toàn bộ các thiết bị tớ trong hệ thống biết rằng giao dịch của chu kỳ trước đó đã hoàn tất.
HRESP	Thiết bị chủ	Phản hồi trạng thái, trạng thái phản hồi giao dịch (OKAY hoặc ERROR) đã được bộ ghép

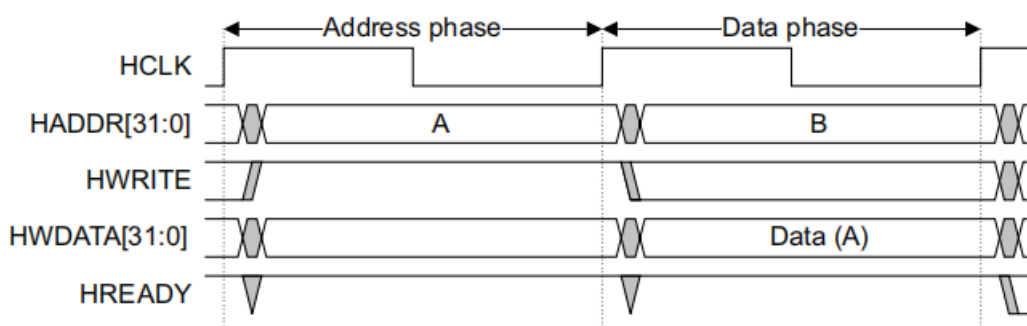
		kênh lựa chọn từ thiết bị tớ để trả về cho thiết bị chủ.
--	--	--

2.2.4 Các loại giao dịch trên AHB-Lite

2.2.4.1 Quá trình đọc và ghi dữ liệu cơ bản

Một giao dịch cơ bản trên bus AHB-Lite luôn trải qua hai giai đoạn phân biệt: giai đoạn địa chỉ (kéo dài đúng một chu kỳ để truyền HADDR, HWRITE, HSIZE, HTRANS) và giai đoạn truy cập dữ liệu (có thể kéo dài một hoặc nhiều chu kỳ để truyền HWDATA/HRDATA) [12]. Trong điều kiện lý tưởng, quá trình đọc và ghi không đợi sẽ hoàn tất gọn gàng trong đúng hai chu kỳ xung nhịp [12].

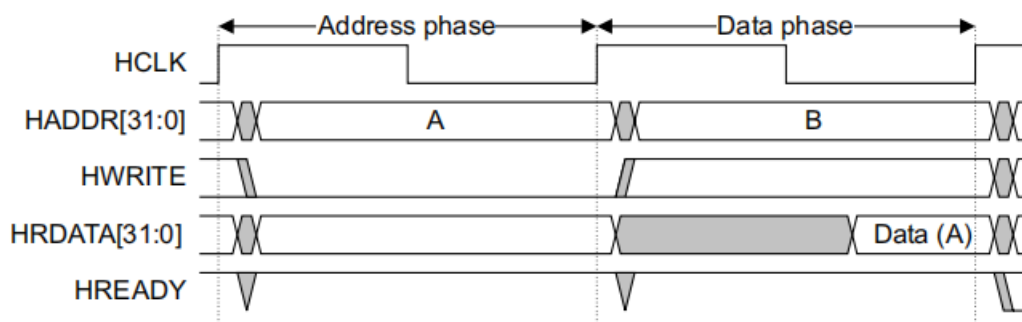
Tuy nhiên, thiết bị tớ có quyền chèn thêm các trạng thái đợi bằng cách kéo tín hiệu HREADY xuống mức logic thấp để yêu cầu thêm thời gian xử lý [12]. Đặc trưng cốt lõi của AHB-Lite là cơ chế đường ống, cho phép giai đoạn địa chỉ của giao dịch nối tiếp được thực thi đồng thời với giai đoạn dữ liệu của giao dịch hiện tại, giúp hệ thống loại bỏ thời gian chết trên bus và đạt thông lượng tối đa [12].



Hình 2.3: Quá trình ghi dữ liệu, không có trạng thái đợi

Như được minh họa trong hình 2.3, quá trình ghi dữ liệu không có trạng thái đợi được thực thi qua các nhịp xung đồng hồ liên tiếp. Tại cạnh lên đầu tiên của HCLK, thiết bị chủ xuất địa chỉ và tín hiệu điều khiển (HWRITE=1). Tại cạnh lên thứ hai, thiết bị tớ bắt đầu lấy mẫu thông tin điều khiển, và ngay lúc này thiết bị chủ cũng đẩy dữ liệu cần ghi lên đường truyền HWDATA. Giao dịch kết thúc khi

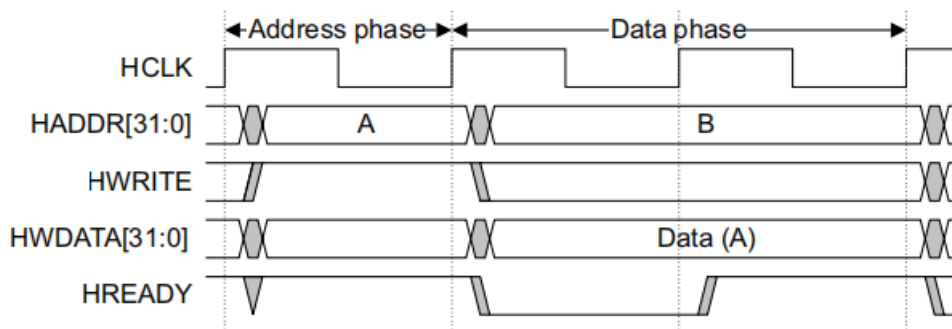
thiết bị tớ ghi nhận dữ liệu và phản hồi tín hiệu HREADY ở mức cao, được hệ thống chốt tại cạnh lên thứ ba của HCLK.



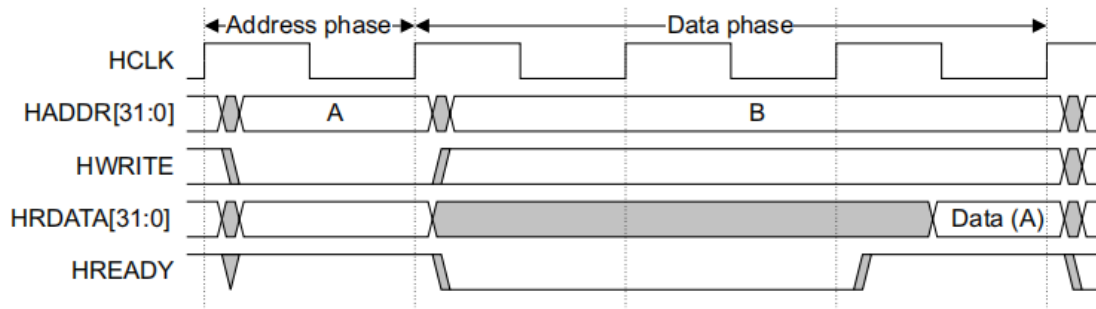
Hình 2.4: Quá trình đọc dữ liệu, không có trạng thái đợi

Tương tự như thao tác ghi, hình 2.4 minh họa quá trình đọc dữ liệu không có trạng thái đợi, trong đó các bước cũng được thực thi liên tục qua các nhịp xung đồng hồ liên tiếp. Tại cạnh lên đầu tiên của xung nhịp HCLK, thiết bị chủ bắt đầu gửi địa chỉ (HADDR) và tín hiệu điều khiển (HWRITE = 0) lên bus. Tại cạnh lên thứ hai của HCLK, thiết bị tớ tiến hành lấy mẫu các thông tin địa chỉ và điều khiển này. Sau một khoảng thời gian xử lý, thiết bị tớ xuất dữ liệu lên bus HRDATA đồng thời đưa tín hiệu sẵn sàng HREADY lên mức cao. Cuối cùng, thiết bị chủ lấy mẫu dữ liệu và tín hiệu HREADY tại cạnh lên thứ ba của HCLK, qua đó xác nhận hoàn thành giao dịch.

Ngoài ra, thiết bị tớ có quyền chèn thêm các trạng thái đợi vào bất kỳ giao dịch nào để mở rộng chu kỳ truy cập dữ liệu khi nó cần thêm thời gian xử lý.

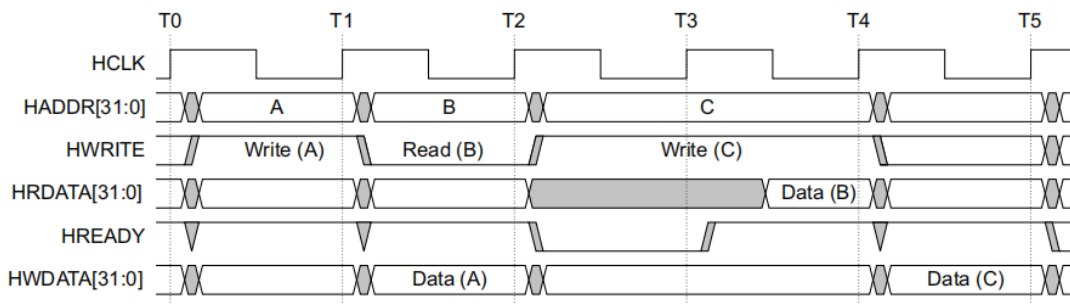


Hình 2.5: Quá trình ghi dữ liệu, có trạng thái đợi



Hình 2.6: Quá trình đọc dữ liệu, có trạng thái đợi

Đối với các giao dịch có trạng thái đợi được minh họa trong hình 2.5 và hình 2.6, đặc điểm khác biệt lớn nhất của quá trình truyền tải diễn ra ở giai đoạn dữ liệu. Khi thiết bị tớ kéo tín hiệu HREADY xuống mức logic thấp, chu kỳ dữ liệu sẽ bị kéo dài. Đối với yêu cầu ghi, thiết bị chủ bắt buộc phải giữ cho dữ liệu trên bus HWDATA luôn ổn định trong suốt các chu kỳ bị kéo dài này. Đối với yêu cầu đọc, dữ liệu trên HRDATA có thể chưa hợp lệ trong các chu kỳ HREADY ở mức thấp, thiết bị tớ chỉ cần đảm bảo cung cấp dữ liệu chính xác khi nó đưa HREADY lên mức cao để kết thúc giao dịch.



Hình 2.7: Đa giao dịch

Quan sát giản đồ xung đa giao dịch ở hình 2.7, khi một giao dịch bị kéo dài bởi trạng thái đợi, nó sẽ tạo ra hiệu ứng phụ là làm kéo dài giai đoạn địa chỉ của giao dịch tiếp theo trên bus. Trình tự vận hành đa giao dịch này cho thấy rõ giai đoạn địa chỉ của bất kỳ giao dịch nào cũng được thực thi đồng thời với giai đoạn dữ liệu của giao dịch ngay trước đó. Sự gộp đầu liên tục giữa thông tin địa chỉ và dữ liệu chính là đặc điểm cốt lõi thể hiện bản chất đường ống của bus AHB Lite,

giúp hệ thống đạt được thông lượng hoạt động tối đa trong khi vẫn đảm bảo thời gian cho thiết bị xử lý.

2.2.4.2 Các loại giao dịch

Hệ thống AHB-Lite sử dụng tín hiệu HTRANS [1:0] để phân loại giao dịch thành bốn định dạng, giúp thiết bị nhận biết trạng thái hiện hành của luồng dữ liệu [12].

- IDLE: Báo hiệu thiết bị chủ không có nhu cầu truyền tải; thiết bị tớ bắt buộc bỏ qua và phản hồi hoàn tất ngay lập tức [12].
- BUSY: Được thiết bị chủ sử dụng để chèn chu kỳ nghỉ giữa một chuỗi truyền theo cụm (burst) [12].
- NONSEQ: Đại diện cho một giao dịch đơn lẻ hoặc nhịp truyền khởi đầu của một chuỗi burst mới [12].
- SEQ: Đại diện cho các nhịp truyền dữ liệu tiếp nối trong một chuỗi burst, với địa chỉ mang tính liên tục [12].

Bảng 2.6: Mã hóa tín hiệu HTRANS [1:0]

Giá trị HTRANS [1:0]	Loại giao dịch	Mô tả chức năng
2'b00	IDLE	Báo hiệu thiết bị chủ không có yêu cầu truyền dữ liệu, bus đang rảnh.
2'b01	BUSY	Thiết bị chủ đang giữ bus nhưng tạm thời trì hoãn chu kỳ truyền trong một chuỗi burst.
2'b10	NONSEQ	Giao. dịch truyền đơn lẻ hoặc nhịp truyền đầu tiên của một chuỗi burst.
2'b11	SEQ	Nhịp truyền dữ liệu tiếp nối mang tính tuần tự trong một chuỗi burst.

2.2.4.3 Kích thước giao dịch

Tín hiệu HSIZE [2:0] chỉ định kích thước gói dữ liệu được luân chuyển trong mỗi nhịp truyền, hỗ trợ linh hoạt từ mức Byte (8-bit) đến tối đa 1024-bit, nhưng không được vượt quá độ rộng vật lý của bus dữ liệu hệ thống [12]. Kích thước này bắt buộc phải được giữ nguyên trong suốt một cụm truyền [12]. Ngoài ra, giao thức yêu cầu nghiêm ngặt về việc căn lề: mọi địa chỉ truyền đi trên bus phải được căn lề tương ứng với kích thước truy cập của nó [12].

Bảng 2.7: Mã hóa tín hiệu HSIZE [2:0]

Giá trị HSIZE [2:0]	Kích thước truyền	Kích thước (Bits)
3'b000	Byte	8
3'b001	Halfword	16
3'b010	Word	32
3'b011	Doubleword	64

2.2.4.4 Giao dịch Burst

Giao dịch Burst là cơ chế cho phép thực thi một chuỗi thao tác truyền tải dữ liệu liên tiếp, được điều khiển bởi tín hiệu HBURST [2:0] [12]. Hệ thống hỗ trợ truyền đơn lẻ, truyền theo cụm độ dài cố định (4, 8, 16 nhịp) và cụm độ dài không xác định [12].

Địa chỉ trong giao dịch burst được định tuyến theo hai phương thức: tăng dần tuyến tính (INCR), trong đó địa chỉ gia tăng tuần tự; và quay vòng (WRAP), trong đó địa chỉ gia tăng nhưng tự động quay vòng khi chạm đến ranh giới bộ nhớ (được tính bằng số lượng nhịp nhân với kích thước mỗi nhịp) [12].

Bảng 2.8: Mã hóa tín hiệu HBURST [2:0]

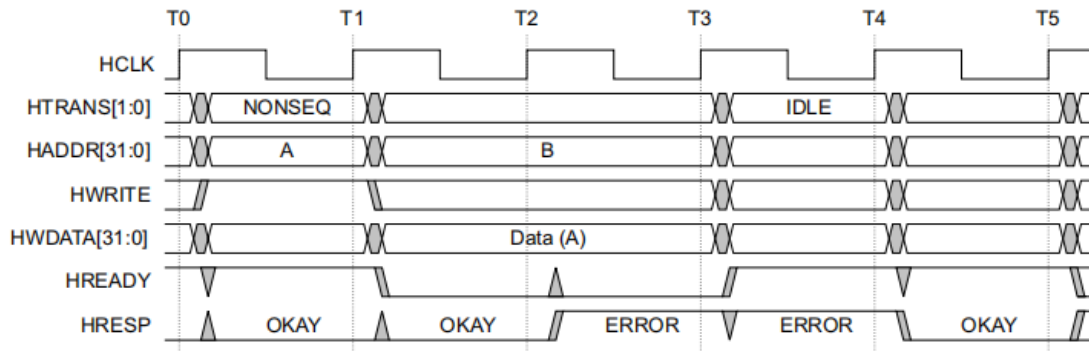
Giá trị HBURST [2:0]	Kiểu Burst	Mô tả
3'b000	SINGLE	Giao dịch truyền đơn lẻ.
3'b001	INCR	Chuỗi truyền tăng dần địa chỉ có độ dài không xác định.

3'b010	WRAP4	Chuỗi truyền quay vòng địa chỉ có chiều dài 4 nhịp.
3'b011	INCR4	Chuỗi truyền tăng dần địa chỉ tuyến tính có chiều dài 4 nhịp.
3'b100	WRAP8	Chuỗi truyền quay vòng địa chỉ có chiều dài 8 nhịp.
3'b101	INCR8	Chuỗi truyền tăng dần địa chỉ tuyến tính có chiều dài 8 nhịp.
3'b110	WRAP16	Chuỗi truyền quay vòng địa chỉ có chiều dài 16 nhịp.
3'b111	INCR16	Chuỗi truyền tăng dần địa chỉ tuyến tính có chiều dài 16 nhịp.

2.2.4.5 Phản hồi giao dịch

Tiến trình truyền tải trên bus được thiết bị tố báo cáo trạng thái thông qua hai tín hiệu HRESP và HREADY [12]. Giao dịch hoàn tất thành công (OKAY) khi HRESP ở mức thấp và HREADY ở mức cao [12].

Đối với trường hợp xảy ra sự cố (như vi phạm quyền truy cập bộ nhớ), thiết bị tố sẽ trả về trạng thái báo lỗi (ERROR). Điểm đặc biệt của cơ chế này là phản hồi ERROR bắt buộc phải kéo dài trong đúng hai chu kỳ xung nhịp [12]. Cụ thể, ở chu kỳ đầu tiên, thiết bị tố kéo HRESP lên mức cao (báo lỗi) đồng thời giữ HREADY ở mức thấp để trì hoãn giao dịch; sang chu kỳ thứ hai, HREADY mới được đưa lên mức cao để chính thức kết thúc giao dịch lỗi [12]. Nguyên lý phản hồi kéo dài hai chu kỳ này cung cấp đủ biên độ an toàn để thiết bị chủ nhận diện sự cố và kịp thời hủy bỏ pha địa chỉ của giao dịch nối tiếp trên đường ống bằng cách chuyển tín hiệu điều khiển HTRANS về trạng thái IDLE [12].



Hình 2.8: Phản hồi lỗi

Hình 2.8 minh họa cơ chế báo lỗi bắt buộc phải kéo dài trong 2 chu kỳ của giao thức AHB Lite. Trình tự diễn ra như sau:

- Khoảng T1-T2 (Trạng thái chờ): Thiết bị tớ chèn một chu kỳ chờ bằng cách kéo HREADY xuống mức thấp và duy trì trạng thái OKAY.
- Khoảng T2-T3 (Chu kỳ lỗi 1): Thiết bị tớ phát hiện sự cố, bắt đầu báo lỗi bằng cách kéo HRESP lên mức cao nhưng vẫn giữ HREADY ở mức THẤP để kéo dài giao dịch.
- Khoảng T3-T4 (Chu kỳ lỗi 2): Thiết bị tớ đưa HREADY lên mức cao để chính thức kết thúc giao dịch lỗi, trong khi HRESP vẫn giữ mức cao. Lúc này, nhờ được cảnh báo từ chu kỳ trước, thiết bị chủ đã kịp chuyển loại giao dịch sang IDLE để hủy bỏ giao dịch kế tiếp tại địa chỉ B.
- Khoảng T4-T5 (Hoàn tất hủy): Thiết bị tớ phản hồi trạng thái OKAY thành công đối với giao dịch IDLE vừa được thiết bị chủ chèn vào.

2.3 TỔNG QUAN VỀ GIAO THỨC APB

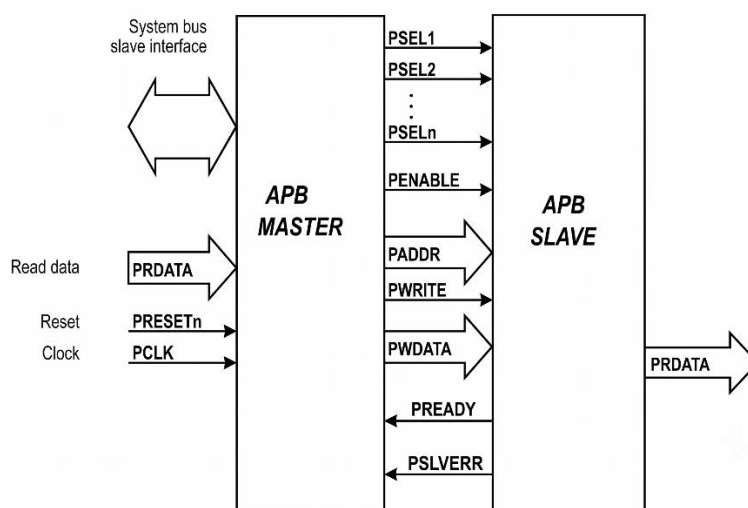
2.3.1 Giới thiệu giao thức APB

Giao thức APB (Advanced Peripheral Bus) là một tiêu chuẩn giao tiếp thuộc họ kiến trúc AMBA, được thiết kế chuyên biệt để tối ưu hóa việc tiêu thụ điện năng và giảm thiểu độ phức tạp của giao diện [13]. Khác với các bus hệ thống tốc độ cao hoạt động theo cơ chế đường ống như AHB, APB là một giao thức không sử dụng đường ống, cực kỳ phù hợp để kết nối với các thiết bị ngoại vi có băng thông thấp và không yêu cầu hiệu suất cao [13]. Mọi sự chuyển đổi tín hiệu trên bus APB

đều được đồng bộ hóa nghiêm ngặt tại cạnh lên của xung nhịp hệ thống, và mỗi giao dịch luôn yêu cầu tối thiểu hai chu kỳ xung nhịp để hoàn tất [13].

Hệ thống APB vận hành dựa trên mô hình thiết bị chủ - thiết bị tớ tập trung. Trong đó, Cầu nối APB đóng vai trò là thiết bị chủ duy nhất, làm nhiệm vụ biên dịch các yêu cầu truy cập từ bus hệ thống cấp cao như AHB thành các giao dịch APB chuẩn mực [14]. Để thực thi giao dịch, thiết bị chủ khởi tạo và điều khiển toàn bộ các tín hiệu định vị địa chỉ (PADDR), hướng giao dịch (PWRITE), tín hiệu chọn thiết bị (PSELx), kích hoạt truy cập (PENABLE) và dữ liệu ghi (PWDATA) [14].

Ngược lại, các module ngoại vi đóng vai trò là thiết bị tớ, hoạt động ở trạng thái hoàn toàn thụ động [14]. Thiết bị tớ chỉ bắt đầu đáp ứng khi nhận được tín hiệu chọn PSELx từ thiết bị chủ [14]. Sau khi tiếp nhận và xử lý yêu cầu, thiết bị tớ có trách nhiệm xuất các tín hiệu phản hồi bao gồm dữ liệu đọc PRDATA, trạng thái sẵn sàng PREADY để quản lý thời gian, và cờ báo lỗi PSLVERR [14]. Do quyền điều khiển tập trung tuyệt đối tại Cầu nối APB, hệ thống không đòi hỏi mạch phân xử phức tạp, giúp tối ưu hóa đáng kể diện tích silicon và năng lượng tiêu thụ cho toàn hệ thống [14].



Hình 2.9: Sơ đồ khối kiến trúc kết nối APB Master và APB Slaves

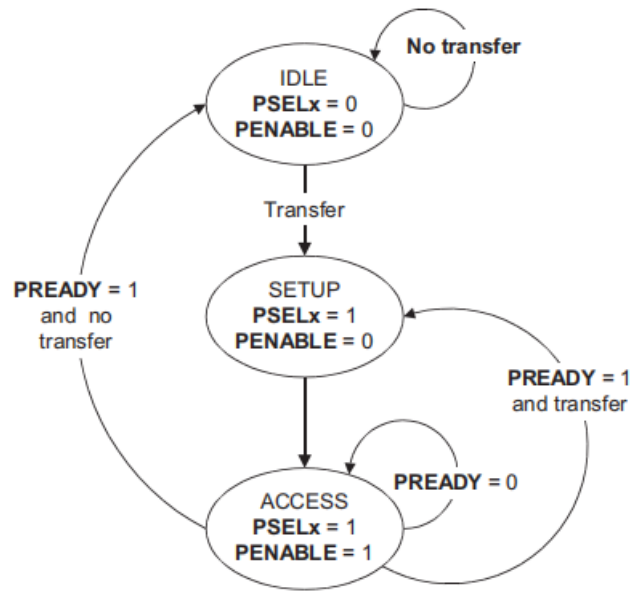
2.3.2 Các tín hiệu của giao thức APB

Các tín hiệu trong giao thức APB được định nghĩa chặt chẽ nhằm đảm bảo quá trình đồng bộ giữa thiết bị chủ và thiết bị tớ. Khác với các phiên bản cũ, chuẩn AMBA 3 APB đã bổ sung thêm tín hiệu PREADY để mở rộng giao dịch và PSLVERR để báo lỗi [13].

Bảng 2.9: Mô tả các tín hiệu của giao thức APB

Tên tín hiệu	Mô tả
PCLK	Xung nhịp hệ thống, mọi tín hiệu đồng bộ tại cạnh lên xung nhịp.
PRESETn	Tín hiệu thiết lập lại hệ thống, tích cực mức thấp.
PADDR	Bus địa chỉ APB, dùng để xác định vị trí thanh ghi nội bộ, có độ rộng lên đến 32-bit.
PSELx	Tín hiệu chọn thiết bị tớ, mỗi slave có một PSELx riêng.
PENABLE	Tín hiệu cho phép, đánh dấu chu kỳ thứ hai của giao dịch.
PWRITE	Điều khiển hướng: mức logic cao là lệnh ghi, mức logic thấp là lệnh đọc.
PWDATA	Bus dữ liệu ghi do thiết bị chủ gửi đến thiết bị tớ đầy khi PWRITE ở mức logic cao, có độ rộng lên đến 32-bit.
PRDATA	Bus dữ liệu đọc do thiết bị tớ gửi về thiết bị chủ khi PWRITE ở mức logic thấp, có độ rộng lên đến 32-bit.
PREADY	Tín hiệu sẵn sàng, thiết bị tớ dùng để kéo dài hoặc kết thúc giao dịch.
PSLVERR	Tín hiệu báo lỗi giao dịch.

2.3.3 Các trạng thái hoạt động của giao thức APB



Hình 2.10: Các trạng thái hoạt động của giao thức APB

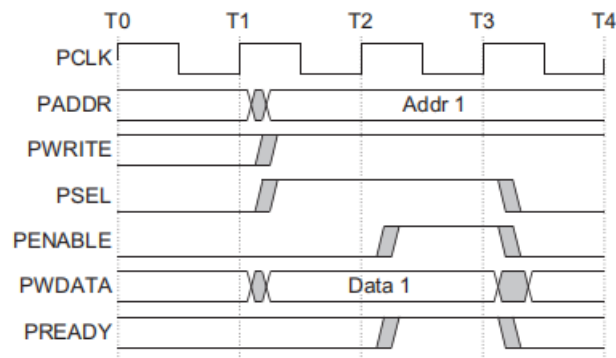
Tiến trình truyền tải dữ liệu trên bus APB được vận hành và kiểm soát bởi một Máy trạng thái hữu hạn (FSM) gồm ba trạng thái cơ bản:

- IDLE (Nghỉ): Đây là trạng thái mặc định khi bus đang rảnh. Tại đây, tín hiệu chọn PSELx và tín hiệu kích hoạt PENABLE đều được duy trì ở mức logic thấp, biểu thị không có yêu cầu giao dịch nào [13].
- SETUP (Thiết lập): Khi có một yêu cầu truy cập, bus ngay lập tức chuyển sang trạng thái SETUP. Thiết bị chủ bắt đầu cấp phát địa chỉ (PADDR), điều khiển hướng (PWRITE) và kéo tín hiệu chọn PSELx lên mức CAO, nhưng vẫn giữ PENABLE ở mức THẤP. Máy trạng thái chỉ tồn tại ở SETUP đúng một chu kỳ xung nhịp và luôn tự động chuyển sang trạng thái ACCESS ở cạnh lên của xung nhịp tiếp theo [13].
- ACCESS (Truy cập): Tín hiệu PENABLE được kéo lên mức CAO, đánh dấu sự kích hoạt thực sự của giao dịch. Trong quá trình này, các tín hiệu địa chỉ, dữ liệu và điều khiển phải được giữ ổn định. Việc thoát khỏi trạng thái ACCESS để hoàn tất giao dịch phụ thuộc hoàn toàn vào mức logic của tín hiệu phản hồi PREADY từ Thiết bị tớ [13].

2.3.4 Quá trình ghi dữ liệu

Quá trình ghi dữ liệu trên APB xảy ra khi thiết bị chủ kéo tín hiệu PWRITE lên mức cao để thiết lập hướng truyền [13]. Toàn bộ thông tin địa chỉ và dữ liệu được thiết bị chủ đưa lên bus và duy trì ổn định để thiết bị tớ ghi nhận [13].

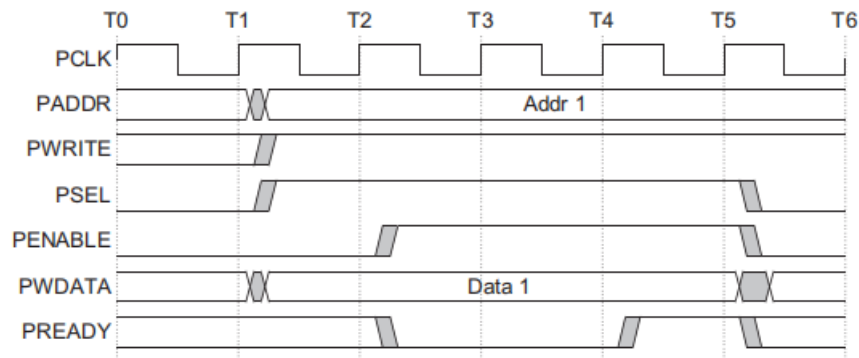
2.3.4.1 Quá trình ghi không có trạng thái chờ



Hình 2.11: Quá trình ghi không có trạng thái chờ

Hình 2.11 minh họa chi tiết về quá trình ghi cơ bản không có trạng thái chờ, được hoàn tất trong đúng hai chu kỳ xung nhịp [13]. Tại chu kỳ đầu tiên (pha SETUP), địa chỉ, dữ liệu ghi, tín hiệu PWRITE và PSELx đều được thiết bị chủ thay đổi và thiết lập ngay sau cạnh lên của xung nhịp, trong khi PENABLE ở mức thấp [13]. Tại chu kỳ thứ hai (pha ACCESS), tín hiệu PENABLE được đưa lên mức cao [13]. Do không có trạng thái đợi, thiết bị tớ giữ tín hiệu PREADY ở mức cao [13]. Tại cuối chu kỳ này, các tín hiệu vẫn duy trì hợp lệ, cho phép thiết bị tớ lấy mẫu và ghi nhận dữ liệu an toàn. Giao dịch kết thúc, tín hiệu PENABLE bị kéo xuống thấp [13].

2.3.4.2 Quá trình ghi có trạng thái chờ



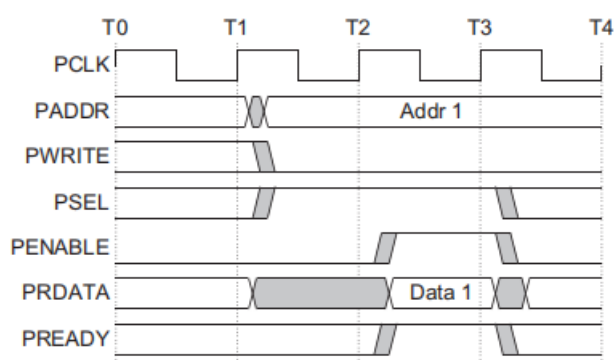
Hình 2.12: Quá trình ghi có trạng thái chờ

Hình 2.12 minh họa chi tiết về quá trình ghi có trạng thái chờ, được diễn ra khi thiết bị tó cần thêm thời gian để xử lý [13]. Trong pha ACCESS (PENABLE = 1), thiết bị tó chủ động kéo tín hiệu PREADY xuống mức thấp [13]. Theo quy chuẩn, thiết bị chủ bắt buộc phải giữ nguyên tất cả các tín hiệu PADDR, PWRITE, PSEL, PENABLE và PWDATA trong suốt các chu kỳ bổ sung này [13]. Trạng thái chờ có thể kéo dài nhiều chu kỳ [13]. Khi thiết bị tó đã xử lý xong, nó đưa PREADY lên mức cao trở lại, và dữ liệu sẽ được ghi nhận tại cạnh lên xung nhịp ngay sau đó, chính thức kết thúc giao dịch [13].

2.3.5 Quá trình đọc dữ liệu

Khác với quá trình ghi, quá trình đọc yêu cầu thiết bị tó phải xuất dữ liệu nội bộ lên bus PRDATA [13]. Tín hiệu PWRITE được thiết bị chủ giữ ở mức thấp trong suốt giao dịch [13]. Thiết bị tó có trách nhiệm đảm bảo dữ liệu hợp lệ trước khi chu kỳ truy cập kết thúc [13].

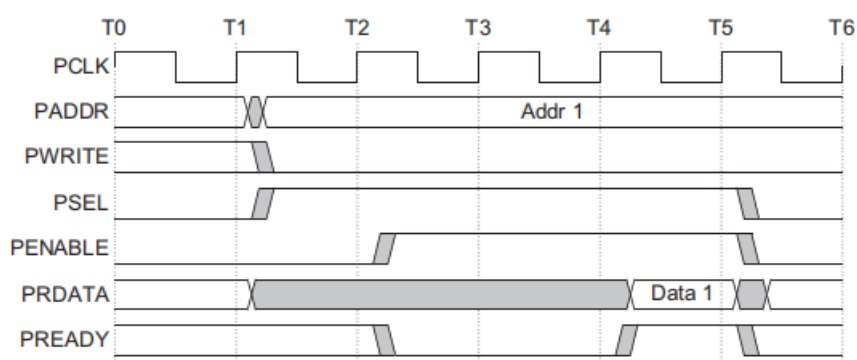
2.3.5.1 Quá trình đọc không có trạng thái chờ



Hình 2.13: Quá trình đọc không có trạng thái chờ

Quá trình đọc không có trạng thái chờ được thể hiện cụ thể qua hình 2.13, việc thiết lập địa chỉ và chọn thiết bị hoàn toàn tương tự như thao tác ghi [13]. Tại chu kỳ SETUP, thiết bị chủ xuất PADDR, kéo PSELx lên cao và giữ PWRITE, PENABLE ở mức thấp [13]. Bước sang chu kỳ ACCESS, PENABLE được kéo lên cao [13]. Thiết bị tớ nhận được yêu cầu sẽ xuất dữ liệu lên bus PRDATA. Do thiết bị tớ phản hồi ngay (PREADY = 1), nó bắt buộc phải cung cấp dữ liệu ổn định và hợp lệ trước khi kết thúc thao tác đọc ở cuối chu kỳ ACCESS này [13]. Thiết bị chủ sẽ lấy mẫu dữ liệu tại cạnh lên của xung nhịp [13].

2.3.5.2 Quá trình đọc có trạng thái chờ



Hình 2.14: Quá trình đọc có trạng thái chờ

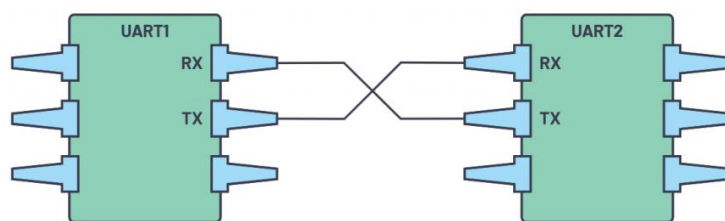
Quá trình đọc có trạng thái chờ được thể hiện cụ thể qua Hình 2.14, tương tự cơ chế ghi có trạng thái chờ, giao dịch đọc có thể được mở rộng bằng cách thiết bị tớ đưa tín hiệu PREADY xuống mức thấp trong pha ACCESS [13]. Giao thức đảm bảo rằng thiết bị chủ sẽ duy trì ổn định trạng thái của PADDR, PWRITE,

PSEL và PENABLE qua các chu kỳ chờ bổ sung này [13]. Trong các chu kỳ PREADY ở mức thấp, dữ liệu trên PRDATA chưa cần phải hợp lệ. Chỉ khi thiết bị tở đã chuẩn bị xong dữ liệu, nó mới đưa PREADY lên mức cao [13]. Lúc này, tín hiệu PRDATA phải chứa giá trị chính xác để thiết bị chủ có thể chốt dữ liệu thành công trước cạnh lên của xung nhịp kết thúc [13].

2.4 TỔNG QUAN VỀ GIAO THỨC UART

2.4.1 Giới thiệu về giao thức UART

Bộ truyền nhận dữ liệu không đồng bộ (Universal Asynchronous Receiver-Transmitter hay UART) là giao thức truyền thông phổ biến không sử dụng xung nhịp chung giữa thiết bị phát và thiết bị thu [15]. Thay vào đó, thiết bị nhận sẽ dựa vào việc phát hiện một tín hiệu khởi đầu và tự động lấy mẫu các bit dữ liệu đến dựa trên một tần số hoạt động đã được thống nhất từ trước [15].



Hình 2.15: Kết nối của hai bộ UART

Giao thức UART mang lại ưu điểm vượt trội về sự đơn giản trong thiết kế phần cứng, bởi nó chỉ yêu cầu tối thiểu hai đường dây tín hiệu cơ bản để thiết lập giao tiếp: đường truyền dữ liệu (TX) và đường nhận dữ liệu (RX) [15]. Sơ đồ đấu nối phần cứng điểm-đối-điểm giữa hai module UART được minh họa trực quan trong hình 2.15, trong đó chân phát của thiết bị này liên kết trực tiếp với chân nhận của thiết bị kia [15]. Cấu trúc này cho phép hệ thống vận hành linh hoạt theo các chế độ một chiều (simplex), bán song công (half-duplex) hoặc song công hoàn toàn (full-duplex) [15]. Chức năng cốt lõi của module UART là thực hiện chuyển đổi qua lại giữa dữ liệu song song từ bộ vi xử lý trung tâm và chuỗi dữ liệu nối tiếp trên đường truyền vật lý [15]. Nhờ ưu điểm tối giản số lượng chân kết nối và độ

tin cậy cao, UART được ứng dụng rộng rãi trong các cấu trúc vi điều khiển và hệ thống trên chip hiện đại [15].

2.4.2 Cấu trúc khung truyền UART

Trong giao thức UART, đường truyền dữ liệu luôn được duy trì ở mức điện áp cao khi không có dữ liệu nào được truyền đi [15]. Dữ liệu thực tế sẽ được đóng gói thành các khung truyền với cấu trúc bao gồm các thành phần sau:

- Start bit: Để khởi tạo quá trình truyền, thiết bị UART TX sẽ kéo đường truyền từ mức điện áp cao xuống mức điện áp thấp [2, 15]. Khi UART RX phát hiện được sự chuyển đổi trạng thái từ cao xuống thấp này, nó sẽ bắt đầu đọc các bit trong khung dữ liệu theo tần số của tốc độ Baud [15].
- Data Frame: Đây là thành phần mang thông tin thực sự cần truyền tải [15]. Khung dữ liệu này có thể có độ dài từ 5 đến 8 bit nếu hệ thống có sử dụng bit kiểm tra chẵn lẻ [15]. Trong trường hợp không sử dụng bit chẵn lẻ, chiều dài dữ liệu có thể lên tới 9 bit. Thông thường, bit có trọng số nhỏ nhất sẽ được ưu tiên truyền đi đầu tiên [2, 15].
- Parity bit: Đây là một cơ chế giúp UART nhận phát hiện xem dữ liệu có bị thay đổi trong quá trình truyền tải hay không [2, 15]. Sau khi đọc khung dữ liệu, UART nhận sẽ đếm số lượng bit có giá trị 1 [15]. Nếu hệ thống dùng cấu hình even parity, tổng số bit 1 phải là một số chẵn, nếu dùng cấu hình odd parity, tổng số bit 1 phải là một số lẻ [15]. Nếu kết quả kiểm tra không khớp, hệ thống sẽ xác định là khung dữ liệu đã xảy ra lỗi [15].
- Stop bits: Để báo hiệu sự kết thúc của một gói dữ liệu, UART phát sẽ điều khiển đường truyền từ mức điện áp thấp trở lại mức điện áp cao trong khoảng thời gian kéo dài ít nhất hai chu kỳ bit hoặc từ 1 đến 2 bit tùy cấu hình [2, 15].

START BIT	D0	D1	D2	D3	D4	D5	D6	D7	D8	PARITY BIT	STOP BIT
-----------	----	----	----	----	----	----	----	----	----	------------	----------

Hình 2.16: Khung dữ liệu UART có 8 bit dữ liệu

2.4.3 Bộ tạo tốc độ Baud và cơ chế Oversampling

Do giao thức UART không chia sẻ đường tín hiệu xung nhịp chung, tính đồng bộ của hệ thống phụ thuộc hoàn toàn vào tốc độ truyền nhận tín hiệu, được gọi là tốc độ Baud. Tốc độ Baud được định nghĩa là số lượng bit dữ liệu được truyền đi trong vòng một giây, với đơn vị tính là bit trên giây (bps). Ý nghĩa vật lý của thông số này quyết định trực tiếp đến thời gian tồn tại của mỗi bit trên đường dây ($T_{bit} = \frac{1}{\text{Tốc độ Baud}}$). Các giá trị tốc độ Baud phổ biến trong công nghiệp bao gồm 4800, 9600, 19200, 57600 và 115200 bps. Điều kiện tiên quyết để hai thiết bị UART có thể giao tiếp thành công là chúng phải được cấu hình hoạt động ở cùng một tốc độ Baud.

Để đảm bảo độ chính xác trong quá trình lấy mẫu dữ liệu tại thiết bị nhận, hệ thống UART thường sử dụng cơ chế oversampling. Bộ tạo tốc độ Baud nội bộ sẽ sử dụng xung nhịp hệ thống để tạo ra một xung nhịp tham chiếu (Baud clock) có tần số lớn gấp 16 lần hoặc 13 lần so với tốc độ Baud thực tế. Giá trị bộ chia để cấu hình cho bộ tạo xung nhịp được xác định theo công thức:

- Chế độ lấy mẫu 16x: $\text{Giá trị bộ chia} = \frac{\text{Tần số xung nhịp đầu vào UART (Hz)}}{\text{Tốc độ Baud} \times 16}$.
- Chế độ lấy mẫu 13x: $\text{Giá trị bộ chia} = \frac{\text{Tần số xung nhịp đầu vào UART (Hz)}}{\text{Tốc độ Baud} \times 13}$.

Bảng 2.10: Giá trị hệ số chia ứng với các tốc độ baud khi UART sử dụng xung clock đầu vào 100 MHz và chế độ Oversampling 16x

Tốc độ baud	Thời gian truyền một bit	Giá trị hệ số chia
2400	416.667 μ s	2604
4800	208.333 μ s	1302
9600	104.167 μ s	651
19200	52.083 μ s	325
38400	26.042 μ s	163
76800	13.021 μ s	81
115200	8.681 μ s	54

Bảng 2.11: Giá trị hệ số chia ứng với các tốc độ baud khi UART sử dụng xung clock đầu vào 100 MHz và chế độ Oversampling 13x

Tốc độ baud	Thời gian truyền một bit	Giá trị hệ số chia
2400	416.667 μ s	3205
4800	208.333 μ s	1602
9600	104.167 μ s	801
19200	52.083 μ s	401
38400	26.042 μ s	200
76800	13.021 μ s	100
115200	8.681 μ s	67

Nhờ xung nhịp tham chiếu tần số cao này, thiết bị nhận có thể định vị chính xác vị trí tâm của một bit dữ liệu để tiến hành lấy mẫu. Cụ thể, hệ thống sẽ thực hiện lấy mẫu tín hiệu tại chu kỳ BCLK thứ 8 đối với chế độ oversampling 16x, và tại chu kỳ BCLK thứ 6 đối với chế độ oversampling 13x. Cơ chế này giúp thiết bị nhận hạn chế tối đa các sai số sinh ra do hiện tượng nhiễu tín hiệu ở hai rìa của chu kỳ bit.

2.4.4 Nguyên lý truyền dữ liệu của UART

Quá trình truyền dữ liệu của UART thực chất là tiến trình chuyển đổi thông tin từ định dạng song song sang nối tiếp. Dữ liệu từ vi xử lý ban đầu được ghi vào bộ đệm truyền và lưu trữ tạm thời tại bộ nhớ đệm fifo phát (thường là 16 byte) để giảm tải tần suất can thiệp của hệ thống trung tâm.

Dựa trên cấu hình khung truyền được thiết lập ở thanh ghi điều khiển, khối điều khiển sẽ nối tiếp hóa dữ liệu và xuất tuần tự từng bit ra ngoài qua chân tín hiệu TX. Từ trạng thái rảnh ở mức logic cao, quá trình truyền bắt đầu bằng Start bit ở mức logic thấp, tiếp theo là chuỗi các bit dữ liệu, Parity bit nếu có, và kết thúc bằng Stop bit ở mức logic cao.

Để phối hợp hoạt động với vi xử lý, khối truyền sử dụng hệ thống tín hiệu ngắt. Cụ thể, tín hiệu ngắt báo bộ đệm fifo phát rỗng báo hiệu hệ thống sẵn sàng

nhận thêm dữ liệu mới, trong khi tín hiệu ngắt báo bộ đệm fifo phát đầy yêu cầu ngừng ghi dữ liệu để tránh hiện tượng mất mát các gói tin tiếp theo.

2.4.5 Nguyên lý nhận dữ liệu của UART

Quá trình nhận dữ liệu thực hiện chức năng chuyển đổi dòng dữ liệu nối tiếp thành định dạng song song. Bộ nhận luôn hoạt động ở trạng thái giám sát đường dây RX. Tiến trình nhận chính thức được kích hoạt khi hệ thống phát hiện sự chuyển đổi trạng thái từ mức điện áp cao xuống thấp, đây chính là dấu hiệu của Start bit.

Ngay khi xác nhận Start bit, hệ thống sử dụng xung nhịp tham chiếu Baud clock để tiến hành lấy mẫu từng bit tiếp theo. Nhờ cơ chế này, hệ thống luôn ưu tiên đọc các bit dữ liệu, Parity bit và Stop bit ở vị trí chính giữa của chu kỳ để đảm bảo tính chính xác cao nhất.

Dữ liệu song song thu được sau đó sẽ được đẩy vào bộ nhớ đệm fifo nhận (thường là 16 byte). Tương tự khối truyền, khối nhận cũng cung cấp hệ thống ngắt: tín hiệu ngắt báo bộ đệm fifo nhận rỗng chỉ ra chưa có dữ liệu mới, và tín hiệu ngắt báo bộ đệm fifo nhận đầy yêu cầu vi xử lý phải lập tức đọc dữ liệu ra để nhường không gian cho các khung truyền kế tiếp.

2.5 PHƯƠNG PHÁP KIỂM CHỨNG UVM

2.5.1 Giới thiệu UVM

Universal Verification Methodology (UVM) là một phương pháp kiểm chứng chuẩn hóa được phát triển bởi tổ chức Accellera dựa trên ngôn ngữ SystemVerilog nhằm hỗ trợ xây dựng các môi trường kiểm chứng có khả năng tái sử dụng, mở rộng và bảo trì hiệu quả. [9]

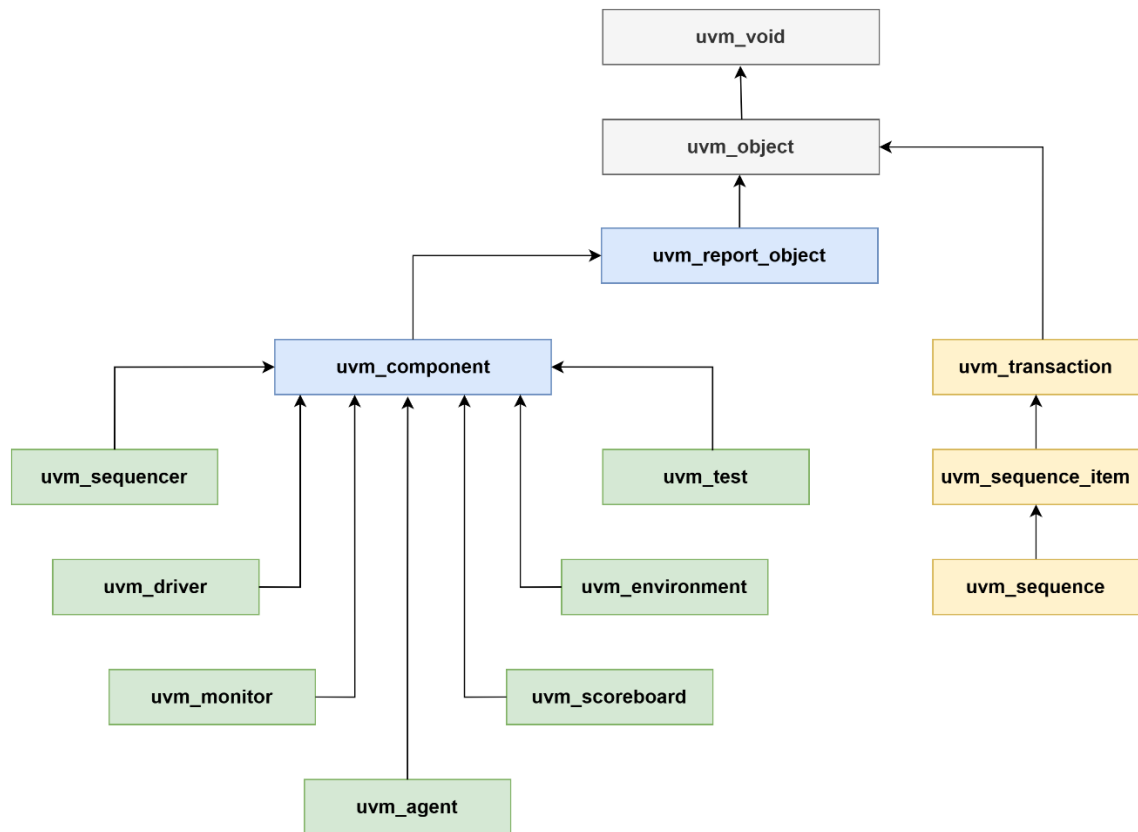
Về bản chất, UVM là một thư viện các lớp được xây dựng trên nền tảng lập trình hướng đối tượng của SystemVerilog. Thông qua việc kế thừa và tùy biến các lớp cơ sở, người dùng có thể xây dựng các thành phần kiểm chứng phù hợp với từng giao thức hoặc thiết kế cụ thể. Điều này giúp giảm thời gian phát triển và nâng cao khả năng tái sử dụng của testbench trong các dự án khác nhau. [9], [10]

Khác với các testbench truyền thống thường thao tác trực tiếp trên từng tín hiệu của thiết kế, UVM áp dụng phương pháp kiểm chứng dựa trên giao dịch, trong đó dữ liệu kiểm tra được biểu diễn ở mức trừu tượng cao dưới dạng các giao dịch. Đồng thời, UVM cung cấp một khuôn mẫu chuẩn để tổ chức môi trường kiểm chứng thành các thành phần chức năng độc lập, giúp testbench trở nên dễ phát triển, bảo trì và tái sử dụng. Nhờ những ưu điểm này, UVM hiện đã trở thành một trong những phương pháp kiểm chứng được sử dụng rộng rãi trong ngành công nghiệp thiết kế vi mạch.

Trong đề tài này, UVM được sử dụng để xây dựng môi trường kiểm chứng cho lõi AHB Lite – UART IP. Việc áp dụng UVM cho phép tổ chức các thành phần kiểm chứng theo kiến trúc phân cấp rõ ràng, hỗ trợ mở rộng các trường hợp kiểm tra và tạo tiền đề cho việc tích hợp các kỹ thuật kiểm chứng nâng cao trong quá trình đánh giá chức năng của hệ thống.

2.5.2 Kiến trúc phân lớp của UVM

UVM được xây dựng dựa trên nguyên lý kế thừa của lập trình hướng đối tượng trong SystemVerilog. Các lớp trong UVM được tổ chức theo cấu trúc phân cấp nhằm tăng khả năng tái sử dụng và mở rộng của môi trường kiểm chứng. Về cơ bản, các lớp này có thể được chia thành hai nhánh chính là `uvm_component` và `uvm_object`. Ngoài ra, lớp `uvm_report_object` đóng vai trò là lớp trung gian, cung cấp cơ chế báo cáo thống nhất cho các thành phần của môi trường kiểm chứng thông qua các mức thông báo như thông tin, cảnh báo và lỗi. Hình 2.17 minh họa mối quan hệ kế thừa giữa các lớp thường được sử dụng trong UVM.



Hình 2.17: Kiến trúc phân lớp của UVM

Hai nhánh `uvm_object` và `uvm_component` được thiết kế để đáp ứng những yêu cầu khác nhau của môi trường kiểm chứng. Các đối tượng thuộc `uvm_object` như `uvm_transaction`, `uvm_sequence_item`, `uvm_sequence` được tạo ra để biểu diễn dữ liệu ở mức giao dịch và không tham gia vào cơ chế pha của UVM. Ngược lại, các đối tượng thuộc `uvm_component` được tổ chức theo cấu trúc phân cấp của testbench, tồn tại xuyên suốt quá trình mô phỏng và tham gia vào hệ thống pha chuẩn của UVM. Việc tách biệt hai loại đối tượng này giúp các giao dịch có thể được tạo và hủy linh hoạt mà không phải trải qua các pha của UVM, từ đó giảm chi phí xử lý khi số lượng giao dịch phát sinh lớn.

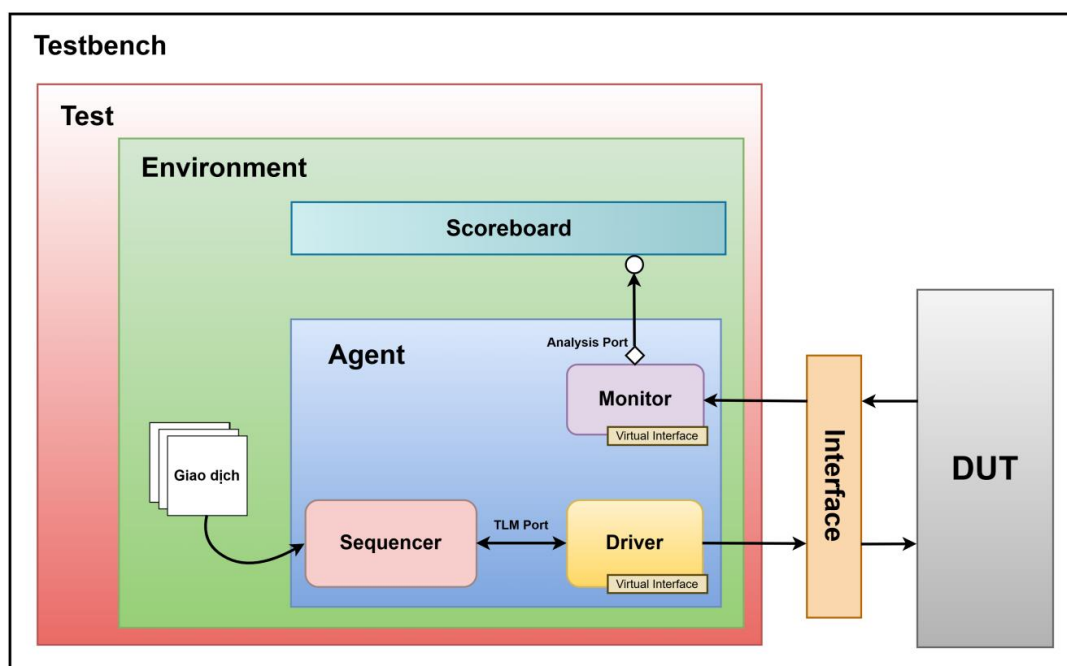
Nhánh `uvm_object` bao gồm các lớp phục vụ cho việc xây dựng và điều khiển các giao dịch kiểm chứng. Trong đó, `uvm_transaction` là lớp cơ sở dùng để xây dựng các đối tượng giao dịch cần trao đổi với thiết kế cần kiểm chứng (Design Under Test – DUT). Lớp `uvm_sequence_item` được sử dụng để mô tả dữ liệu và

các thuộc tính của từng giao dịch cụ thể, trong khi `uvm_sequence` chịu trách nhiệm tạo và điều phối chuỗi các giao dịch nhằm tạo ra các kích thích cho DUT phục vụ quá trình kiểm chứng.

Đối với nhánh `uvm_component`, các lớp được sử dụng để xây dựng cấu trúc của môi trường kiểm chứng UVM. Một số thành phần thường gặp gồm `uvm_sequencer`, `uvm_driver`, `uvm_monitor`, `uvm_agent`, `uvm_scoreboard`, `uvm_environment` và `uvm_test`. Các thành phần này phối hợp với nhau để tạo thành một môi trường kiểm chứng hoàn chỉnh, có khả năng tái sử dụng và mở rộng cao. Vai trò và chức năng cụ thể của từng thành phần sẽ được trình bày trong Mục 2.5.3.

2.5.3 Các thành phần chính của UVM

Các `uvm_component` phối hợp với nhau để tạo thành một môi trường kiểm chứng UVM hoàn chỉnh, trong đó mỗi thành phần đảm nhận một chức năng riêng biệt. Hình 2.18 minh họa kiến trúc tổng quát của một môi trường kiểm chứng UVM cơ bản.



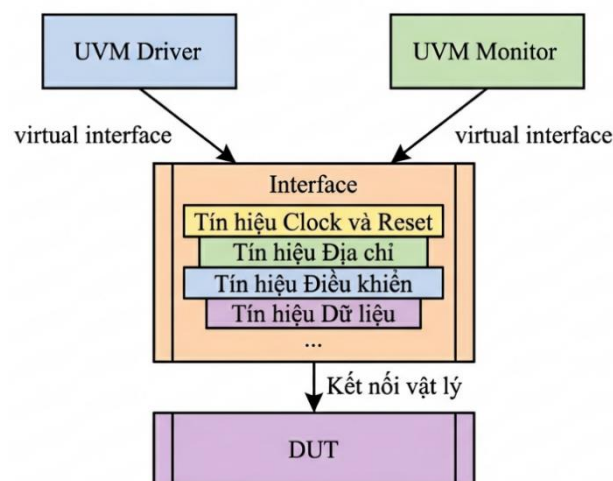
Hình 2.18: Kiến trúc tổng quát của một môi trường kiểm chứng UVM

Test là thành phần cấp cao nhất, chịu trách nhiệm cấu hình tham số và lựa chọn kịch bản kiểm tra để khởi tạo môi trường kiểm chứng phù hợp với mục tiêu đề ra.

Environment đóng gói và quản lý toàn bộ các thành phần bên trong như agent và scoreboard, đồng thời thiết lập các kết nối giữa chúng, ví dụ kết nối giữa monitor và scoreboard.

Agent tập hợp sequencer, driver và monitor phục vụ một giao diện cụ thể của DUT. Agent có thể hoạt động ở chế độ chủ động (active - đủ cả ba thành phần) hoặc chế độ bị động (passive - chỉ có monitor).

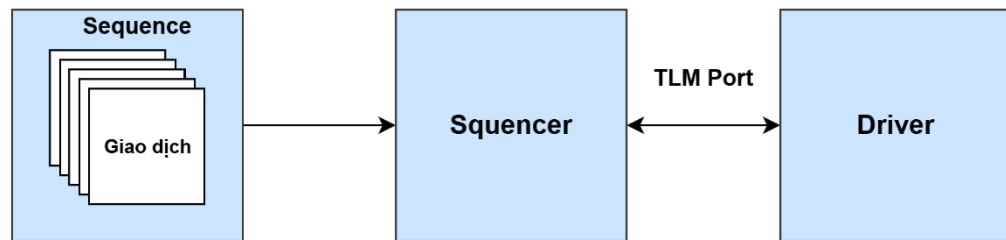
Interface là cấu trúc SystemVerilog dùng để tập hợp các tín hiệu giao tiếp của DUT vào một thực thể thống nhất. Vì driver và monitor là các đối tượng động thuộc class, chúng không thể truy cập interface trực tiếp mà phải thông qua virtual interface - một biến tham chiếu đến interface thực trong quá trình mô phỏng. Hình 2.19 minh họa vai trò của interface và virtual interface trong môi trường UVM.



Hình 2.19: Vai trò của interface trong môi trường kiểm chứng UVM

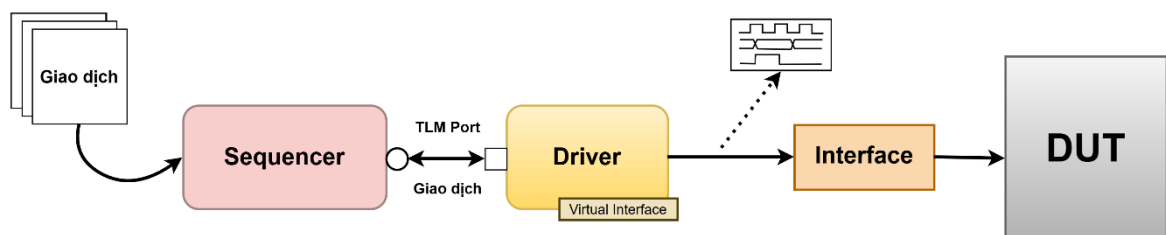
Sequencer có nhiệm vụ quản lý và điều phối các giao dịch được tạo ra từ sequence trước khi chuyển đến driver. Thành phần này đóng vai trò trung gian giữa sequence và driver, đảm bảo các giao dịch được thực thi theo đúng trình tự mong

muốn. Việc trao đổi giao dịch giữa sequencer và driver được thực hiện thông qua cơ chế TLM Port. Cơ chế này giúp truyền các giao dịch từ sequencer đến driver, đồng thời tách biệt quá trình điều phối giao dịch khỏi quá trình tạo tín hiệu tác động lên DUT. Hình 2.20 minh họa quá trình điều phối giao dịch giữa sequence, sequencer và driver trong môi trường kiểm chứng UVM.



Hình 2.20: Cơ chế điều phối giao dịch giữa sequence, sequencer và driver

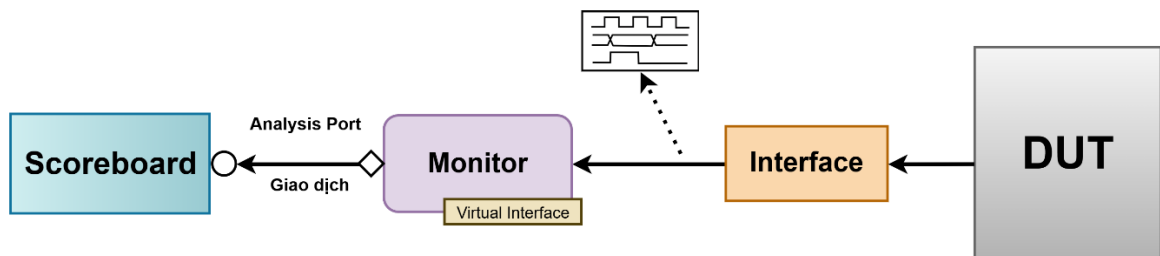
Driver chịu trách nhiệm nhận các giao dịch từ sequencer và chuyển đổi chúng thành các tín hiệu tương ứng để tác động lên DUT thông qua giao diện kết nối. Thành phần này đóng vai trò cầu nối giữa môi trường kiểm chứng ở mức giao dịch và thiết kế cần kiểm chứng ở mức tín hiệu, đảm bảo các kích thích được áp dụng đúng theo nội dung của từng giao dịch. Hình 2.21 minh họa quá trình driver nhận giao dịch từ sequencer và chuyển đổi chúng thành các tín hiệu điều khiển DUT.



Hình 2.21: Quá trình chuyển đổi giao dịch thành tín hiệu điều khiển DUT của driver

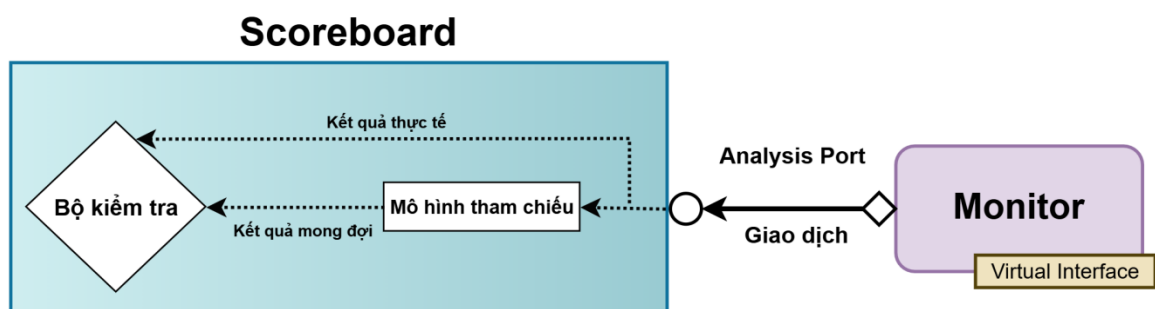
Monitor có nhiệm vụ quan sát các tín hiệu trên giao diện của DUT mà không tác động đến hoạt động của hệ thống. Thông qua virtual interface, monitor có thể

truy cập và thu thập dữ liệu từ DUT, sau đó chuyển đổi các tín hiệu quan sát được thành các giao dịch phục vụ cho quá trình kiểm chứng. Các giao dịch này được gửi đến các thành phần khác như scoreboard thông qua Analysis Port, cho phép nhiều thành phần có thể đồng thời sử dụng cùng một nguồn dữ liệu. Hình 2.22 minh họa quá trình monitor thu thập tín hiệu từ DUT, chuyển đổi thành các giao dịch và gửi đến scoreboard thông qua Analysis Port.



Hình 2.22: Quá trình thu thập, chuyển đổi giao dịch của monitor

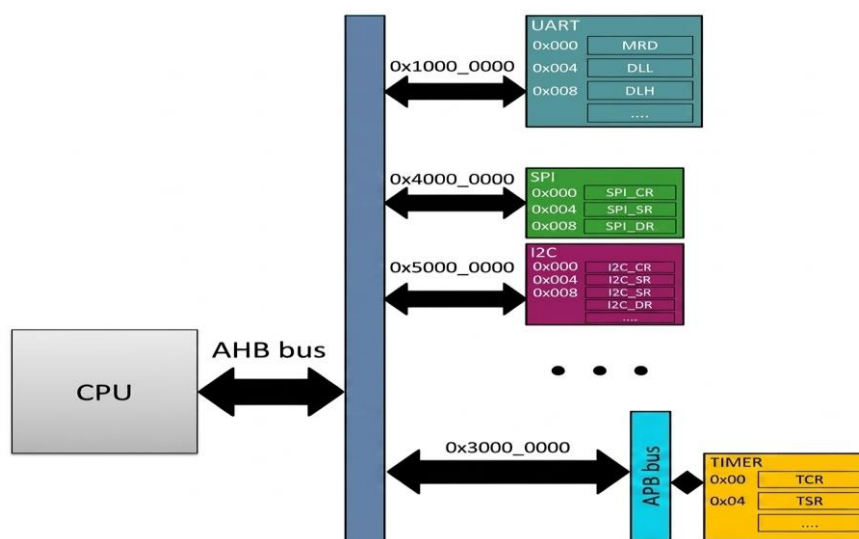
Scoreboard là thành phần thực hiện chức năng kiểm tra tính đúng đắn của DUT. Scoreboard nhận các giao dịch được monitor gửi đến thông qua Analysis Port, sau đó sử dụng mô hình tham chiếu để tạo ra kết quả mong đợi tương ứng với các giao dịch này. Đồng thời, các giao dịch phản ánh hoạt động thực tế của DUT cũng được chuyển đến bộ kiểm tra để thực hiện quá trình so sánh với kết quả mong đợi. Quá trình so sánh này giúp đánh giá tính đúng đắn của DUT dựa trên các tiêu chí kiểm tra đã được xây dựng trước. Hình 2.23 minh họa quá trình kiểm tra kết quả của DUT bên trong scoreboard.



Hình 2.23: Quá trình so sánh kết quả trong scoreboard

2.5.4 Lớp trừu tượng hóa thanh ghi

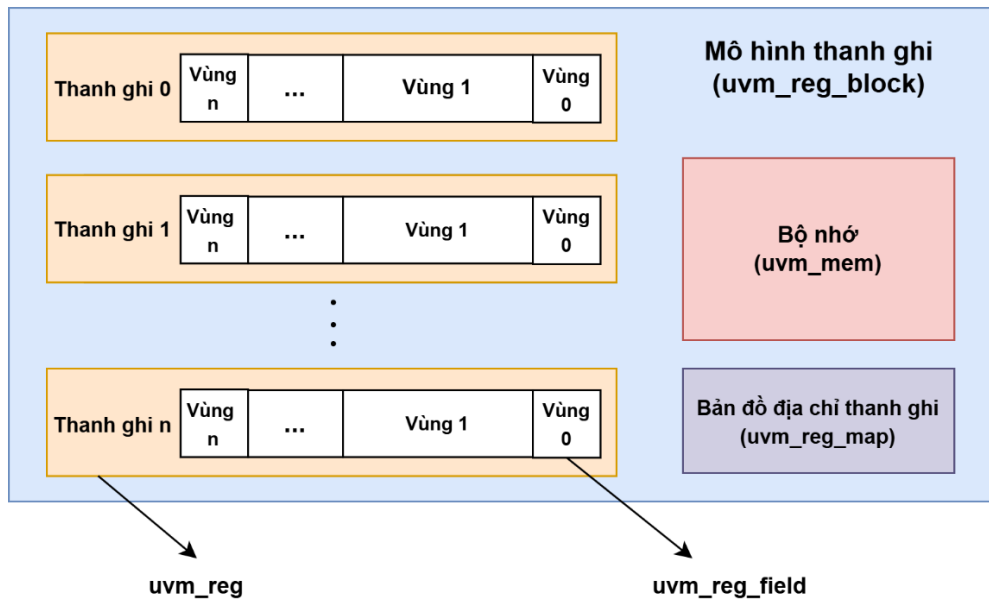
Trong các hệ thống SoC hiện đại, DUT thường chứa số lượng lớn thanh ghi phân bố trên nhiều vùng địa chỉ khác nhau. Việc truy cập trực tiếp qua giao thức bus đòi hỏi người kiểm chứng phải quản lý thủ công địa chỉ, định dạng dữ liệu và đặc điểm từng giao thức, khiến các bài kiểm tra trở nên khó bảo trì khi đặc tả thay đổi. Hình 2.24 minh họa tổ chức không gian địa chỉ điển hình trong hệ thống SoC.



Hình 2.24: Minh họa tổ chức thanh ghi của các ngoại vi trong hệ thống SoC

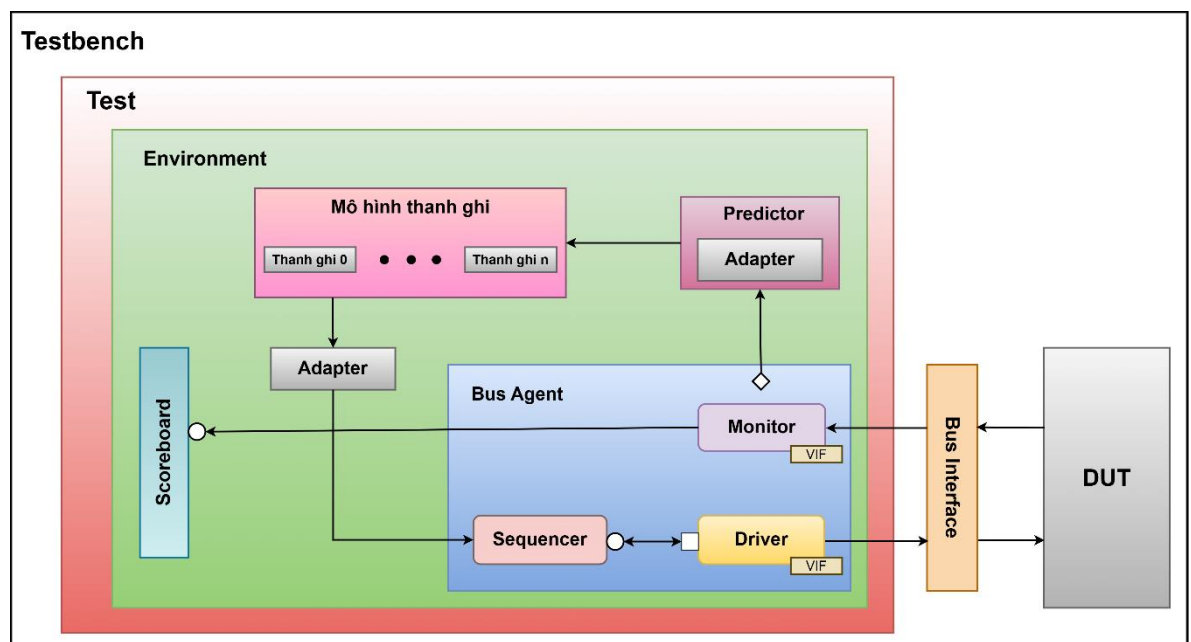
Để giải quyết vấn đề này, UVM cung cấp lớp trừu tượng hóa thanh ghi (Register Abstraction Layer - RAL), cho phép xây dựng mô hình hướng đối tượng cho toàn bộ thanh ghi của DUT. Thay vì thao tác trực tiếp trên giao dịch bus, người kiểm chứng chỉ cần thực hiện các thao tác đọc/ghi thông qua các đối tượng trong mô hình thanh ghi.[9]

Hình 2.25 minh họa cấu trúc phân cấp của mô hình thanh ghi, bao gồm các lớp chính: `uvm_reg_block` mô hình hóa toàn bộ khối thanh ghi của DUT; `uvm_reg` biểu diễn từng thanh ghi; `uvm_reg_field` mô tả các trường dữ liệu bên trong thanh ghi; `uvm_reg_map` xác định ánh xạ địa chỉ; và `uvm_mem` hỗ trợ mô hình hóa vùng bộ nhớ nếu có.



Hình 2.25: Cấu trúc tổng quát của mô hình thanh ghi

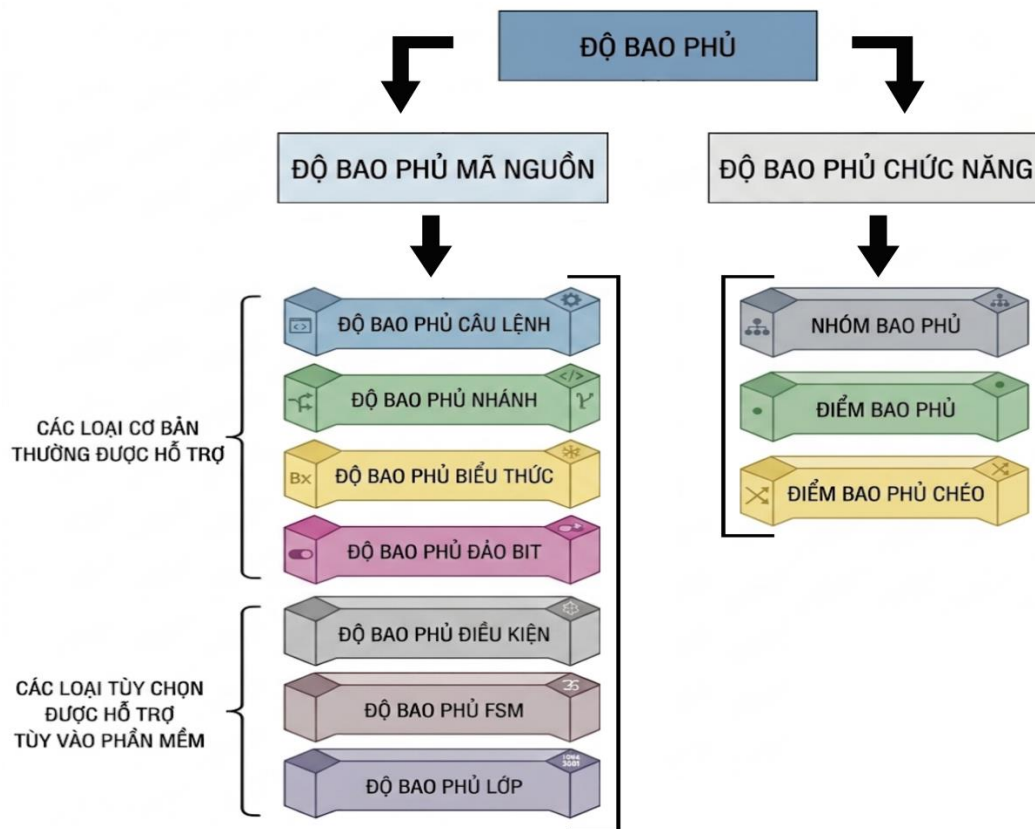
Khi sử dụng RAL, Adapter đảm nhận việc chuyển đổi các yêu cầu truy cập thanh ghi từ mô hình thành giao dịch bus tương ứng để truyền đến sequencer và driver. Đồng thời, Predictor sử dụng các giao dịch do monitor thu thập để cập nhật lại giá trị trong mô hình thanh ghi, đảm bảo mô hình luôn phản ánh đúng trạng thái thực tế của DUT. Hình 2.26 minh họa cơ chế hoạt động của hai thành phần này.



Hình 2.26: Cơ chế hoạt động của adapter và predictor

2.6 KIỂM CHỨNG DỰA TRÊN ĐỘ BAO PHỦ

Trong quá trình kiểm chứng, việc phát hiện lỗi thông qua các trường hợp kiểm tra là chưa đủ để đánh giá mức độ hoàn thiện của môi trường kiểm chứng. Mặc dù một tập hợp bài kiểm tra có thể được thực thi thành công, vẫn tồn tại khả năng một số phần của thiết kế hoặc một số mục tiêu kiểm chứng chưa được xem xét đầy đủ. Do đó, các độ đo bao phủ được sử dụng nhằm đánh giá mức độ mà thiết kế và các mục tiêu kiểm chứng đã được kiểm tra, từ đó hỗ trợ người kiểm chứng xây dựng thêm các kịch bản phù hợp để nâng cao chất lượng kiểm chứng. Trong thực tế, độ bao phủ mã nguồn và độ bao phủ chức năng là hai loại độ bao phủ được sử dụng phổ biến. Hình 2.27 minh họa mối quan hệ giữa các loại độ bao phủ thường được sử dụng trong quá trình kiểm chứng.



Hình 2.27: Phân loại các độ đo bao phủ trong kiểm chứng

2.6.1 Độ bao phủ mã nguồn

Độ bao phủ mã nguồn được sử dụng để đánh giá mức độ mà mã RTL của DUT đã được thực thi trong quá trình mô phỏng. Các loại độ bao phủ mã nguồn

thường được hỗ trợ bao gồm độ bao phủ câu lệnh, độ bao phủ nhánh, độ bao phủ biểu thức và độ bao phủ đảo bit. Ngoài ra, một số công cụ mô phỏng còn hỗ trợ các loại độ bao phủ khác như độ bao phủ điều kiện, độ bao phủ máy trạng thái hữu hạn và độ bao phủ lớp. Kết quả độ bao phủ mã nguồn thường được công cụ mô phỏng tự động thu thập và tạo báo cáo sau khi quá trình kiểm chứng kết thúc. Thông qua các báo cáo này, người kiểm chứng có thể xác định những phần của mã RTL chưa được thực thi để xây dựng thêm các bài kiểm tra phù hợp [9].

2.6.2 Độ bao phủ chức năng

Độ bao phủ chức năng được sử dụng để đánh giá mức độ đạt được của các mục tiêu kiểm chứng đã được xác định trước. Khác với độ bao phủ mã nguồn được công cụ tự động thu thập, độ bao phủ chức năng cần được người kiểm chứng chủ động xây dựng dựa trên đặc tả thiết kế và kế hoạch kiểm chứng. Trong SystemVerilog, độ bao phủ chức năng được hiện thực thông qua nhóm bao phủ, điểm bao phủ và bao phủ giao chéo. Kết quả độ bao phủ chức năng giúp xác định những trường hợp chức năng chưa được kiểm tra đầy đủ, từ đó hỗ trợ xây dựng thêm các trường hợp kiểm tra nhằm đáp ứng các mục tiêu kiểm chứng đã đề ra [9], [10].

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CỦA HỆ THỐNG

Để triển khai thành công IP UART tích hợp giao tiếp AHB-Lite, cần xác định rõ các yêu cầu hệ thống nhằm bảo đảm tính đúng đắn trong truyền nhận, khả năng cấu hình linh hoạt và mức độ tích hợp phù hợp với kiến trúc SoC. Các yêu cầu được chia thành bốn nhóm chính:

3.1.1 Yêu cầu người dùng

- Khối UART IP cần tích hợp sẵn giao diện chuẩn AHB-Lite để kết nối thẳng vào hệ thống SoC nhằm giúp tối ưu hóa diện tích chip và rút ngắn thời gian thiết kế mà không cần qua mạch chuyển đổi trung gian.
- Thiết kế cần đi kèm tài liệu Register Map mô tả chi tiết địa chỉ offset, thuộc tính đọc hoặc ghi cùng chức năng của từng bit để các kỹ sư firmware có thể lập trình driver điều khiển một cách chính xác.
- IP phải hỗ trợ cơ chế sinh tín hiệu ngắt chủ động và phân tách rõ ràng các cờ báo lỗi trạng thái trên các bit riêng biệt giúp kỹ sư hệ thống dễ dàng theo dõi cũng như khoanh vùng debug nhanh chóng khi xảy ra sự cố.
- Thiết kế phải được xác thực nghiêm ngặt qua môi trường UVM chuẩn công nghiệp nhằm bao phủ toàn bộ các kịch bản ngẫu nhiên và góc khuất chức năng, đảm bảo độ tin cậy tuyệt đối trước khi đưa khối IP này vào luồng thiết kế SoC thực tế.

3.1.2 Yêu cầu chức năng

- Hệ thống hỗ trợ truyền và nhận dữ liệu nối tiếp một cách độc lập qua hai đường tín hiệu TX và RX theo cơ chế song công hoàn chỉnh.
- Khối IP UART tương thích tốt với giao tiếp bus AHB-Lite và hỗ trợ đầy đủ các tín hiệu chuẩn giao thức để thực hiện chính xác các giao dịch đọc hoặc ghi vào hệ thống thanh ghi nội bộ.

- Người dùng có thể cấu hình linh hoạt các tham số hoạt động của UART như tốc độ baud rate, độ dài khung dữ liệu, chế độ parity và số bit stop thông qua các thanh ghi điều khiển.
- Việc tích hợp sẵn các bộ đệm FIFO truyền và nhận giúp lưu trữ tạm thời dữ liệu nhằm giải quyết triệt để vấn đề mất mát thông tin khi có sự bất đồng bộ về tốc độ giữa CPU và đường truyền UART.
- Hệ thống chịu trách nhiệm quản lý và cập nhật liên tục các cờ trạng thái như FIFO đầy, FIFO rỗng hay lỗi parity, đồng thời chủ động sinh tín hiệu ngắt kịp thời để thông báo cho CPU xử lý các sự kiện quan trọng.

3.1.3 Yêu cầu kỹ thuật và kiến trúc

- Thiết kế đảm bảo sự tương thích hoàn toàn với đặc tả AHB-Lite thuộc chuẩn AMBA nhằm giúp khối IP dễ dàng tích hợp vào các kiến trúc SoC hiện đại ngày nay.
- Hệ thống được tổ chức theo cấu trúc phân tầng mô-đun tách biệt rõ ràng giữa khối giao tiếp AHB-Lite, khối cầu nối sang APB và các khối chức năng UART nội bộ để tạo sự thuận lợi cho việc bảo trì hoặc mở rộng sau này.
- Toàn bộ hệ thống thanh ghi điều khiển và trạng thái đều được ánh xạ vào một vùng không gian địa chỉ cụ thể giúp CPU có thể thực hiện việc truy cập đồng nhất qua bus.
- Tất cả các khối chức năng bên trong đều hoạt động đồng bộ theo hệ thống xung nhịp clock chung và hỗ trợ tín hiệu reset tích cực mức thấp để bảo đảm quá trình khởi tạo trạng thái ban đầu diễn ra chính xác.

3.1.4 Yêu cầu hiệu năng

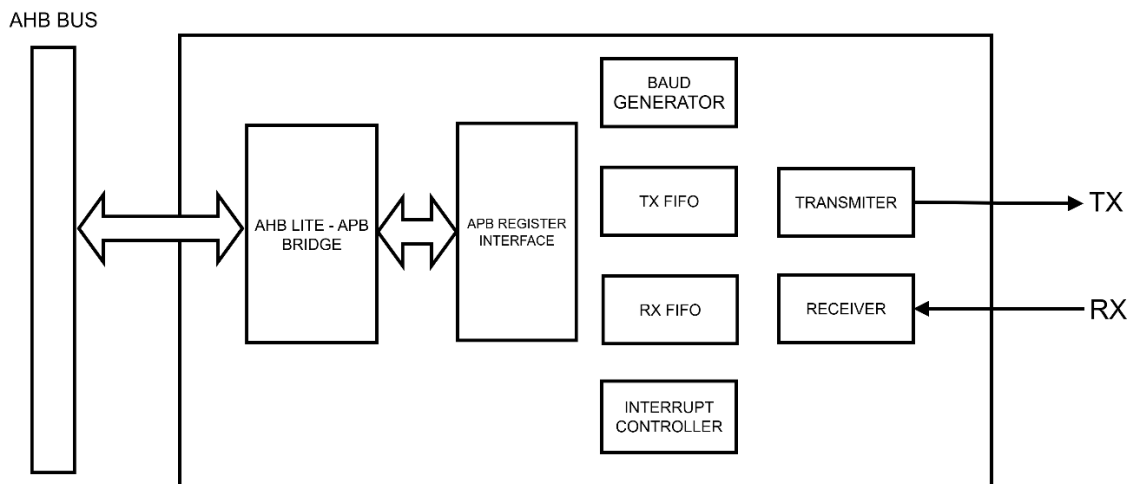
- Hệ thống đáp ứng khả năng cấu hình đa tốc độ thông qua bộ tạo baud rate có thể lập trình linh hoạt bằng thanh ghi nhằm tương thích tốt với nhiều tốc độ truyền nhận khác nhau.
- Khối giao tiếp bus cần phải phản hồi các tín hiệu điều khiển đúng chu kỳ nhằm bảo đảm mọi giao dịch đọc và ghi diễn ra chính xác mà không làm nghẽn mạch bus của toàn hệ thống.

- Cơ chế vận hành của bộ đệm FIFO phải đảm bảo quá trình trao đổi dữ liệu luôn diễn ra liên tục, xuyên suốt và không bị gián đoạn hay gặp các lỗi về tràn hoặc thiếu dữ liệu đệm.
- Các cờ hiệu trạng thái và tín hiệu ngắt cần được cập nhật tức thời để CPU có thể nhận biết và phản ứng nhanh chóng với mọi sự kiện phát sinh trong suốt quá trình hoạt động.

3.2 THIẾT KẾ HỆ THỐNG

3.2.1 Sơ đồ khối tổng quát

AHB Lite-UART IP bao gồm nhiều khối chính được kết nối với nhau. Sơ đồ khối của IP được thể hiện theo như hình 3.1:



Hình 3.1: Sơ đồ khối của AHB Lite – UART IP

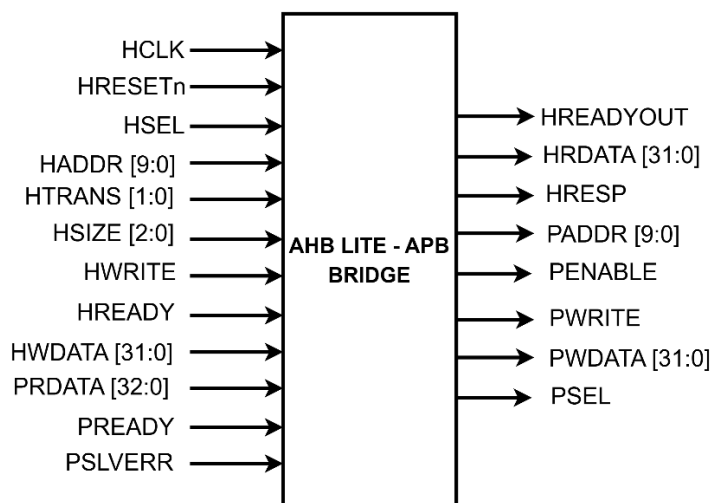
Dựa vào các tính năng đã đề ra như trên, các khối chính của IP bao gồm:

- Khối AHB Lite to APB Bridge: Thực hiện chuyển đổi giao dịch từ giao thức AHB Lite sang APB, đồng thời tạo các tín hiệu điều khiển và phản hồi phù hợp giữa hai bus.
- Khối APB Register Interface: Thực hiện giải mã địa chỉ, quản lý các thanh ghi cấu hình và trạng thái của UART, đồng thời trao đổi dữ liệu giữa bus APB và các khối chức năng bên trong hệ thống.

- Khối Baud Generator: Tạo tín hiệu baud clock dựa trên giá trị hệ số chia được cấu hình. Tín hiệu này được sử dụng để đồng bộ hoạt động của khối truyền và khối nhận UART.
- Khối TX FIFO: Lưu trữ tạm thời dữ liệu cần truyền từ hệ thống trước khi được đưa tới khối phát UART.
- Khối UART Transmitter: Nhận dữ liệu từ TX FIFO, tạo khung dữ liệu UART và truyền dữ liệu nối tiếp ra chân TX.
- Khối RX FIFO: Lưu trữ tạm thời dữ liệu nhận được từ khối UART Receiver để hệ thống có thể đọc thông qua giao diện thanh ghi.
- Khối UART Receiver: Nhận dữ liệu nối tiếp từ chân RX, thực hiện giải mã khung dữ liệu UART và chuyển dữ liệu nhận được tới RX FIFO.
- Khối Interrupt Controller: Giám sát các tín hiệu trạng thái và sự kiện từ các khối chức năng, đồng thời tạo tín hiệu ngắt phục vụ việc xử lý của hệ thống.

3.2.2 Thiết kế khối AHB Lite to APB Bridge

Khối cầu nối AHB Lite sang APB đóng vai trò kết nối giữa bus AHB Lite và ngoại vi UART sử dụng giao thức APB. Khối thực hiện chuyển đổi giao dịch giữa hai giao thức và điều khiển quá trình trao đổi địa chỉ, dữ liệu cũng như tín hiệu điều khiển. Sơ đồ khối tổng quát của cầu nối AHB Lite sang APB được trình bày trong hình 3.2:



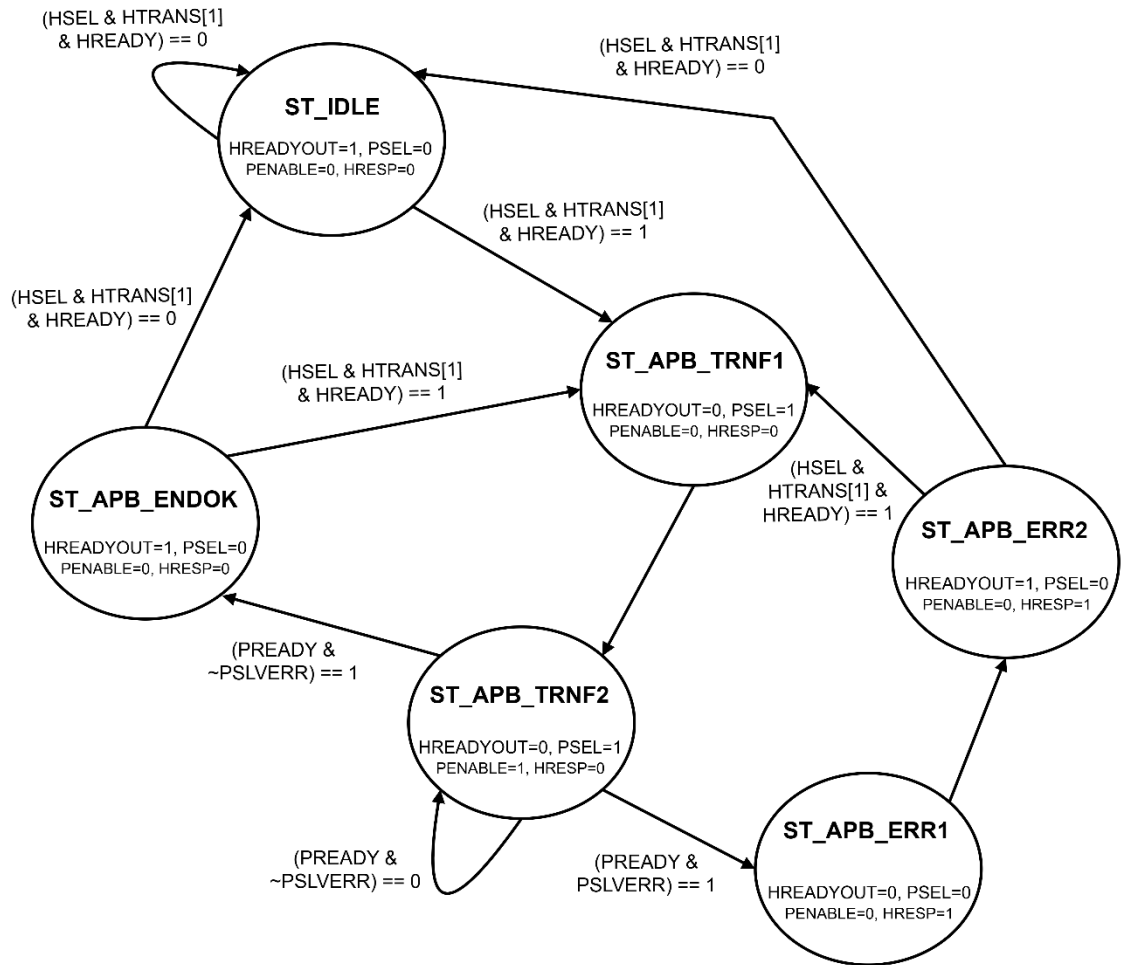
Hình 3.2: Sơ đồ tổng quát khối AHB Lite to APB Bridge

Tín hiệu của khối AHB Lite to APB Bridge được mô tả theo bảng 3.1:

Bảng 3.1: Mô tả tín hiệu khối AHB Lite to APB Bridge

Tên tín hiệu	Loại tín hiệu	Mô tả
HCLK	Tín hiệu vào	Xung clock hệ thống đồng bộ cho cả AHB Lite và APB.
HRESETn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp.
HSEL	Tín hiệu vào	Tín hiệu chọn thiết bị từ hệ thống bus AHB Lite.
HADDR [9:0]	Tín hiệu vào	Địa chỉ giao dịch AHB Lite.
HTRANS [1:0]	Tín hiệu vào	Loại giao dịch AHB Lite.
HSIZE [2:0]	Tín hiệu vào	Kích thước dữ liệu truyền.
HWRITE	Tín hiệu vào	Xác định giao dịch đọc hoặc ghi.
HREADY	Tín hiệu vào	Báo hiệu giao dịch AHB Lite trước đã hoàn tất.
HWDATA [31:0]	Tín hiệu vào	Dữ liệu ghi từ AHB Lite Master.
HREADYOUT	Tín hiệu ra	Báo trạng thái sẵn sàng của Bridge.
HRDATA [31:0]	Tín hiệu ra	Dữ liệu đọc trả về AHB Lite Master.
HRESP	Tín hiệu ra	Tín hiệu phản hồi lỗi AHB Lite.
PADDR [9:0]	Tín hiệu ra	Địa chỉ truy cập APB.
PENABLE	Tín hiệu ra	Tín hiệu Enable của APB.
PWRITE	Tín hiệu ra	Xác định giao dịch đọc hoặc ghi trên APB
PWDATA [31:0]	Tín hiệu ra	Dữ liệu ghi tới APB Slave.
PSEL	Tín hiệu ra	Tín hiệu chọn APB Slave.
PRDATA [31:0]	Tín hiệu vào	Dữ liệu đọc từ APB Slave.
PREADY	Tín hiệu vào	Báo hiệu APB Slave hoàn thành giao dịch.
PSLVERR	Tín hiệu vào	Tín hiệu báo lỗi từ APB Slave.

3.2.2.1 Sơ đồ máy trạng thái của khối điều khiển trong cầu nối AHB Lite sang APB



Hình 3.3: Sơ đồ máy trạng thái của khối điều khiển

Hình 3.3 trình bày kiến trúc trạng thái (FSM) của khối điều khiển trong cầu nối AHB Lite sang APB, đảm nhiệm vai trò chuyển đổi giao thức giữa bus AHB Lite và bus APB. FSM nhận các tín hiệu HSEL, HTRANS [1], HREADY từ phía AHB Lite để xác định giao dịch hợp lệ ở pha địa chỉ, cùng PREADY, PSLVERR từ slave APB để xác định kết quả giao dịch. Đầu ra gồm HREADYOUT, HRESP cấp cho AHB Lite master và PSEL, PENABLE cấp cho slave APB, với giá trị cố định theo từng trạng thái.

Tại trạng thái ST_IDLE, FSM chờ giao dịch mới với HREADYOUT = 1, PSEL = 0, PENABLE = 0, HRESP = 0. Khi (HSEL & HTRANS [1] & HREADY) == 1, FSM chuyển sang ST_APB_TRNF1, ngược lại tự lặp lại tại ST_IDLE.

Tại trạng thái ST_APB_TRNF1, ứng với pha SETUP của APB, FSM thiết lập PSEL=1, PENABLE = 0, HREADYOUT = 0 để giữ AHB Lite master chờ, và luôn chuyển không điều kiện sang ST_APB_TRNF2.

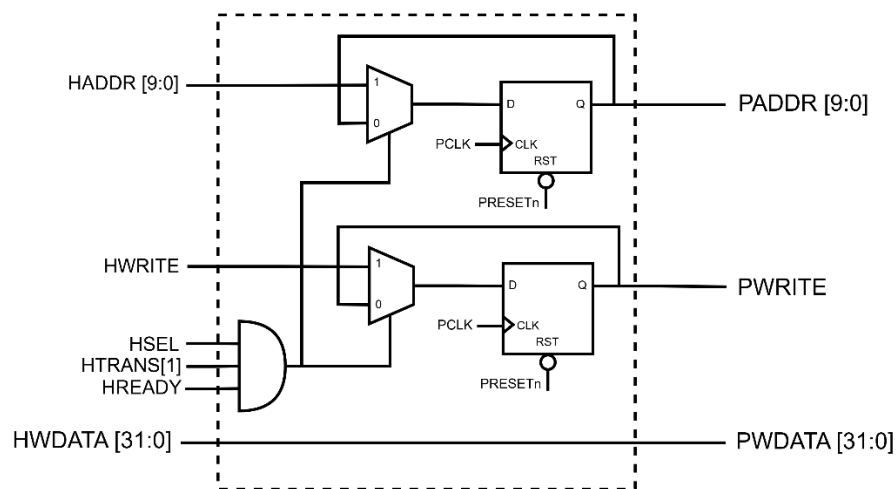
Tại trạng thái ST_APB_TRNF2, ứng với pha ENABLE của APB, FSM thiết lập PSEL = 1, PENABLE = 1, HREADYOUT = 0, và chờ phản hồi từ slave. Nếu (PREADY & ~PSLVERR) == 0, FSM tự lặp lại để tạo wait state; nếu (PREADY & ~PSLVERR) == 1, chuyển sang ST_APB_ENDOK; nếu (PREADY & PSLVERR) == 1, chuyển sang ST_APB_ERR1.

Tại trạng thái ST_APB_ENDOK, FSM thiết lập PSEL = 0, PENABLE = 0, kéo HREADYOUT = 1 để dành một chu kỳ chốt dữ liệu đọc. Nếu có giao dịch mới hợp lệ, FSM chuyển ngay sang ST_APB_TRNF1, nếu không sẽ quay về ST_IDLE.

Tại trạng thái ST_APB_ERR1, FSM thiết lập PSEL = 0, PENABLE = 0, kéo HRESP = 1 để bắt đầu báo lỗi, HREADYOUT vẫn giữ 0, và luôn chuyển không điều kiện sang ST_APB_ERR2.

Tại trạng thái ST_APB_ERR2, FSM tiếp tục giữ HRESP = 1, đồng thời kéo HREADYOUT = 1 để hoàn tất đáp ứng lỗi hai chu kỳ theo đúng quy định AHB, PSEL = 0, PENABLE = 0. Nếu có giao dịch mới hợp lệ, FSM chuyển sang ST_APB_TRNF1, nếu không thì quay về ST_IDLE.

3.2.2.2 Khởi chốt địa chỉ và tín hiệu điều khiển AHB Lite

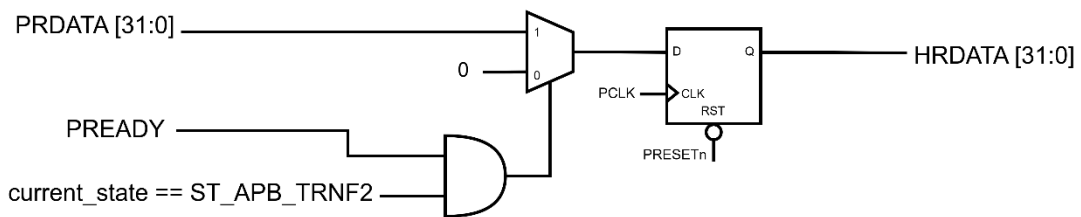


Hình 3.4: Sơ đồ mạch khởi chốt địa chỉ và tín hiệu điều khiển AHB Lite

Hình 3.4 mô tả cấu trúc phần cứng của khối chốt tín hiệu điều khiển và dữ liệu giữa AHB Lite và APB. Khối có nhiệm vụ lưu giữ các tín hiệu địa chỉ và điều khiển từ phía AHB Lite để tạo các tín hiệu tương ứng trên giao diện APB.

Các tín hiệu HSEL, HTRANS [1] và HREADY được kết hợp để nhận biết giao dịch AHB Lite hợp lệ. Khi điều kiện này được thỏa mãn, địa chỉ HADDR [9:0] và tín hiệu điều khiển ghi HWRITE được chốt vào các thanh ghi đồng bộ và xuất ra dưới dạng PADDR [9:0] và PWRITE. Trong khi đó, dữ liệu ghi HWDATA [31:0] được truyền trực tiếp tới PWDATA [31:0] mà không qua thanh ghi trung gian. Cấu trúc này đảm bảo các tín hiệu địa chỉ và điều khiển được duy trì ổn định trong suốt quá trình truy cập APB, đồng thời cho phép dữ liệu ghi được chuyển tiếp trực tiếp từ AHB Lite sang APB.

3.2.2.3 khối chốt dữ liệu đọc APB

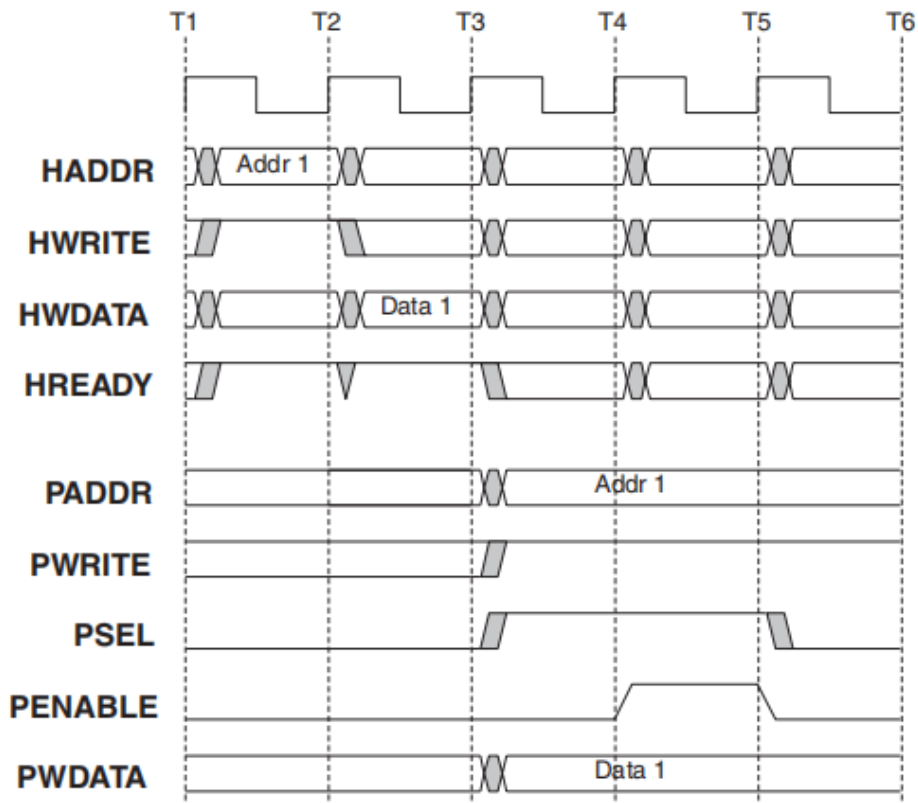


Hình 3.5: Sơ đồ mạch khối chốt dữ liệu đọc APB

Hình 3.5 mô tả cấu trúc phần cứng của khối chốt dữ liệu đọc APB. Khối có nhiệm vụ lưu trữ dữ liệu đọc từ APB slave và cung cấp dữ liệu phản hồi cho bus AHB Lite thông qua tín hiệu HRDATA [31:0].

Tín hiệu PRDATA [31:0] được đưa vào thanh ghi thông qua bộ chọn dữ liệu. Tín hiệu điều khiển bộ chọn được tạo từ phép AND giữa PREADY và điều kiện FSM đang ở trạng thái ST_APB_TRNF2. Khi giao dịch APB hoàn thành, dữ liệu đọc được chốt vào thanh ghi tại cạnh lên của xung clock và xuất ra HRDATA [31:0]. Cấu trúc này đảm bảo dữ liệu phản hồi chỉ được cập nhật khi giao dịch APB hoàn tất hợp lệ và được giữ ổn định cho phía AHB Lite.

3.2.2.4 Giảm độ thời gian giao dịch ghi và đọc



Hình 3.6: Giảm độ thời gian giao dịch ghi

Tại T1, AHB Lite phát sinh giao dịch ghi với địa chỉ Addr1, tín hiệu HWRITE được thiết lập ở mức cao.

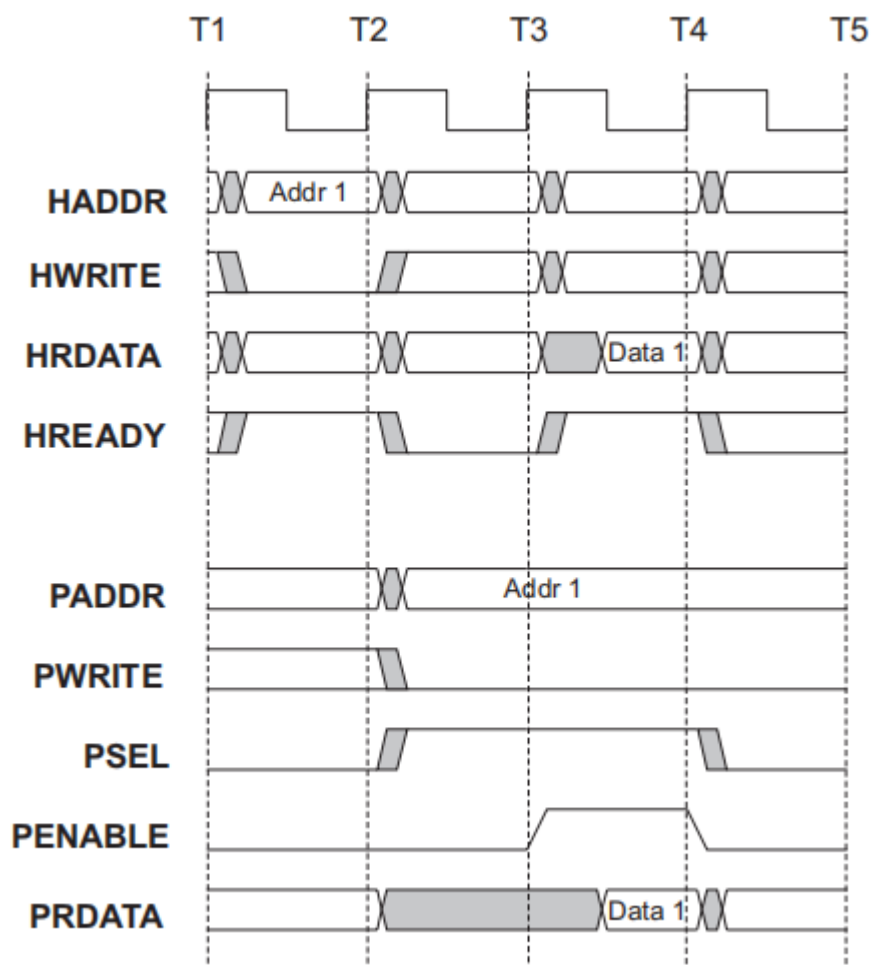
Tại T2, Dữ liệu ghi Data1 xuất hiện trên bus HWDATA, cầu nối chốt địa chỉ và tín hiệu điều khiển từ phía AHB Lite.

Tại T3, Pha Setup của APB được bắt đầu, các tín hiệu PADDR, PWRITE và PSEL được kích hoạt, đồng thời dữ liệu Data1 được đưa tới PWDATA.

Tại T4, Pha Access của APB được thực hiện khi PENABLE được thiết lập. APB slave thực hiện ghi dữ liệu trong chu kỳ này.

Tại T5, Giao dịch APB hoàn tất, PSEL và PENABLE được hủy, tín hiệu HREADY trở lại mức cao để cho phép AHB Lite tiếp tục giao dịch mới.

Tại T6, Hệ thống trở về trạng thái chờ và sẵn sàng xử lý các giao dịch tiếp theo.



Hình 3.7: Giảm đồ thời gian giao dịch đọc

Tại T1, AHB Lite master phát sinh giao dịch đọc với địa chỉ Addr1. Tín hiệu HWRITE ở mức thấp để chỉ ra đây là giao dịch đọc.

Tại T2, Cầu nối chốt địa chỉ từ AHB Lite và bắt đầu pha Setup của APB bằng cách thiết lập PADDR = Addr1, PSEL = 1 và PWRITE = 0. Đồng thời HREADY được kéo xuống để giữ AHB Lite master chờ phản hồi.

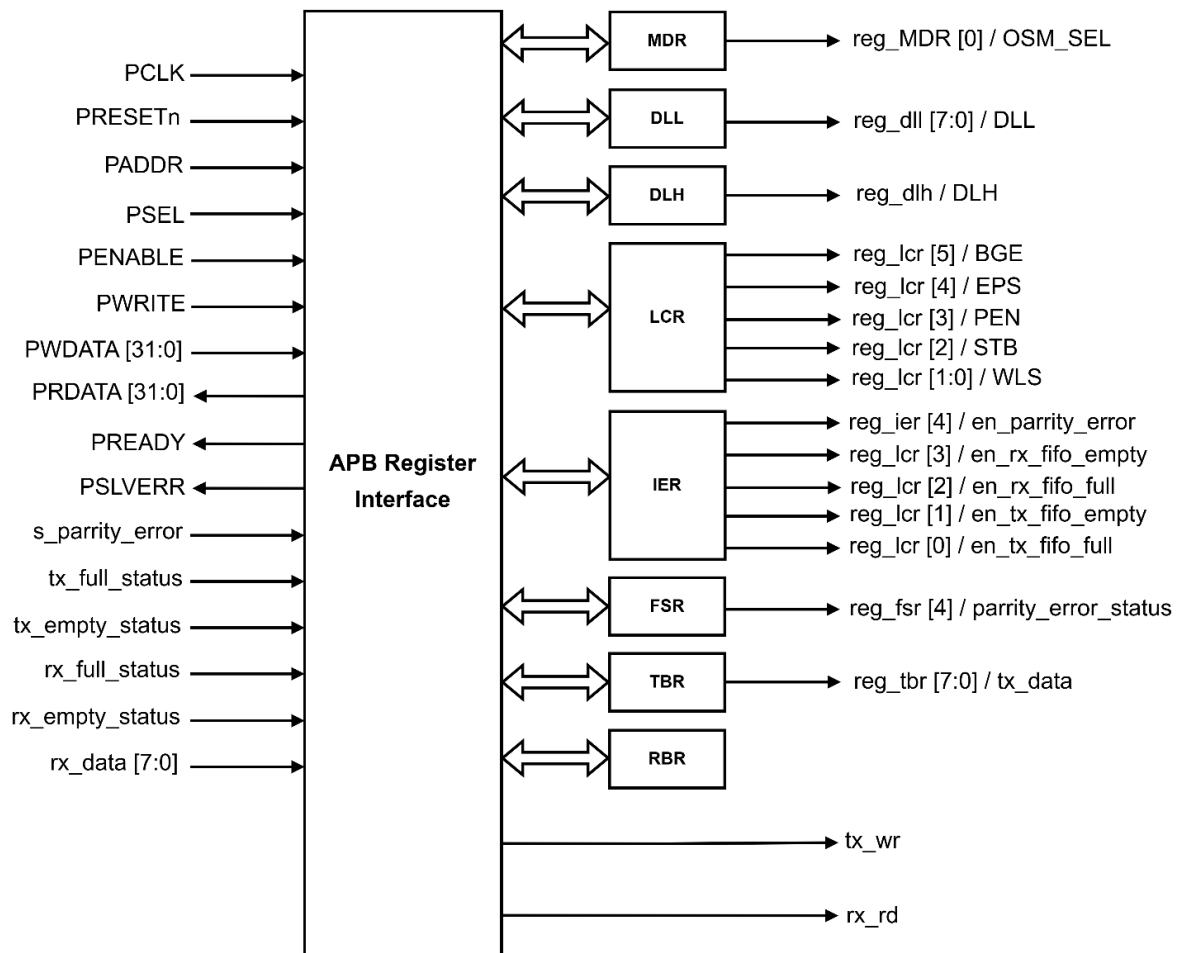
Tại T3, Pha Enable của APB được thực hiện khi PENABLE được kích hoạt. APB slave truy xuất dữ liệu tại địa chỉ yêu cầu và đưa dữ liệu lên bus PRDATA.

Tại T4, Dữ liệu đọc Data1 được cầu nối nhận từ PRDATA, lưu vào thanh ghi dữ liệu đọc và xuất lên HRDATA. Giao dịch đọc được hoàn tất.

Tại T5, Các tín hiệu PSEL và PENABLE được hủy, HREADY trở lại mức cao để cho phép AHB Lite master tiếp tục thực hiện các giao dịch tiếp theo.

3.2.3 Thiết kế khối APB Register Interface

Khối APB Register Interface đóng vai trò là giao diện kết nối giữa cầu nối AHB Lite sang APB và các khối chức năng bên trong UART. Khối thực hiện giải mã địa chỉ truy cập, quản lý các thanh ghi điều khiển và trạng thái, đồng thời trao đổi dữ liệu với các khối UART FIFO, UART Transmitter, UART Receiver, Baud Generator và UART Interrupt Controller. Sơ đồ khối tổng quát của APB Register Interface được trình bày trong Hình 3.6.



Hình 3.8: Sơ đồ khối tổng quát APB Register Interface

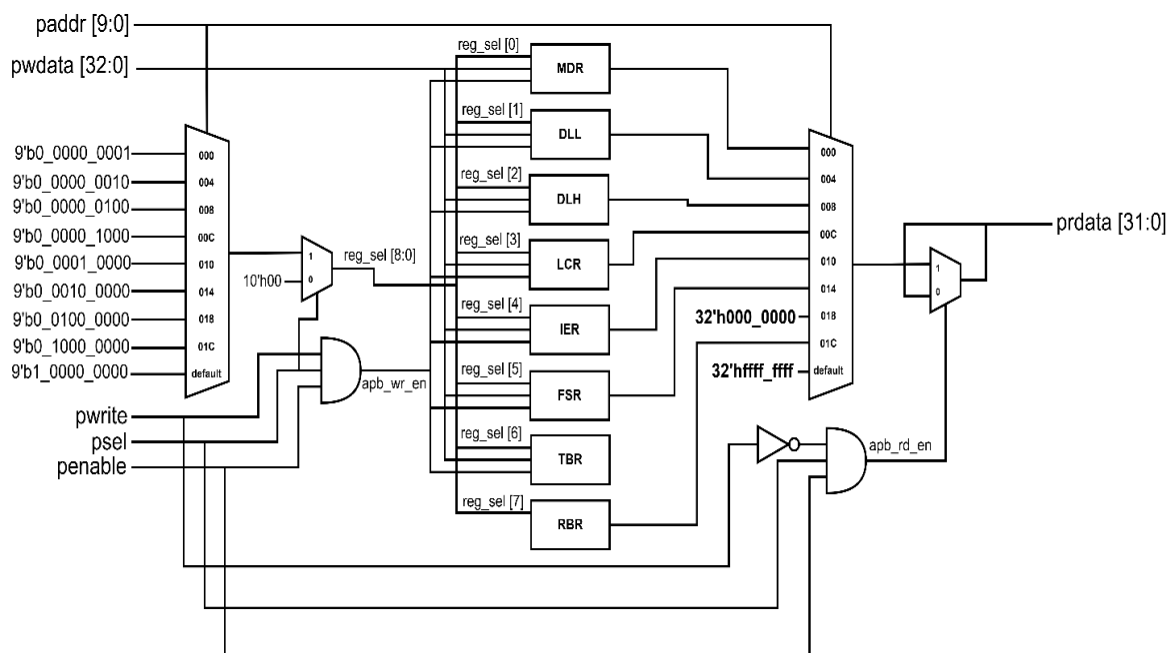
Tín hiệu của khối APB Register Interface được mô tả theo bảng 3.2:

Bảng 3.2: Mô tả tín hiệu khối APB Register Interface

Tên tín hiệu	Loại tín hiệu	Mô tả
pclk	Tín hiệu vào	Xung clock hệ thống đồng bộ.
presetn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp.
paddr [9:0]	Tín hiệu vào	Bus địa chỉ từ master APB truyền đến.
psel	Tín hiệu vào	Tín hiệu chọn ngoại vi từ bus APB.
penable	Tín hiệu vào	Tín hiệu cho phép chu kỳ truyền dữ liệu APB.
pwrite	Tín hiệu vào	Tín hiệu định hướng truy cập (1: Ghi, 0: Đọc).
pwdata [31:0]	Tín hiệu vào	Bus dữ liệu ghi nhận từ bus APB.
pready	Tín hiệu ra	Tín hiệu báo sẵn sàng phản hồi
prdata [31:0]	Tín hiệu ra	Bus dữ liệu đọc trả về cho bus APB.
pslverr	Tín hiệu ra	Tín hiệu báo lỗi khi truy cập sai vùng địa chỉ.
osm_sel	Tín hiệu ra	Tín hiệu chọn chế độ oversampling.
dll [7:0]	Tín hiệu ra	Byte thấp cấu hình hệ số chia Baud.
dlh [7:0]	Tín hiệu ra	Byte cao cấu hình hệ số chia Baud.
bge	Tín hiệu ra	Tín hiệu cho phép bộ tạo tốc độ Baud.
eps	Tín hiệu ra	Tín hiệu cấu hình kiểu Parity chẵn/lẻ.
pen	Tín hiệu ra	Tín hiệu cho phép kiểm tra lỗi Parity.
stb	Tín hiệu ra	Cấu hình số lượng Stop bit.
wls [1:0]	Tín hiệu ra	Cấu hình độ rộng từ dữ liệu UART.
en_tx_fifo_full	Tín hiệu ra	Tín hiệu cho phép ngắt khi TX FIFO đầy.
en_tx_fifo_empty	Tín hiệu ra	Tín hiệu cho phép ngắt khi TX FIFO rỗng.
en_rx_fifo_full	Tín hiệu ra	Tín hiệu cho phép ngắt khi RX FIFO đầy.
en_rx_fifo_empty	Tín hiệu ra	Tín hiệu cho phép ngắt khi RX FIFO rỗng.
en_parrity_error	Tín hiệu ra	Tín hiệu cho phép ngắt khi lỗi Parity.
s_parrity_error	Tín hiệu vào	Tín hiệu báo trạng thái lỗi Parity từ khối nhận.
parrity_error_status	Tín hiệu ra	Cờ lưu trạng thái lỗi Parity trong hệ thống.
tx_full_status	Tín hiệu vào	Cờ báo trạng thái đầy từ khối TX FIFO.
tx_empty_status	Tín hiệu vào	Cờ báo trạng thái rỗng từ khối TX FIFO.

tx_wr	Tín hiệu ra	Tín hiệu điều khiển ghi dữ liệu vào TX FIFO.
tx_data	Tín hiệu ra	Dữ liệu truyền gửi sang khối TX FIFO.
rx_full_status	Tín hiệu vào	Cờ báo trạng thái đầy từ khối RX FIFO.
rx_empty_status	Tín hiệu vào	Cờ báo trạng thái rỗng từ khối RX FIFO.
rx_data	Tín hiệu vào	Dữ liệu nhận đọc từ khối RX FIFO.
rx_rd	Tín hiệu ra	Tín hiệu điều khiển đọc dữ liệu từ RX FIFO.

3.2.3.1 Giải mã địa chỉ và truy cập thanh ghi trong APB Register Interface

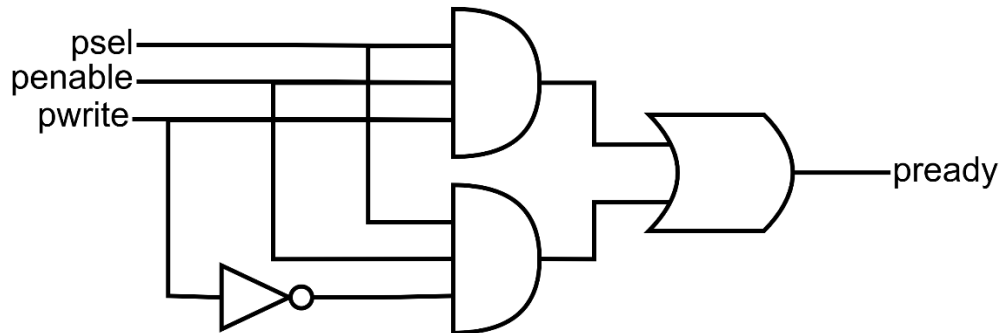


Hình 3.9: Sơ đồ giải mã địa chỉ và truy cập thanh ghi

Hình 3.9 mô tả cấu trúc phần cứng của khối APB Register Interface, đóng vai trò giao tiếp giữa bus APB và các thanh ghi điều khiển, trạng thái bên trong UART. Khối thực hiện chức năng giải mã địa chỉ truy cập, điều khiển quá trình đọc/ghi thanh ghi và cung cấp dữ liệu phản hồi cho bus APB.

Bên trong khối, địa chỉ `paddr [9:0]` được đưa vào bộ giải mã địa chỉ để tạo tín hiệu chọn thanh ghi `reg_sel [8:0]`. Các tín hiệu `reg_sel [0]` đến `reg_sel [7]` lần lượt tương ứng với các thanh ghi MDR, DLL, DLH, LCR, IER, FSR, TBR và RBR, trong khi `reg_sel [8]` biểu thị vùng địa chỉ không hợp lệ. Đồng thời, các tín hiệu điều khiển của giao thức APB được kết hợp để tạo tín hiệu ghi hợp lệ `apb_wr_en` và tín hiệu đọc hợp lệ `apb_rd_en`.

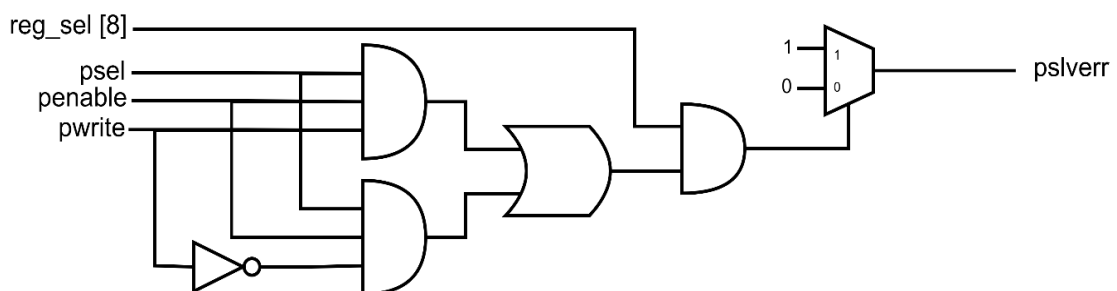
Trong quá trình ghi, dữ liệu từ pwwdata [31:0] được ghi vào thanh ghi được lựa chọn bởi tín hiệu reg_sel. Trong quá trình đọc, nội dung của thanh ghi được chọn được đưa tới bộ chọn dữ liệu để tạo tín hiệu prdata [31:0] trả về bus APB. Đối với các địa chỉ không hợp lệ, bộ chọn dữ liệu trả về các giá trị mặc định đã được xác định trước. Nhờ đó, khối đảm bảo việc truy cập các thanh ghi UART được thực hiện đúng địa chỉ và đúng loại giao dịch.



Hình 3.10: Sơ đồ mạch tạo tín hiệu sẵn sàng APB

Hình 3.10 mô tả cấu trúc phần cứng của mạch tạo tín hiệu sẵn sàng APB. Mạch sử dụng các tín hiệu pselect, penable và pwrite để nhận biết giao dịch đọc hoặc ghi đang diễn ra, sau đó kết hợp các điều kiện này thông qua các cổng logic để tạo tín hiệu pready.

Trong thiết kế này, giao thức APB được triển khai theo cơ chế no wait-state nên pready luôn được khẳng định khi xảy ra giao dịch truy cập hợp lệ. Nhờ đó, các giao dịch đọc và ghi được hoàn thành ngay trong chu kỳ truy cập mà không phát sinh thêm trạng thái chờ.



Hình 3.11: Sơ đồ mạch tạo tín hiệu báo lỗi truy cập APB

Hình 3.11 mô tả cấu trúc phần cứng của mạch tạo tín hiệu lỗi truy cập APB. Mạch sử dụng các tín hiệu pselect, penable và pwrite để xác định một giao dịch APB

hợp lệ, sau đó kết hợp với tín hiệu `reg_sel` [8] từ bộ giải mã địa chỉ nhằm phát hiện các truy cập đến địa chỉ không hợp lệ.

Khi xảy ra truy cập sai địa chỉ, tín hiệu `pslverr` được đưa lên mức logic cao; ngược lại tín hiệu này được giữ ở mức logic thấp. Nhờ đó, mạch có thể phát hiện và thông báo lỗi truy cập theo cơ chế báo lỗi của giao thức APB.

3.2.3.2 Các thanh ghi cấu hình và trạng thái

Khối APB Register Interface thanh ghi gồm 8 thanh ghi là: MDR, DLL, DLH, LCR, IER, FSR, TBR và RBR. Các thanh ghi được mô tả theo bảng 3.3:

Bảng 3.3: Tổng hợp các thanh ghi trong khối APB Register Interface

Địa chỉ	Tên thanh ghi	Mô tả
0x000	Mode Definition Register (MDR)	Lựa chọn chế độ lấy mẫu (Oversampling Mode).
0x004	Divisor Latches Low (DLL)	Lưu 8 bit thấp của hệ số chia dùng để tạo xung baud trong bộ tạo baud.
0x008	Divisor Latches High (DLH)	Lưu 8 bit cao của hệ số chia dùng để tạo xung baud trong bộ tạo baud.
0x00C	Line Control Register (LCR)	Cấu hình khung truyền UART và điều khiển bộ tạo baud.
0x010	Interrupt Enable Register (IER)	Cho phép các nguồn ngắt phần cứng của UART.
0x014	FIFO Status Register (FSR)	Lưu trạng thái của FIFO truyền và FIFO nhận.
0x018	Transmitter Buffer Register (TBR)	Chứa dữ liệu cần truyền tới chân <code>uart_txd</code> .
0x01C	Receiver Buffer Register (RBR)	Chứa dữ liệu nhận được từ chân <code>uart_rxd</code> .

0x020 – 0x3FF	Reserved region	Thao tác ghi không có tác dụng, thao tác đọc trả về giá trị 32'hFFFFFF_FFFF.
---------------	-----------------	--

Bảng 3.3 trình bày danh sách các thanh ghi được triển khai trong UART cùng địa chỉ tương ứng. Các thanh ghi này phục vụ chức năng cấu hình bộ tạo baud, thiết lập tham số truyền thông, quản lý ngắt, giám sát trạng thái FIFO và trao đổi dữ liệu giữa UART với hệ thống thông qua giao diện APB. Các địa chỉ không được ánh xạ tới thanh ghi hợp lệ được quy định là vùng Reserved. Chi tiết các thanh ghi như sau:

- Mode Definition Register (MDR, Địa chỉ = 0x000)

Bảng 3.4: Mode Definition Register

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:1]	-	-	Reserved	
0	OSM_SEL	RW	0	Lựa chọn chế độ lấy mẫu: 1'b0: 16× Oversampling. 1'b1: 13× Oversampling.

- Divisor Latches Low (DLL, Địa chỉ = 0x004)

Bảng 3.5: Divisor Latches Low

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:8]	-	-	Reserved	
[7:0]	DLL	RW	0	Lưu 8 bit thấp của hệ số chia 16 bit dùng để tạo xung baud trong khối Baud Generator.

- Divisor Latches High (DLH, Địa chỉ = 0x008)

Bảng 3.6: Divisor Latches High

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:8]	-	-	Reserved	-
[7:0]	DLH	RW	0	Lưu 8 bit cao (MSB) của hệ số chia 16 dùng để tạo xung baud trong khối Baud Generator

– Line Control Register (LCR, Địa chỉ = 0x00C)

Bảng 3.7: Line Control Register

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:6]	-	-	Reserved	
5	BGE	RW	0	Cho phép bộ tạo baud: Giá trị 1: bật bộ tạo baud. Giá trị 0: tắt bộ tạo baud.
4	EPS	RW	0	Lựa chọn kiểu parity chẵn/lẻ khi PEN=1: Giá trị 0: Odd Parity. Giá trị 1: Even Parity.
3	PEN	RW	0	Cho phép kiểm tra và tạo bit parity: Giá trị 0: không sử dụng parity. Giá trị 1: bật parity.
2	STB	RW	0	Lựa chọn số bit stop: Giá trị 0: 1 bit stop. Giá trị 1: 2 bit stop.

[1:0]	WLS	RW	11	Lựa chọn độ dài dữ liệu truyền nhận: 00: 5 bit. 01: 6 bit. 10: 7 bit. 11: 8 bit.
-------	-----	----	----	--

– Interrupt Enable Register (IER, Địa chỉ = 0x010)

Bảng 3.8: Interrupt Enable Register

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:5]	-	-	Reserved	
4	en_parity_error	RW	0	Cho phép ngắt khi FIFO truyền đầy.
3	en_rx_fifo_empty	RW	0	Cho phép ngắt khi FIFO truyền rỗng.
2	en_rx_fifo_full	RW	0	Cho phép ngắt khi FIFO nhận đầy.
1	en_tx_fifo_empty	RW	0	Cho phép ngắt khi FIFO nhận rỗng.
0	en_tx_fifo_full	RW	0	Cho phép ngắt khi FIFO truyền đầy.

- FIFO Status Register (FSR, Địa chỉ = 0x014)

Bảng 3.9: FIFO Status Register

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:5]	-	-	Reserved	
4	parity_error_status	R/W1C	0	Trạng thái phát hiện lỗi parity của dữ liệu nhận được.
3	rx_empty_status	RO	1	Trạng thái FIFO nhận rỗng.
2	rx_full_status	RO	0	Trạng thái FIFO nhận đầy.
1	tx_empty_status	RO	0	Trạng thái FIFO truyền rỗng.
0	tx_full_status	RO	0	Trạng thái FIFO truyền đầy.

- Transmitter Buffer Register (TBR, Địa chỉ = 0x018)

Bảng 3.10: Transmitter Buffer Register

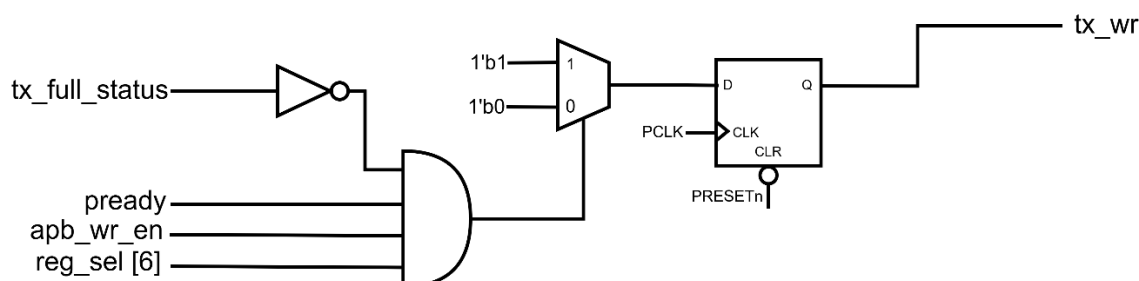
Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:8]	-	-	Reserved	
[7:0]	tx_data	WO	00	Chứa dữ liệu cần truyền đến khối UART Transmitter. Khi đọc thanh ghi này luôn trả về giá trị 8'h00.

- Receiver Buffer Register (RBR, Địa chỉ = 0x01C)

Bảng 3.11: Receiver Buffer Register

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
[31:8]	-	-	Reserved	
[7:0]	rx_data	RO	xx	Chứa dữ liệu nhận được từ khối UART Receiver. Giá trị sau khi reset không xác định.

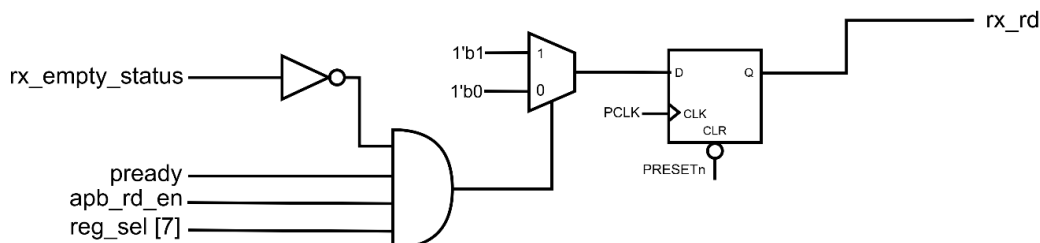
3.2.3.3 Giao tiếp với FIFO truyền và FIFO nhận



Hình 3.12: Sơ đồ mạch tạo tín hiệu ghi dữ liệu vào TX FIFO

Hình 3.12 mô tả cấu trúc phần cứng của mạch tạo tín hiệu tx_wr. Tín hiệu tx_full_status được đảo và kết hợp với các tín hiệu pready, apb_wr_en và reg_sel[6] thông qua cổng AND để xác định một giao dịch ghi hợp lệ tới thanh ghi TBR.

Kết quả của phép kết hợp logic được chốt bởi thanh ghi đồng bộ theo cạnh lên của PCLK để tạo tín hiệu tx_wr. Cơ chế này đảm bảo yêu cầu ghi chỉ được phát sinh khi TX FIFO chưa đầy và giao dịch ghi dữ liệu được thực hiện hợp lệ.



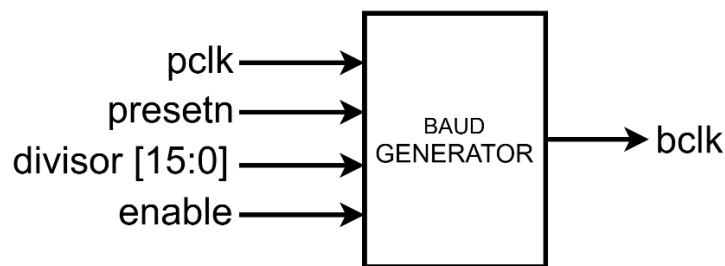
Hình 3.13: Sơ đồ mạch tạo tín hiệu đọc dữ liệu từ RX FIFO

Hình 3.13 mô tả cấu trúc phần cứng của mạch tạo tín hiệu rx_rd. Tín hiệu rx_empty_status được đảo và kết hợp với các tín hiệu pready, apb_rd_en và reg_sel[7] thông qua cổng AND để xác định một giao dịch đọc hợp lệ tới thanh ghi RBR.

Kết quả của phép kết hợp logic được chốt bởi thanh ghi đồng bộ theo cạnh lên của PCLK để tạo tín hiệu rx_rd. Nhờ đó, yêu cầu đọc dữ liệu từ RX FIFO chỉ được phát sinh khi FIFO không rỗng và giao dịch đọc được thực hiện hợp lệ.

3.2.4 Thiết kế khối Baud Generator

Khối Baud Generator có nhiệm vụ tạo xung clock bclk từ xung pclk của hệ thống bên trong. Xung bclk được sử dụng cho quá trình hoạt động của hai khối Transmitter và Receiver. Sơ đồ khối tổng quát như hình 3.12:



Hình 3.14: Sơ đồ tổng quát khối Baud Generator

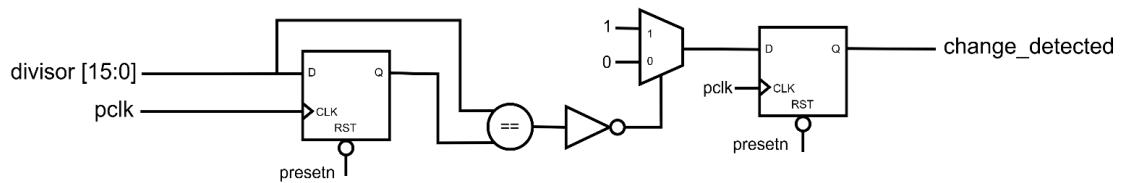
Tín hiệu của khối Baud Generator được mô tả theo bảng 3.12:

Bảng 3.12: Mô tả tín hiệu khối Baud Generator

Tên tín hiệu	Loại tín hiệu	Mô tả
pclk	Tín hiệu vào	Xung clock hệ thống, đồng bộ các khối thanh ghi và logic tuần tự bên trong mạch.
resetn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp để khởi tạo lại mạch đếm và các trạng thái lưu trữ nội bộ về giá trị ban đầu.
enable	Tín hiệu vào	Tín hiệu cho phép khối tạo tốc độ Baud hoạt động; khi đưa về mức thấp, mạch đếm sẽ giữ nguyên giá trị hiện hành.

divisor [15:0]	Tín hiệu vào	Bus giá trị cấu hình hệ số chia tần số, quyết định chu kỳ và tốc độ của xung clock Baud ngõ ra.
bclk	Tín hiệu ra	Xung clock Baud ngõ ra, tích cực mức cao trong một chu kỳ hệ thống khi mạch đếm đạt giá trị đích, dùng làm mốc đồng bộ thời gian truyền nhận cho UART.

3.2.4.1 Khối phát hiện thay đổi hệ số chia



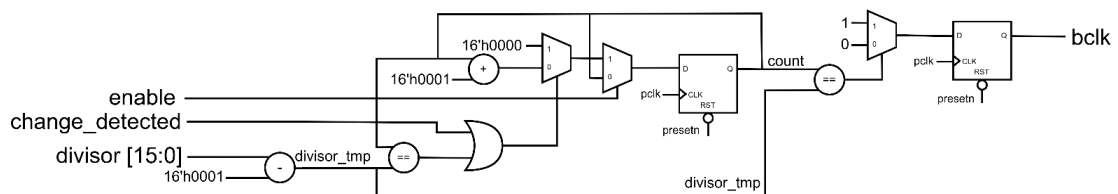
Hình 3.15: Sơ đồ mạch khối phát hiện thay đổi hệ số chia

Hình 3.15 mô tả cấu trúc phần cứng của khối phát hiện thay đổi hệ số chia. Khối này có nhiệm vụ giám sát giá trị divisor[15:0] và tạo tín hiệu change_detected khi phát hiện hệ số chia được cập nhật.

Giá trị hiện tại của divisor[15:0] được lưu trong thanh ghi và so sánh với giá trị mới thông qua bộ so sánh. Khi hai giá trị khác nhau, tín hiệu change_detected được kích hoạt và đồng bộ qua thanh ghi đầu ra trước khi đưa ra ngoài khối.

Tín hiệu này được sử dụng để thông báo cho khối tạo xung baud khởi tạo lại bộ đếm theo giá trị hệ số chia mới.

3.2.4.2 Khối tạo xung baud

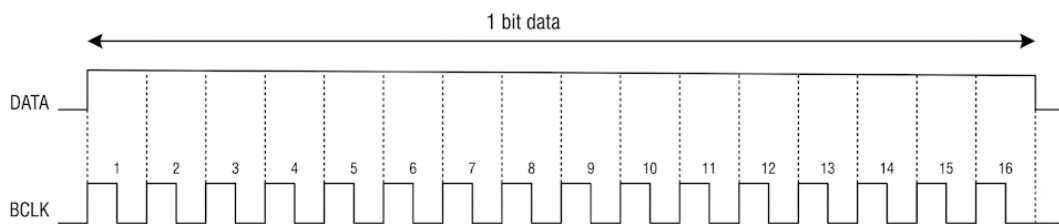


Hình 3.16: Sơ đồ mạch khối tạo xung baud

Hình 3.16 mô tả cấu trúc phần cứng của khối tạo xung baud. Khối này có nhiệm vụ tạo tín hiệu bclk dựa trên giá trị hệ số chia divisor [15:0], qua đó xác định tốc độ truyền dữ liệu của UART.

Giá trị divisor [15:0] được giảm đi một đơn vị để tạo tín hiệu divisor_tmp, đóng vai trò giá trị đích của bộ đếm. Khi tín hiệu enable được kích hoạt, bộ đếm tăng theo xung pclk và được so sánh liên tục với divisor_tmp. Khi bộ đếm đạt giá trị đích hoặc xuất hiện tín hiệu change_detected, bộ đếm được khởi tạo lại để bắt đầu chu kỳ đếm mới.

Kết quả so sánh được sử dụng để tạo tín hiệu bclk, cho phép khôi sinh ra xung baud với chu kỳ phụ thuộc vào hệ số chia đã cấu hình.

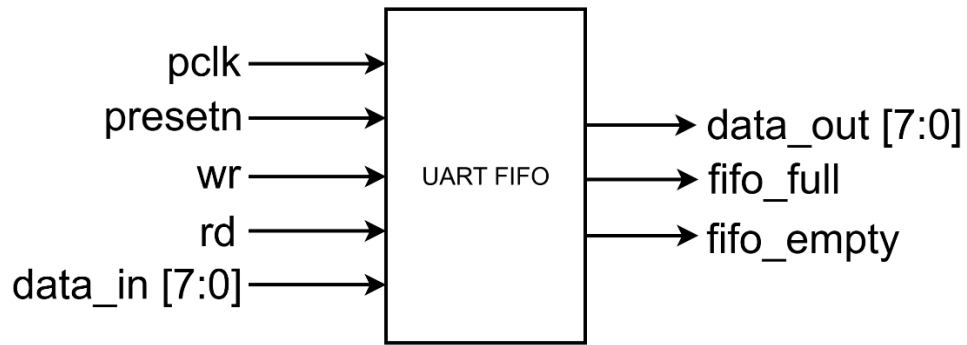


Hình 3.17: Xung bclk trong một chu kỳ bit dữ liệu

Hình 3.17 minh họa mối quan hệ giữa tín hiệu bclk và một bit dữ liệu trong UART ở chế độ oversampling 16x. Trong chế độ này, mỗi bit dữ liệu tương ứng với 16 chu kỳ bclk. Tín hiệu bclk được sử dụng làm xung tham chiếu cho quá trình truyền và nhận dữ liệu, đồng thời hỗ trợ cơ chế lấy mẫu vượt mức nhằm nâng cao độ chính xác của hệ thống UART.

3.2.5 Thiết kế khối FIFO

Trong các hệ thống truyền thông dữ liệu như UART, khối FIFO (First-In-First-Out) đóng vai trò là một vùng đệm lưu trữ tạm thời vô cùng thiết yếu. Bằng cách điều hòa luồng dữ liệu vào và ra, FIFO giải quyết triệt để bài toán chênh lệch tốc độ giữa bên ghi và bên đọc, từ đó ngăn ngừa hiện tượng mất mát dữ liệu. Sơ đồ khối tổng quát như hình 3.16:



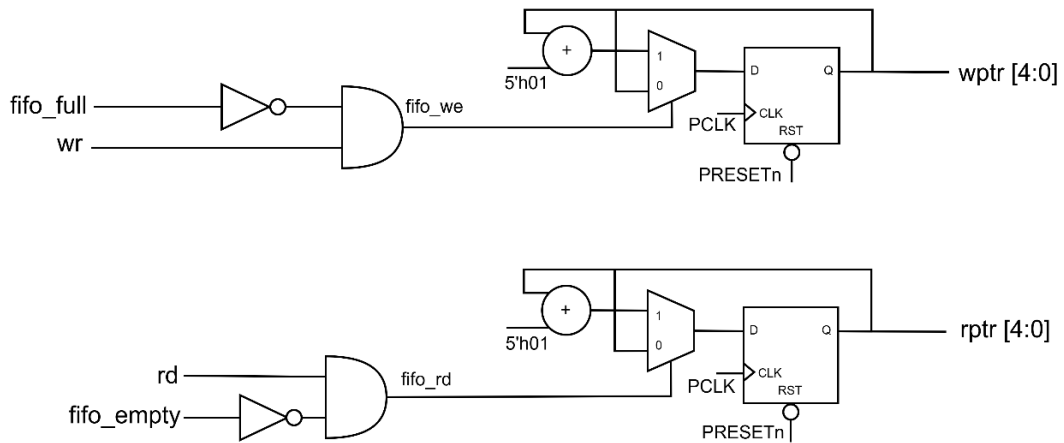
Hình 3.18: Sơ đồ tổng quát khối FIFO

Tín hiệu của khối FIFO được mô tả theo bảng 3.13:

Bảng 3.13: Mô tả tín hiệu khối FIFO

Tên tín hiệu	Loại tín hiệu	Mô tả
pclk	Tín hiệu vào	Xung clock hệ thống, đồng bộ toàn bộ các phần tử lưu trữ và logic tuần tự trong mạch.
presetn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp, khởi tạo giá trị các thanh ghi về trạng thái ban đầu.
wr	Tín hiệu vào	Tín hiệu yêu cầu ghi dữ liệu từ ngoại vi vào bộ đệm FIFO.
rd	Tín hiệu vào	Tín hiệu yêu cầu đọc dữ liệu từ bộ đệm FIFO ra ngoại vi.
data_in [7:0]	Tín hiệu vào	Bus dữ liệu ngõ vào mang giá trị cần lưu trữ vào FIFO.
data_out [7:0]	Tín hiệu ra	Bus dữ liệu ngõ ra mang giá trị được đọc từ bộ nhớ FIFO.
fifo_full	Tín hiệu ra	Cờ báo trạng thái bộ nhớ đệm đã đầy hoàn toàn, ngăn chặn hành vi ghi đè.
fifo_empty	Tín hiệu ra	Cờ báo trạng thái bộ nhớ đệm đang rỗng, ngăn chặn hành vi đọc sai lệch dữ liệu.

3.2.5.1 Khối điều khiển con trỏ đọc và ghi FIFO

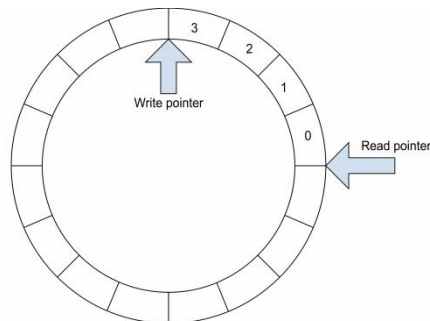


Hình 3.19: Sơ đồ mạch khối điều khiển con trỏ đọc và ghi FIFO

Hình 3.17 mô tả cấu trúc của khối điều khiển con trỏ ghi và con trỏ đọc trong FIFO. Khối này sử dụng hai con trỏ độc lập $wptr [4:0]$ và $rptr [4:0]$ để quản lý vị trí ghi và đọc dữ liệu trong bộ nhớ FIFO.

Đối với quá trình ghi, tín hiệu fif_we được tạo từ tín hiệu yêu cầu ghi wr và trạng thái FIFO chưa đầy. Khi fif_we được kích hoạt, con trỏ ghi được tăng thêm một đơn vị và cập nhật tại cạnh lên của xung PCLK, ngược lại giá trị con trỏ được giữ nguyên.

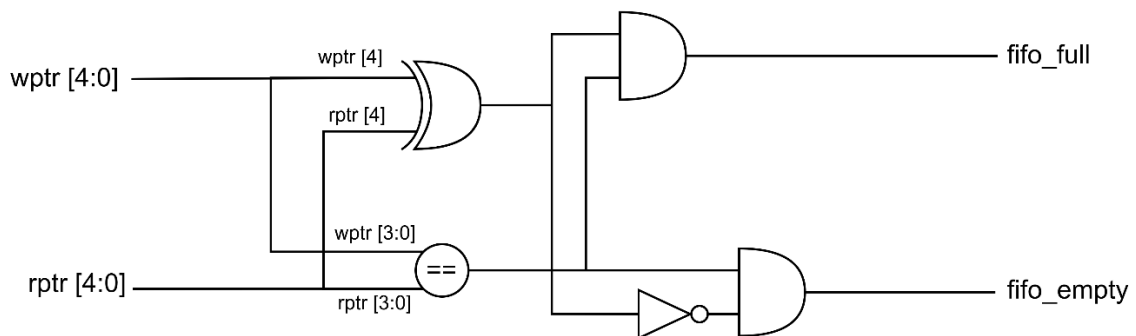
Tương tự, tín hiệu fif_rd được tạo từ tín hiệu yêu cầu đọc rd và trạng thái FIFO không rỗng. Khi fif_rd được kích hoạt, con trỏ đọc được tăng thêm một đơn vị và cập nhật theo xung clock. Cả hai con trỏ đều được khởi tạo về giá trị ban đầu khi tín hiệu $PRESETn$ được kích hoạt. Cơ chế này cho phép FIFO quản lý tuần tự vị trí đọc và ghi dữ liệu theo nguyên tắc bộ đệm vòng.



Hình 3.20: Cơ chế vòng đệm của FIFO

Hình 3.20 minh họa cơ chế bộ đệm vòng của FIFO. Bộ nhớ FIFO được truy cập thông qua hai con trỏ độc lập là con trỏ ghi và con trỏ đọc. Khi có thao tác ghi hoặc đọc hợp lệ, các con trỏ sẽ dịch sang vị trí kế tiếp và quay trở lại đầu bộ nhớ khi đạt đến ô cuối cùng. Cơ chế này cho phép tái sử dụng các ô nhớ đã đọc mà không cần dịch chuyển dữ liệu, đồng thời đảm bảo dữ liệu được xử lý theo nguyên tắc vào trước ra trước.

3.2.5.2 Khối xác định trạng thái FIFO

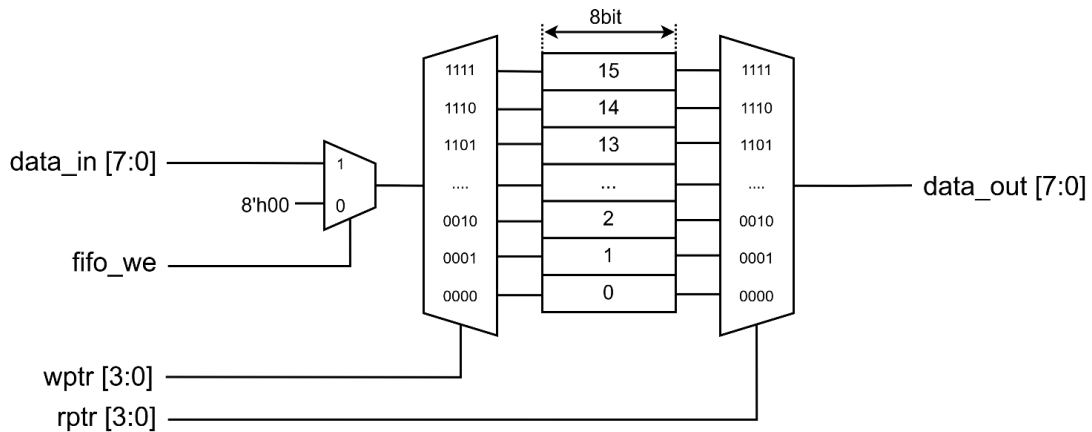


Hình 3.21: Sơ đồ mạch Khối xác định trạng thái FIFO

Hình 3.21 mô tả cấu trúc phần cứng của khối xác định trạng thái FIFO. Khối này sử dụng con trỏ ghi wptr [4:0] và con trỏ đọc rptr [4:0] để tạo ra hai tín hiệu trạng thái fifo_full và fifo_empty.

Phần địa chỉ thấp wptr [3:0] và rptr [3:0] được so sánh để kiểm tra sự trùng khớp vị trí truy cập, trong khi bit cao nhất wptr [4] và rptr [4] được đưa vào cổng XOR để xác định trạng thái vòng lặp của hai con trỏ. Từ các tín hiệu này, fifo_empty được kích hoạt khi hai con trỏ hoàn toàn trùng nhau, còn fifo_full được kích hoạt khi phần địa chỉ bằng nhau nhưng bit vòng lặp khác nhau. Cơ chế này cho phép phân biệt chính xác trạng thái đầy và trống của FIFO.

3.2.5.3 Khối bộ nhớ FIFO



Hình 3.22: Sơ đồ mạch khối bộ nhớ FIFO

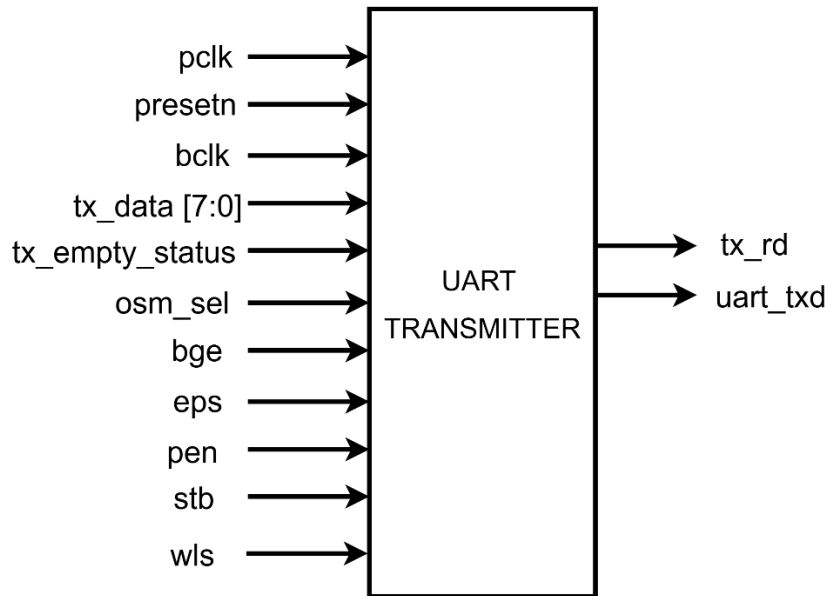
Hình 3.22 mô tả cấu trúc phần cứng của khối bộ nhớ FIFO. Khối này có nhiệm vụ lưu trữ tạm thời dữ liệu giữa khối phát và khối nhận theo nguyên tắc vào trước ra trước.

Bộ nhớ FIFO được tổ chức thành 16 ô nhớ, mỗi ô có độ rộng 8 bit. Dữ liệu đầu vào `data_in [7:0]` chỉ được ghi vào bộ nhớ khi tín hiệu cho phép ghi `fifo_we` được kích hoạt, với vị trí ghi được xác định bởi `wptr [3:0]`. Ở phía đọc, dữ liệu tại địa chỉ do `rptr [3:0]` chỉ ra được đưa trực tiếp ra cổng `data_out [7:0]`.

Việc sử dụng hai con trỏ độc lập cho phép FIFO quản lý dữ liệu tuần tự và hỗ trợ cơ chế bộ đệm vòng, giúp tái sử dụng các vị trí nhớ sau khi dữ liệu đã được đọc ra.

3.2.6 Thiết kế khối UART Transmitter

Khối UART Transmitter chịu trách nhiệm truyền dữ liệu nối tiếp ra đường truyền UART. Dữ liệu từ FIFO sẽ được phát ra theo chuẩn UART gồm các bit: start, data, parity (nếu có) và stop. Sơ đồ khối tổng quát như hình 3.21:



Hình 3.23: Sơ đồ tổng quát của khối UART Transmitter

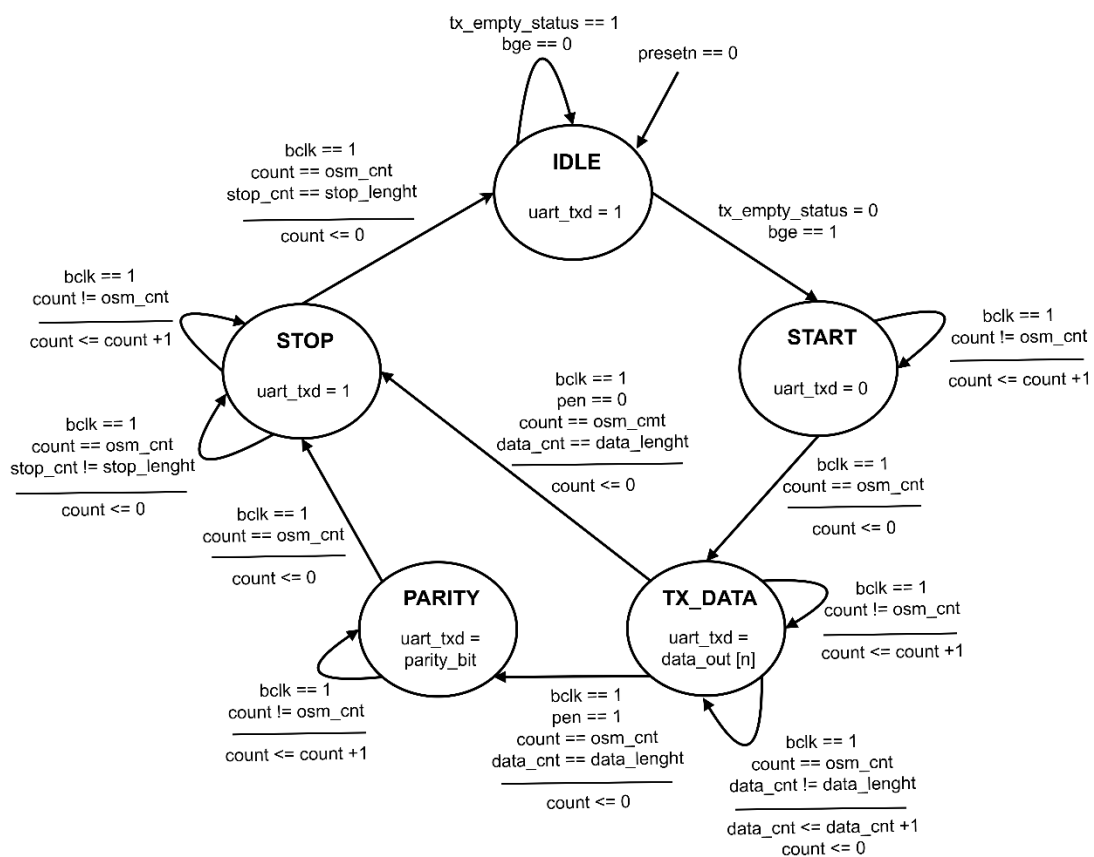
Tín hiệu của khối UART Transmitter được mô tả theo bảng 3.14:

Bảng 3.14: Mô tả tín hiệu khối UART Transmitter

Tên tín hiệu	Loại tín hiệu	Mô tả
pclk	Tín hiệu vào	Xung clock hệ thống đồng bộ.
presetn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp.
bclk	Tín hiệu vào	Xung clock Baud dùng để đồng bộ tốc độ dịch bit truyền.
tx_data [7:0]	Tín hiệu vào	Bus dữ liệu truyền nhận từ bộ đệm TX FIFO.
tx_empty_status	Tín hiệu vào	Cờ báo trạng thái rỗng từ khối TX FIFO.
tx_rd	Tín hiệu ra	Tín hiệu điều khiển yêu cầu đọc dữ liệu từ TX FIFO.
osm_sel	Tín hiệu vào	Tín hiệu chọn chế độ oversampling.
bge	Tín hiệu vào	Tín hiệu cho phép bộ tạo tốc độ Baud hoạt động.
eps	Tín hiệu vào	Cấu hình kiểu Parity (1: Chẵn, 0: Lẻ).

pen	Tín hiệu vào	Tín hiệu cho phép chèn bit Parity vào khung truyền.
stb	Tín hiệu vào	Cấu hình số lượng Stop bit (0: 1 bit, 1: 2 bit).
wls [1:0]	Tín hiệu vào	Cấu hình độ rộng từ dữ liệu (5, 6, 7 hoặc 8 bit).
uart_txd	Tín hiệu ra	Đường truyền dữ liệu nối tiếp ngõ ra của UART.

3.2.6.1 Sơ đồ máy trạng thái của khối UART Transmitter



Hình 3.24: Sơ đồ máy trạng thái của khối UART Transmitter

Dựa vào hình 3.22 hoạt động của máy trạng thái được mô tả như sau:

Trạng thái IDLE: tín hiệu `uart_txd` được giữ ở mức logic cao. FSM duy trì ở trạng thái này khi TX FIFO rỗng hoặc bộ tạo baud chưa được kích hoạt. Khi TX FIFO

chứa dữ liệu và tín hiệu bge được kích hoạt, FSM chuyển sang trạng thái START để bắt đầu quá trình truyền dữ liệu.

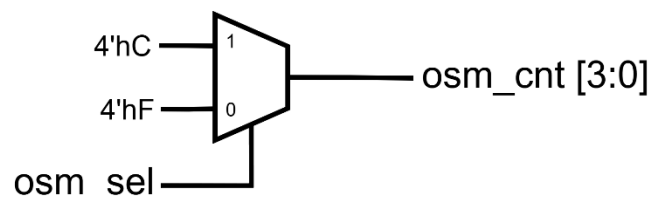
Trạng thái START: được sử dụng để phát bit bắt đầu của khung UART, do đó tín hiệu `uart_txd` được đưa xuống mức logic thấp. Bộ đếm count được tăng theo các xung `belk` cho đến khi đạt giá trị `osm_cnt`. Khi đó, bộ đếm được đặt lại về 0 và FSM chuyển sang trạng thái TX_DATA.

Trạng thái TX_DATA: các bit dữ liệu từ `data_out[n]` được truyền tuần tự lên đường `uart_txd`. Bộ đếm count tiếp tục được sử dụng để xác định thời gian tồn tại của từng bit dữ liệu. Mỗi khi count đạt giá trị `osm_cnt`, một bit dữ liệu được xem là đã truyền hoàn tất và bộ đếm `data_cnt` được tăng lên một đơn vị. Quá trình này lặp lại cho đến khi `data_cnt` đạt giá trị `data_length`. Nếu tín hiệu `pen` ở mức cao, FSM chuyển sang trạng thái PARITY, ngược lại FSM chuyển trực tiếp sang trạng thái STOP.

Trạng thái PARITY: được sử dụng để truyền bit kiểm tra parity. Trong trạng thái này, tín hiệu `uart_txd` mang giá trị của `parity_bit`. Tương tự các trạng thái trước, FSM duy trì trạng thái này cho đến khi bộ đếm count đạt giá trị `osm_cnt`, sau đó chuyển sang trạng thái STOP.

Trạng thái STOP: tín hiệu `uart_txd` được giữ ở mức logic cao để phát bit dừng. Bộ đếm count được sử dụng để xác định thời gian của từng stop bit, trong khi bộ đếm `stop_cnt` theo dõi số lượng stop bit đã truyền. Khi `stop_cnt` đạt giá trị `stop_length`, FSM kết thúc khung truyền hiện tại, đặt lại các bộ đếm và quay về trạng thái IDLE để chờ dữ liệu mới.

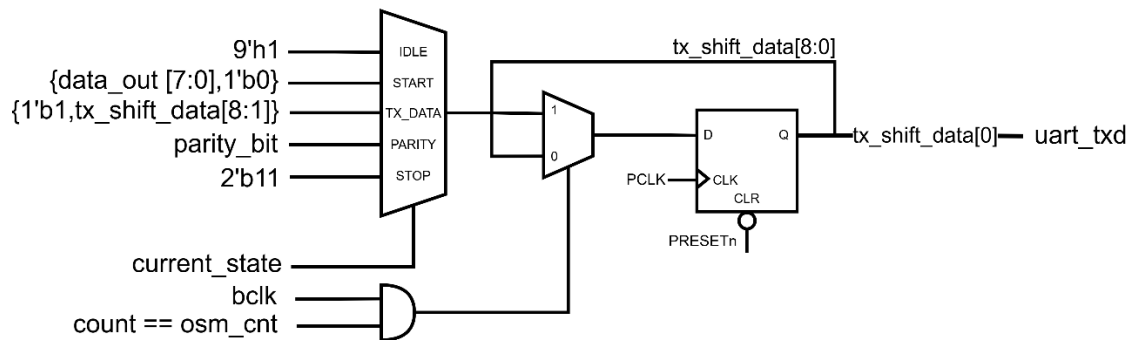
3.2.6.2 Thanh ghi dịch dữ liệu



Hình 3.25: Sơ đồ mạch lựa chọn chế độ oversampling

Hình 3.25 mô tả cấu trúc phần cứng của mạch lựa chọn chế độ oversampling trong khối UART Transmitter. Mạch sử dụng tín hiệu `osm_sel` để lựa chọn một trong hai giá trị hằng và tạo ra tín hiệu `osm_cnt` [3:0].

Khi `osm_sel` ở mức logic cao, ngõ ra `osm_cnt` nhận giá trị 4'hC, ngược lại giá trị 4'hF được lựa chọn. Tín hiệu `osm_cnt` được sử dụng làm giá trị tham chiếu cho bộ đếm thời gian, từ đó xác định thời gian duy trì của từng bit trong quá trình truyền dữ liệu UART.



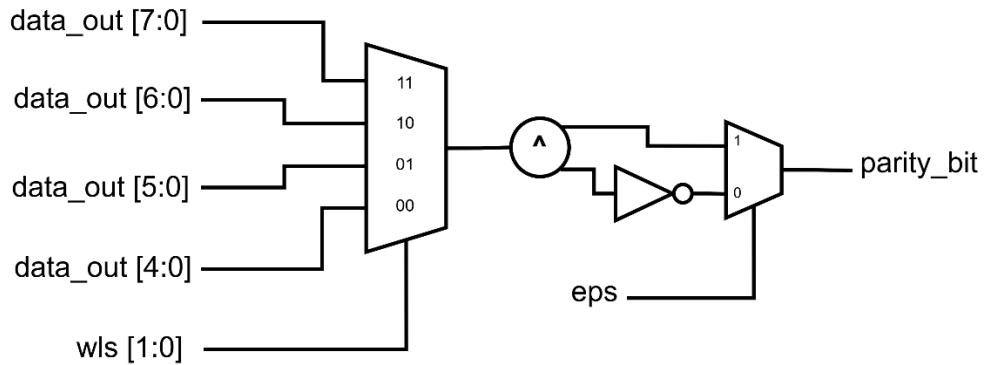
Hình 3.26: Sơ đồ mạch của thanh ghi dịch dữ liệu

Hình 3.26 mô tả cấu trúc phần cứng của thanh ghi dịch dữ liệu của bộ phát UART. Khối này có nhiệm vụ tạo và dịch tuần tự khung dữ liệu truyền dưới sự điều khiển của máy trạng thái hữu hạn FSM. Thông qua cơ chế dịch bit, dữ liệu được xuất tuần tự lên đường truyền `uart_txd` theo đúng định dạng khung UART. Bên trong khối, một bộ chọn dữ liệu được điều khiển bởi `current_state` để nạp giá trị phù hợp cho từng trạng thái:

- Trạng thái IDLE: thanh ghi được nạp giá trị toàn mức cao để giữ đường truyền ở trạng thái nghỉ.
- Trạng thái START: dữ liệu truyền được ghép với bit start mức thấp.
- Trạng thái TX_DATA: dữ liệu trong thanh ghi được dịch phải từng bước và chèn mức logic cao ở đầu vào dịch để lần lượt đưa các bit dữ liệu ra `uart_txd`.
- Trạng thái PARITY: thanh ghi nạp giá trị parity tương ứng.
- Trạng thái STOP: thanh ghi được nạp giá trị mức cao để phát bit dừng.

Việc cập nhật thanh ghi chỉ diễn ra khi có xung belk và bộ đếm oversampling đạt ngưỡng `osm_cnt`, nhờ đó mỗi bit UART được giữ đúng thời gian cấu hình trước khi chuyển sang bit kế tiếp.

3.2.6.3 Mạch tạo parity bit



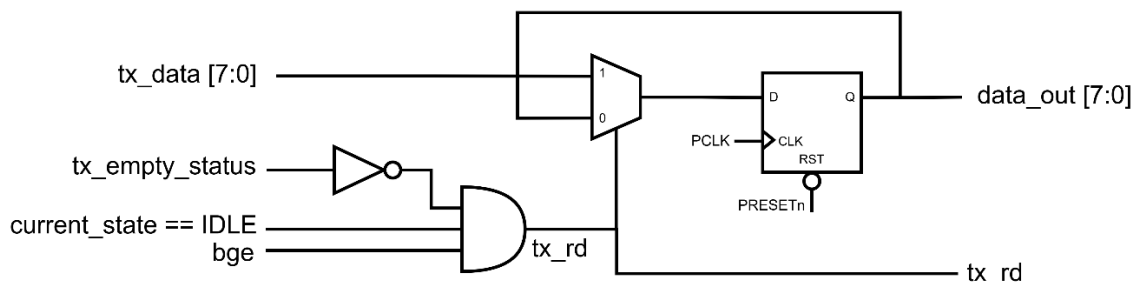
Hình 3.27: Sơ đồ mạch tạo parity bit

Hình 3.27 mô tả cấu trúc phân cứng của khối tạo bit parity trong bộ phát UART. Mạch này có nhiệm vụ sinh tín hiệu `parity_bit` dựa trên dữ liệu truyền và cấu hình parity.

Tín hiệu `wls[1:0]` điều khiển bộ chọn dữ liệu để lấy số lượng bit tương ứng với độ dài từ dữ liệu được cấu hình. Nhóm dữ liệu được chọn sau đó được đưa vào mạch XOR rút gọn để xác định tính chẵn lẻ của dữ liệu truyền.

Giá trị parity thu được từ mạch XOR được đưa tới tầng lựa chọn cuối cùng, trong đó tín hiệu `eps` quyết định sử dụng trực tiếp hoặc đảo giá trị parity nhằm tạo bit parity chẵn hoặc parity lẻ. Tín hiệu đầu ra `parity_bit` được sử dụng trong quá trình truyền dữ liệu UART.

3.2.6.4 Mạch đọc TX FIFO và lưu dữ liệu truyền



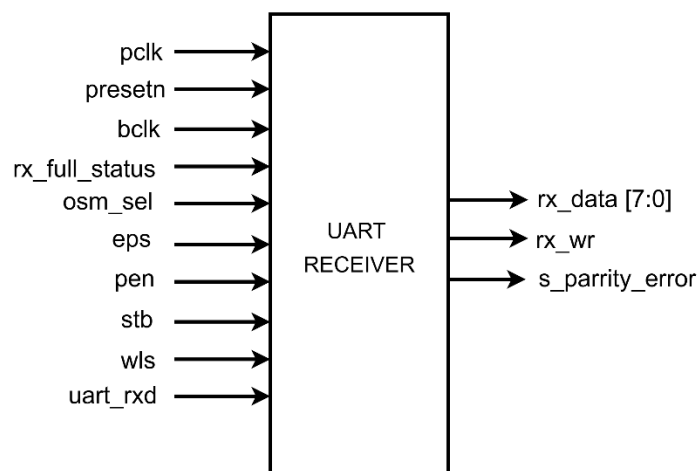
Hình 3.28: Sơ đồ mạch đọc TX FIFO và lưu dữ liệu truyền

Hình 3.28 mô tả cấu trúc phần cứng của mạch tạo tín hiệu tx_rd và nạp dữ liệu từ TX FIFO vào thanh ghi đệm của bộ phát UART. Mạch có nhiệm vụ phát sinh yêu cầu đọc FIFO và lưu dữ liệu nhận được vào thanh ghi data_out [7:0] để phục vụ quá trình truyền.

Tín hiệu tx_rd được tạo từ phép AND giữa điều kiện FIFO không rỗng, FSM đang ở trạng thái IDLE và bộ tạo baud được cho phép bởi bge. Khi tx_rd được kích hoạt, dữ liệu tx_data [7:0] từ TX FIFO được nạp vào thanh ghi data_out [7:0]. Ngược lại, thanh ghi giữ nguyên giá trị hiện tại thông qua đường hồi tiếp. Cơ chế này bảo đảm dữ liệu được chốt ổn định trước khi đưa vào khối truyền UART.

3.2.7 Thiết kế khối UART Receiver

Khối UART_Receiver có nhiệm vụ nhận dữ liệu từ tín hiệu uart_rxd, thực hiện lấy mẫu, giải mã khung truyền UART và chuyển đổi dữ liệu nối tiếp thành dữ liệu song song. Dữ liệu nhận được sau đó được lưu vào RX FIFO để phục vụ quá trình đọc dữ liệu của hệ thống. Hình 3.29 trình bày sơ đồ khối tổng quát của khối UART_Receiver.



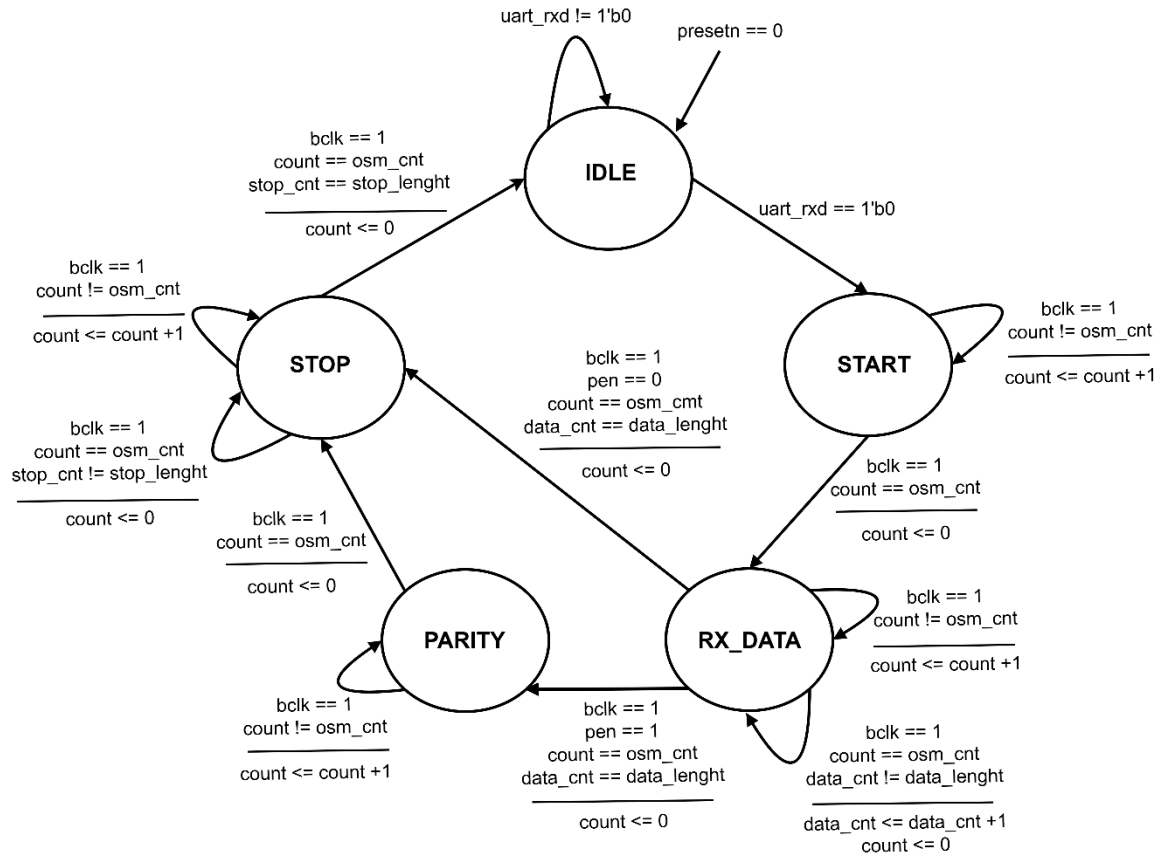
Hình 3.29: Sơ đồ tổng quát của khối UART Receiver

Tín hiệu của khối UART Receiver được mô tả theo bảng 3.15:

Bảng 3.15: Mô tả tín hiệu khối UART Receiver

Tên tín hiệu	Loại tín hiệu	Mô tả
pclk	Tín hiệu vào	Xung clock hệ thống đồng bộ.
presetn	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp.
bclk	Tín hiệu vào	Xung clock Baud dùng để đồng bộ tốc độ dịch bit truyền.
rx_data [7:0]	Tín hiệu ra	Bus dữ liệu song song xuất sang bộ đệm RX FIFO.
rx_full_status	Tín hiệu vào	Cờ báo trạng thái đầy từ khối RX FIFO.
rx_wr	Tín hiệu ra	Tín hiệu điều khiển yêu cầu ghi dữ liệu vào RX FIFO.
osm_sel	Tín hiệu vào	Tín hiệu chọn chế độ oversampling.
eps	Tín hiệu vào	Cấu hình kiểu Parity (1: Chẵn, 0: Lẻ).
pen	Tín hiệu vào	Tín hiệu cho phép chèn bit Parity vào khung truyền.
stb	Tín hiệu vào	Cấu hình số lượng Stop bit (0: 1 bit, 1: 2 bit).
wls [1:0]	Tín hiệu vào	Cấu hình độ rộng từ dữ liệu (5, 6, 7 hoặc 8 bit).
s_parity_error	Tín hiệu ra	Cờ báo trạng thái phát hiện lỗi kiểm tra chẵn lẻ.
uart_rxd	Tín hiệu vào	Đường nhận dữ liệu nối tiếp ngõ vào của UART.

3.2.7.1 Sơ đồ máy trạng thái của khối UART Receiver



Hình 3.30: Sơ đồ máy trạng thái của khối UART Receiver

Dựa vào hình 3.28 hoạt động của máy trạng thái được mô tả như sau:

Trạng thái IDLE: FSM duy trì đường truyền ở trạng thái chờ và chỉ chuyển sang quá trình nhận khi `uart_rxd` xuống mức logic thấp, tương ứng với việc phát hiện bit start. Nếu đường truyền vẫn ở mức cao, FSM tiếp tục giữ nguyên trạng thái này. Khi bắt đầu nhận, bộ đếm `count` được xóa về 0 để chuẩn bị cho chu kỳ lấy mẫu đầu tiên.

Trạng thái START: được dùng để kiểm tra bit start trong một chu kỳ oversampling. Bộ đếm `count` tăng theo xung `bclk` cho đến khi đạt giá trị `osm_cnt`, sau đó được khởi tạo lại. Khi điều kiện này thỏa mãn, FSM chuyển sang trạng thái `RX_DATA` để bắt đầu thu các bit dữ liệu.

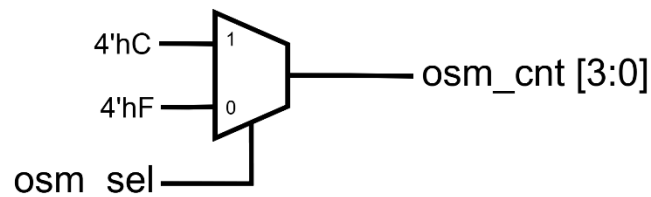
Trạng thái `RX_DATA`: dữ liệu trên `uart_rxd` được lấy mẫu tuần tự theo nhịp `bclk` và được dịch vào thanh ghi nhận dữ liệu. Mỗi khi `count` đạt `osm_cnt`, bộ đếm `data_cnt` được tăng thêm một đơn vị để theo dõi số bit đã nhận. Quá trình này lặp

lại cho đến khi data_cnt đạt data_length. Nếu chế độ parity được bật bởi tín hiệu pen, FSM chuyển sang trạng thái PARITY; ngược lại, FSM chuyển trực tiếp sang trạng thái STOP.

Trạng thái PARITY: được sử dụng để lấy mẫu và kiểm tra bit parity của khung nhận. Bộ đếm count tiếp tục được sử dụng để duy trì thời gian của bit parity trong đúng một chu kỳ bit. Khi count đạt osm_cnt, FSM hoàn tất giai đoạn kiểm tra parity và chuyển sang trạng thái STOP.

Trạng thái STOP: FSM theo dõi bit dừng của khung UART. Bộ đếm stop_cnt được tăng sau mỗi chu kỳ bit hoàn chỉnh để xác định số bit dừng đã đi qua. Khi stop_cnt đạt stop_length, FSM kết thúc quá trình nhận hiện tại, xóa các bộ đếm liên quan và quay trở về trạng thái IDLE để sẵn sàng cho khung dữ liệu tiếp theo.

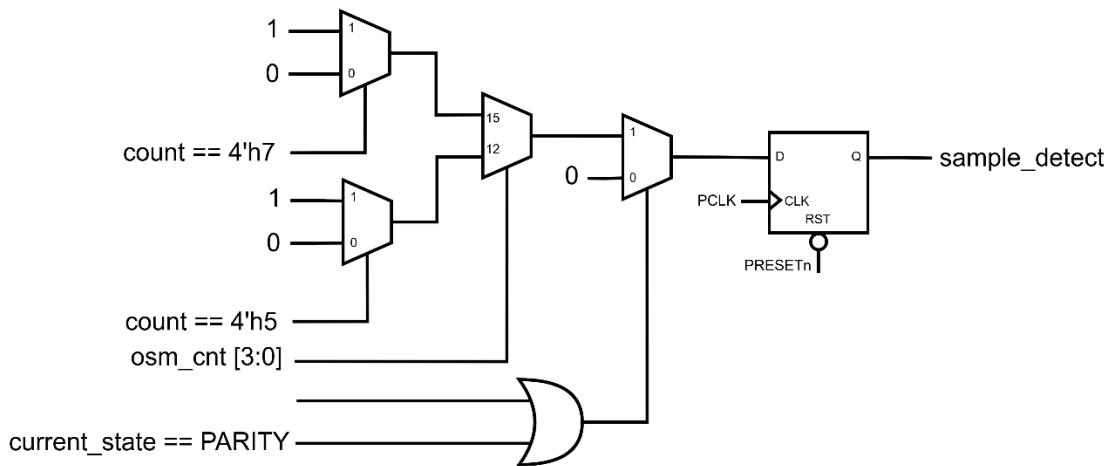
3.2.7.2 Mạch tạo tín hiệu lấy mẫu dữ liệu giữa bit



Hình 3.31: Sơ đồ mạch lựa chọn chế độ oversampling

Hình 3.31 mô tả cấu trúc phần cứng của mạch lựa chọn chế độ oversampling. Mạch sử dụng tín hiệu osm_sel để lựa chọn một trong hai giá trị hằng và tạo ra tín hiệu osm_cnt [3:0].

Khi osm_sel ở mức logic cao, ngõ ra osm_cnt nhận giá trị 4'hC, ngược lại giá trị 4'hF được lựa chọn. Tín hiệu osm_cnt được sử dụng làm giá trị tham chiếu cho các mạch đếm và mạch tạo tín hiệu lấy mẫu trong quá trình nhận dữ liệu UART.



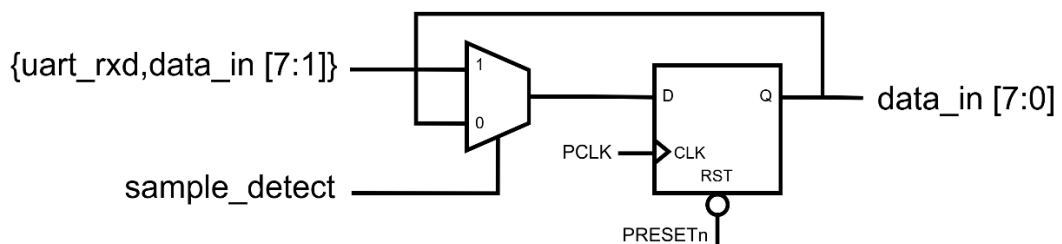
Hình 3.32: Sơ đồ mạch tạo tín hiệu lấy mẫu giữ bit

Hình 3.32 mô tả cấu trúc phần cứng của mạch tạo tín hiệu lấy mẫu dữ liệu trong khối UART Receiver. Mạch có nhiệm vụ xác định thời điểm lấy mẫu dữ liệu trong quá trình nhận nối tiếp.

Bên trong mạch, bộ đếm count được so sánh với các giá trị mốc lấy mẫu tương ứng với chế độ oversampling được lựa chọn bởi tín hiệu osm_cnt. Kết quả so sánh tạo điều kiện lấy mẫu tại thời điểm thích hợp. Tín hiệu lấy mẫu chỉ được cho phép khi FSM đang ở trạng thái RX_DATA hoặc PARITY.

Khi các điều kiện trên đồng thời thỏa mãn, tín hiệu được chốt qua thanh ghi đồng bộ theo xung PCLK để tạo ra tín hiệu sample_detect, phục vụ quá trình nhận dữ liệu và kiểm tra parity.

3.2.7.3 Thanh ghi lưu dữ liệu tạm



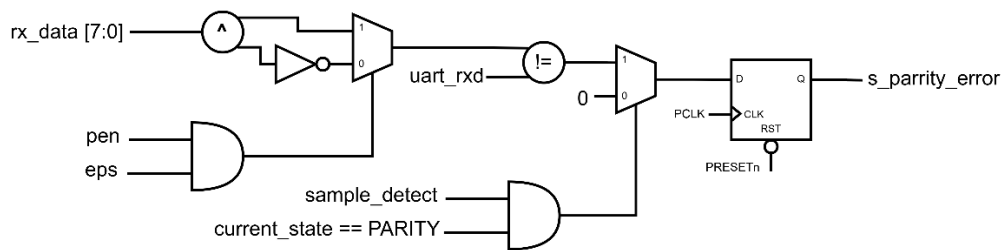
Hình 3.33: Sơ đồ mạch của thanh ghi lưu dữ liệu tạm thời

Hình 3.33 mô tả cấu trúc phần cứng của mạch lưu trữ dữ liệu tạm thời trong bộ thu UART. Mạch có nhiệm vụ dịch và lưu các bit dữ liệu nhận được từ tín hiệu uart_rxd để tạo thành từ dữ liệu song song data_in [7:0].

Bên trong mạch, tín hiệu `sample_detect` điều khiển bộ chọn dữ liệu. Khi tín hiệu này được kích hoạt, bit dữ liệu mới từ `uart_rxd` được dịch vào thanh ghi `data_in`. Ngược lại, dữ liệu hiện tại được giữ tiếp để giữ nguyên nội dung thanh ghi.

Thanh ghi được cập nhật theo xung `PCLK` và được xóa bởi tín hiệu `PRESETn`, đảm bảo dữ liệu chỉ thay đổi tại các thời điểm lấy mẫu hợp lệ.

3.2.7.4 Mạch tính toán và kiểm tra parity



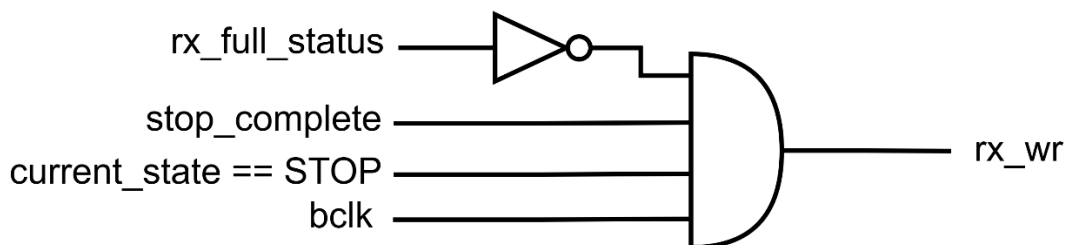
Hình 3.34: Sơ đồ mạch tính toán và kiểm tra parity

Hình 3.34 mô tả cấu trúc phần cứng của mạch tính toán và kiểm tra parity trong bộ thu UART. Dữ liệu `rx_data [7:0]` được đưa vào mạch tính parity, sau đó giá trị parity tạo ra được lựa chọn theo cấu hình `pen` và `eps`.

Giá trị parity tính toán được so sánh với bit parity nhận từ `uart_rxd`. Kết quả so sánh chỉ được cập nhật khi tín hiệu `sample_detect` được kích hoạt và FSM đang ở trạng thái `PARITY`.

Cuối cùng, kết quả được chốt vào thanh ghi đồng bộ theo xung `PCLK` để tạo tín hiệu `s_parity_error`, phục vụ việc phát hiện lỗi parity trong dữ liệu nhận.

3.2.7.5 mạch tạo tín hiệu ghi vào RX FIFO



Hình 3.35: Sơ đồ mạch tạo tín hiệu ghi vào RX FIFO

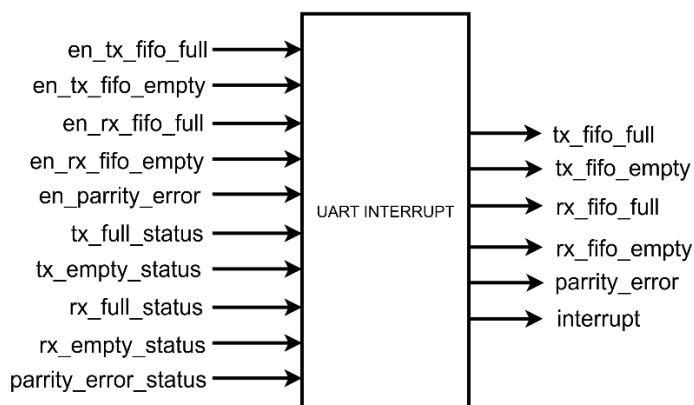
Hình 3.35 mô tả cấu trúc phần cứng của mạch tạo tín hiệu ghi vào RX FIFO trong bộ thu UART. Mạch sử dụng một cổng AND để kết hợp các điều kiện ghi dữ liệu và tạo tín hiệu rx_wr.

Tín hiệu rx_full_status được đảo mức trước khi đưa vào cổng AND nhằm đảm bảo FIFO chưa đầy. Đồng thời, các tín hiệu stop_complete, current_state == STOP và bclk được sử dụng để xác định thời điểm kết thúc quá trình nhận một khung dữ liệu UART.

Tín hiệu rx_wr chỉ được kích hoạt khi tất cả các điều kiện trên đồng thời thỏa mãn, bảo đảm dữ liệu chỉ được ghi vào RX FIFO sau khi nhận hoàn chỉnh khung dữ liệu và FIFO còn khả năng lưu trữ.

3.2.8 Thiết kế khối Interrupt Controller

Khối Interrupt Controller có nhiệm vụ giám sát các trạng thái của FIFO truyền, FIFO nhận và lỗi parity để tạo ra các tín hiệu ngắt tương ứng. Cơ chế ngắt giúp bộ xử lý kịp thời phát hiện các sự kiện quan trọng mà không cần liên tục kiểm tra trạng thái của UART. Sơ đồ khối tổng quát được trình bày trong hình 3.34:



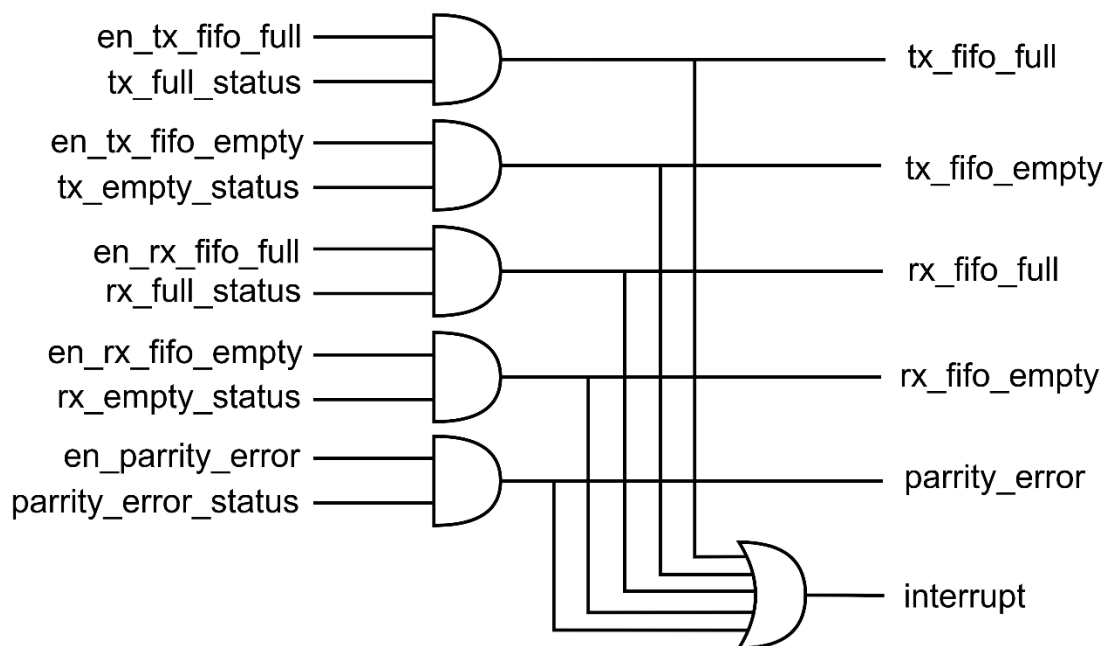
Hình 3.36: Sơ đồ tổng quát khối Interrupt Controller

Tín hiệu của khối Interrupt Controller được mô tả theo bảng 3.16:

Bảng 3.16: Mô tả tín hiệu khối Interrupt Controller

Tên tín hiệu	Loại tín hiệu	Mô tả
en_tx_fifo_full	Tín hiệu vào	Tín hiệu điều khiển cho phép ngắt khi bộ đệm truyền đầy, cấu hình từ thanh ghi.
en_tx_fifo_empty	Tín hiệu vào	Tín hiệu điều khiển cho phép ngắt khi bộ đệm truyền rỗng, cấu hình từ thanh ghi.
en_rx_fifo_full	Tín hiệu vào	Tín hiệu điều khiển cho phép ngắt khi bộ đệm nhận đầy, cấu hình từ thanh ghi.
en_rx_fifo_empty	Tín hiệu vào	Tín hiệu điều khiển cho phép ngắt khi bộ đệm nhận rỗng, cấu hình từ thanh ghi.
en_parity_error	Tín hiệu vào	Tín hiệu điều khiển cho phép ngắt khi xảy ra lỗi kiểm tra chẵn lẻ, cấu hình từ thanh ghi.
tx_full_status	Tín hiệu vào	Cờ trạng thái báo hiệu bộ đệm truyền dữ liệu đã đầy.
tx_empty_status	Tín hiệu vào	Cờ trạng thái báo hiệu bộ đệm truyền dữ liệu đang rỗng.
rx_full_status	Tín hiệu vào	Cờ trạng thái báo hiệu bộ đệm nhận dữ liệu đã đầy.
rx_empty_status	Tín hiệu vào	Cờ trạng thái báo hiệu bộ đệm nhận dữ liệu đang rỗng.
parity_error_status	Tín hiệu vào	Cờ trạng thái báo hiệu phát hiện lỗi kiểm tra chẵn lẻ trong khung truyền.
tx_fifo_full	Tín hiệu ra	Trạng thái ngắt hiệu lực cho sự kiện bộ đệm truyền đầy.
tx_fifo_empty	Tín hiệu ra	Trạng thái ngắt hiệu lực cho sự kiện bộ đệm truyền rỗng.
rx_fifo_full	Tín hiệu ra	Trạng thái ngắt hiệu lực cho sự kiện bộ đệm nhận đầy.

rx_fifo_empty	Tín hiệu ra	Trạng thái ngắt hiệu lực cho sự kiện bộ đệm nhận rỗng.
parrrity_error	Tín hiệu ra	Trạng thái ngắt hiệu lực cho sự kiện lỗi kiểm tra chẵn lẻ.
interrupt	Tín hiệu ra	Tín hiệu yêu cầu ngắt tổng hợp, tích cực mức cao, được chuyển đến bộ điều khiển ngắt trung tâm hoặc CPU.



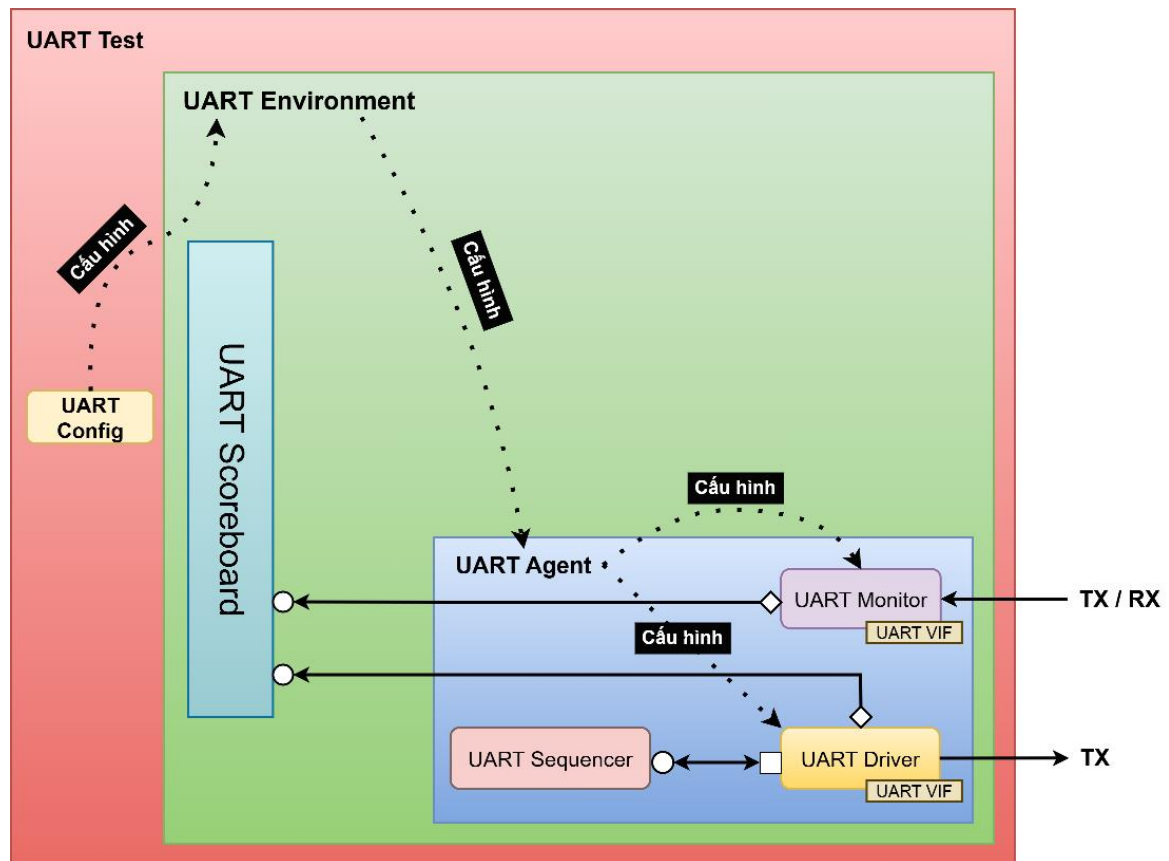
Hình 3.37: Sơ đồ mạch của khối Interrupt Controller

Hình 3.37 mô tả cấu trúc phần cứng của khối Interrupt Controller. Khối này có nhiệm vụ giám sát các trạng thái hoạt động của FIFO truyền, FIFO nhận và lỗi parity để tạo ra các tín hiệu ngắt tương ứng, đồng thời sinh tín hiệu ngắt tổng phục vụ cho bộ xử lý.

Khối bao gồm năm mạch tạo ngắt độc lập tương ứng với các sự kiện: TX FIFO đầy, TX FIFO rỗng, RX FIFO đầy, RX FIFO rỗng và lỗi parity. Mỗi tín hiệu ngắt được tạo bằng cách kết hợp tín hiệu trạng thái và tín hiệu cho phép ngắt tương ứng thông qua cổng AND. Do đó, chỉ những sự kiện được cho phép mới có khả năng kích hoạt tín hiệu ngắt.

Trong lớp UART Test, đối tượng UART Config được sử dụng để lưu trữ các tham số cấu hình của AHB Lite - UART IP như tốc độ baud (2400–115200 bps), số bit dữ liệu (5–8 bit), chế độ parity và số stop bit (1 hoặc 2 bit). Các tham số này được khởi tạo tại lớp kiểm thử và phân phối theo cơ chế top-down thông qua kiến trúc phân lớp của UVM đã trình bày ở Hình 2.20, từ UART Test đến UART Environment, sau đó tới các thành phần sử dụng cấu hình như UART Agent.

Hình 3.39 minh họa cơ chế phân phối cấu hình UART trong môi trường kiểm chứng UVM. Để đơn giản hóa việc trình bày, hình chỉ thể hiện các thành phần trực tiếp liên quan đến quá trình cấu hình UART, tạm thời bỏ qua lớp mô hình thanh ghi và AHB Agent.



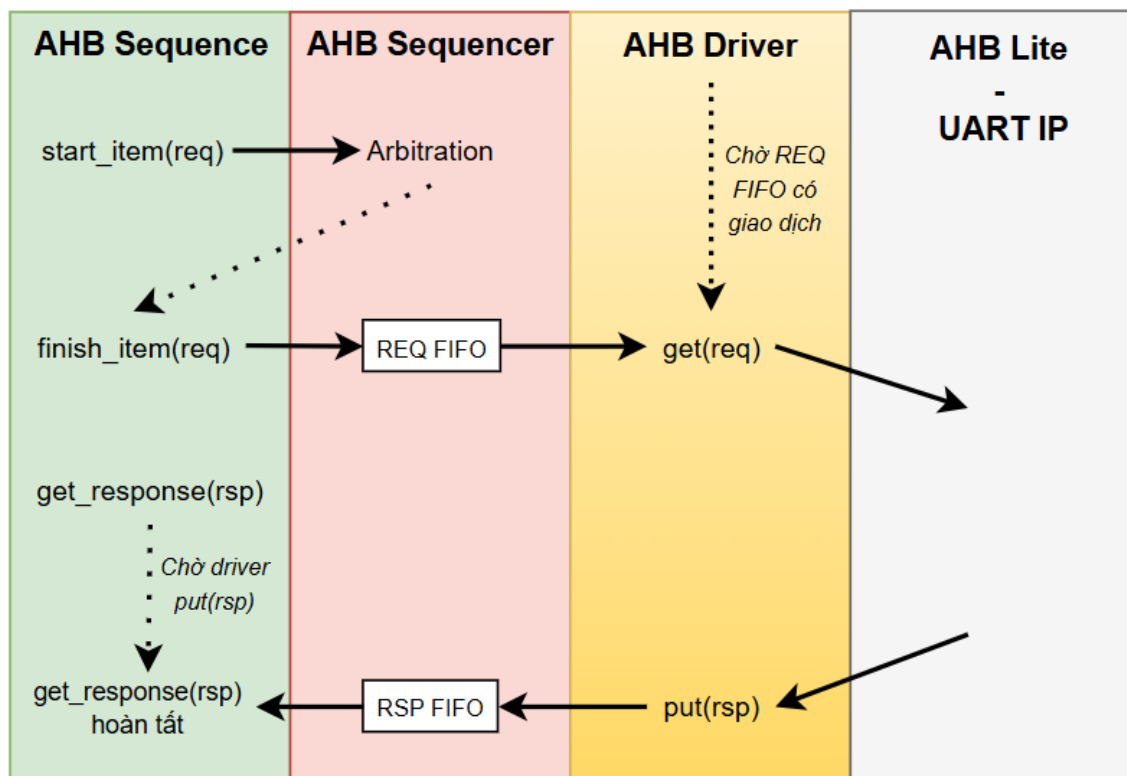
Hình 3.39: Cơ chế phân phối cấu hình UART trong UVM

Sau khi nhận được đối tượng cấu hình, UART Driver sử dụng các tham số đã thiết lập để tạo dạng sóng truyền dữ liệu trên giao diện UART theo đúng định dạng khung truyền yêu cầu. Đồng thời, UART Monitor sử dụng cùng bộ tham số

cấu hình để xác định thời điểm lấy mẫu dữ liệu, giải mã chính xác các trường dữ liệu, bit parity và stop bit tương ứng với tốc độ baud đang được sử dụng. Nhờ đó, cả quá trình phát và thu dữ liệu trong môi trường kiểm chứng luôn được đồng bộ với cấu hình hiện hành của DUT, bảo đảm tính chính xác của các kết quả kiểm chứng.

3.3.2 Thiết kế AHB Agent

AHB Agent là thành phần chịu trách nhiệm tạo ra và giám sát các giao dịch trên bus AHB nhằm truy cập các thanh ghi của AHB Lite - UART IP. Trong môi trường kiểm chứng của đề tài, DUT đóng vai trò AHB Slave nên AHB Agent được xây dựng ở chế độ active và hoạt động như một AHB Master để thực hiện các giao dịch đọc và ghi vào thanh ghi.



Hình 3.40: Cơ chế bắt tay giữa Sequence, Sequencer và Driver trong AHB Agent

Hình 3.40 minh họa cơ chế bắt tay giữa sequence, sequencer và driver trong AHB Agent. Sequence tạo giao dịch và gửi tới sequencer thông qua cặp hàm

start_item(req) và finish_item(req). Sau khi transaction được đưa vào hàng đợi yêu cầu (REQ FIFO), driver sử dụng hàm get(req) để nhận giao dịch từ sequencer và chuyển đổi giao dịch thành các tín hiệu mức chân của giao thức AHB như HADDR, HWRITE, HTRANS, HSEL và HWDATA để điều khiển DUT. Khi giao dịch hoàn tất, driver tạo giao dịch phản hồi và gửi trở lại sequencer bằng hàm put(rsp). Sequence nhận kết quả thông qua hàm get_response(rsp) để phục vụ quá trình kiểm tra dữ liệu đọc hoặc trạng thái giao dịch.

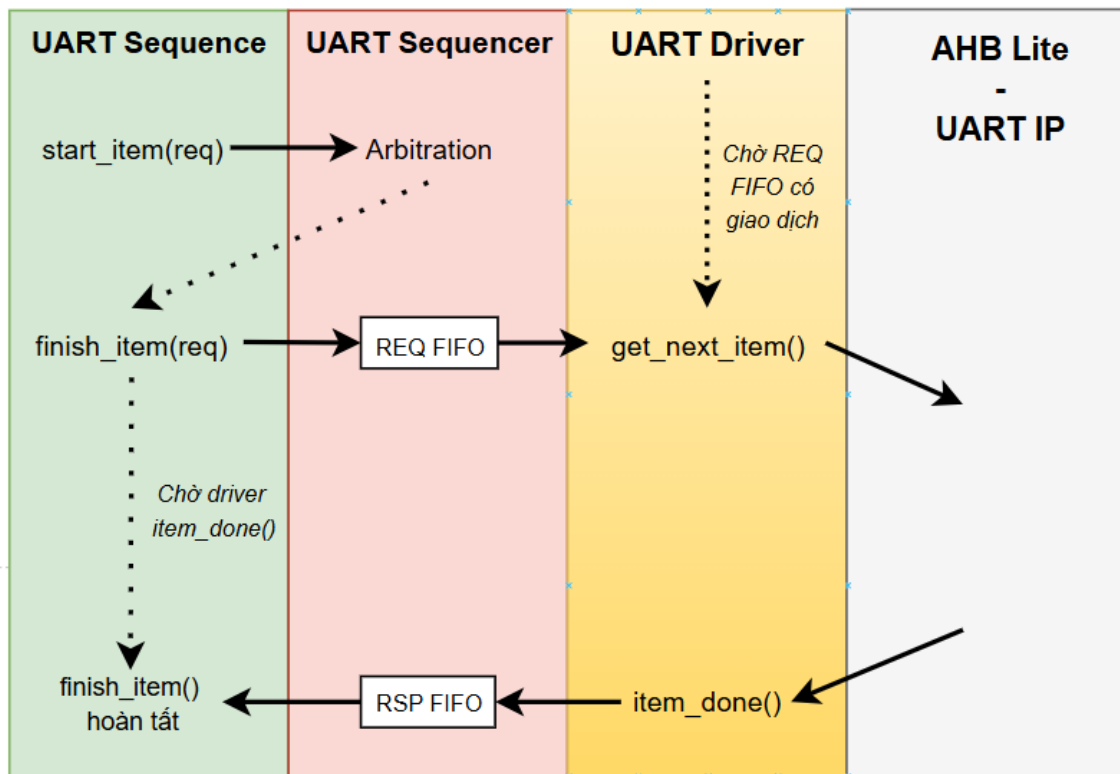
Việc sử dụng cơ chế bắt tay get(req) và put(rsp) cho phép sequence tiếp tục phát sinh các giao dịch mới mà không cần chờ driver hoàn tất giao dịch hiện tại. Các giao dịch được lưu trong REQ FIFO và xử lý tuần tự bởi driver, trong khi dữ liệu phản hồi được trả về thông qua RSP FIFO. Cơ chế này phù hợp với các giao thức bus như AHB do cần trả về dữ liệu đọc hoặc trạng thái giao dịch từ DUT.

Song song với quá trình điều khiển bus, AHB Monitor giám sát các tín hiệu HADDR, HWRITE, HTRANS, HWDATA, HRDATA và HREADYOUT để tái tạo lại các giao dịch thực tế trên bus. Các giao dịch thu thập được sau đó được gửi tới scoreboard và khối độ bao phủ chức năng nhằm phục vụ quá trình kiểm tra chức năng và đánh giá độ bao phủ.

Nhờ sự phối hợp giữa sequence, sequencer, driver và monitor, AHB Agent vừa tạo kích thích cho DUT vừa ghi nhận đầy đủ các giao dịch thực tế trên bus. Điều này cho phép kiểm chứng chính xác các chức năng đọc, ghi và cấu hình thanh ghi của AHB Lite - UART IP trong quá trình mô phỏng.

3.3.3 Thiết kế UART Agent

UART Agent được xây dựng nhằm mô phỏng môi trường UART bên ngoài DUT, cho phép kiểm chứng các chức năng truyền và nhận dữ liệu của AHB Lite - UART IP dưới nhiều cấu hình khác nhau. Không giống AHB Agent tập trung vào việc truy cập thanh ghi, UART Agent làm việc trực tiếp với các khung truyền nối tiếp trên giao diện UART.



Hình 3.41: Cơ chế bắt tay giữa Sequence, Sequencer và Driver trong UART Agent

Hình 3.39 minh họa cơ chế trao đổi transaction giữa sequence, sequencer và driver trong UART Agent. Sequence tạo các giao dịch chứa dữ liệu cần truyền cùng các tham số cấu hình UART và gửi tới sequencer thông qua cặp hàm `start_item(req)` và `finish_item(req)`. Driver nhận giao dịch bằng hàm `get_next_item()`, sau đó chuyển đổi dữ liệu ở mức giao thành khung truyền UART tương ứng. Mỗi khung truyền bao gồm start bit, các bit dữ liệu, parity bit (nếu được cấu hình) và stop bit. Sau khi hoàn tất quá trình truyền trên tín hiệu RX, driver gọi hàm `item_done()` để thông báo giao dịch đã được xử lý xong.

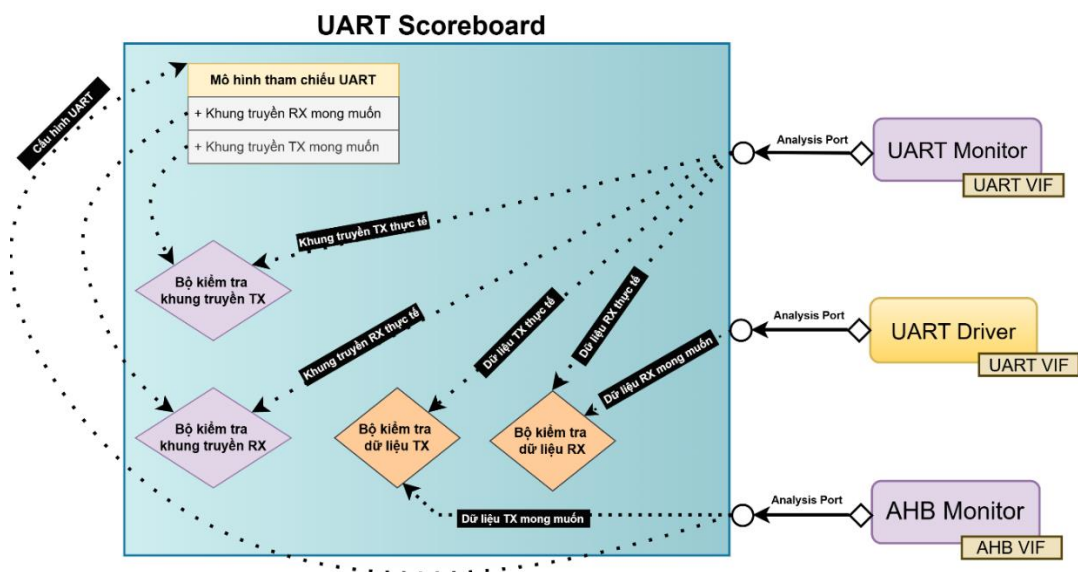
UART Agent sử dụng cơ chế bắt tay `get_next_item()` và `item_done()` do không cần trả về giao dịch phản hồi từ driver. Mỗi giao dịch tương ứng với một khung truyền UART hoàn chỉnh nên sequence sẽ chờ giao dịch hiện tại kết thúc sau đó thực hiện giao dịch tiếp theo. Cơ chế này phù hợp với đặc điểm hoạt động tuần tự của giao thức UART và giúp đơn giản hóa quá trình trao đổi dữ liệu giữa sequence và driver.

Bên cạnh việc tạo dữ liệu đầu vào cho DUT, UART Agent còn thực hiện giám sát dữ liệu được truyền ra thông qua UART Monitor. Monitor theo dõi tín hiệu RX, phát hiện start bit và thực hiện lấy mẫu dữ liệu theo tốc độ baud đã cấu hình để tái tạo lại khung truyền UART. Các trường dữ liệu, parity bit và stop bit sau đó được giải mã và đóng gói thành một giao dịch ở mức trừu tượng. Các giao dịch này được gửi tới scoreboard để kiểm tra tính chính xác của dữ liệu truyền nhận, đồng thời được sử dụng để thu thập độ bao phủ chức năng.

Thông qua UART Agent, môi trường kiểm chứng có thể mô phỏng đầy đủ hoạt động của một thiết bị UART thực tế, từ đó đánh giá khả năng truyền nhận dữ liệu của DUT dưới các cấu hình khác nhau về tốc độ baud, độ dài dữ liệu, parity và số stop bit.

3.3.4 Thiết kế Scoreboard

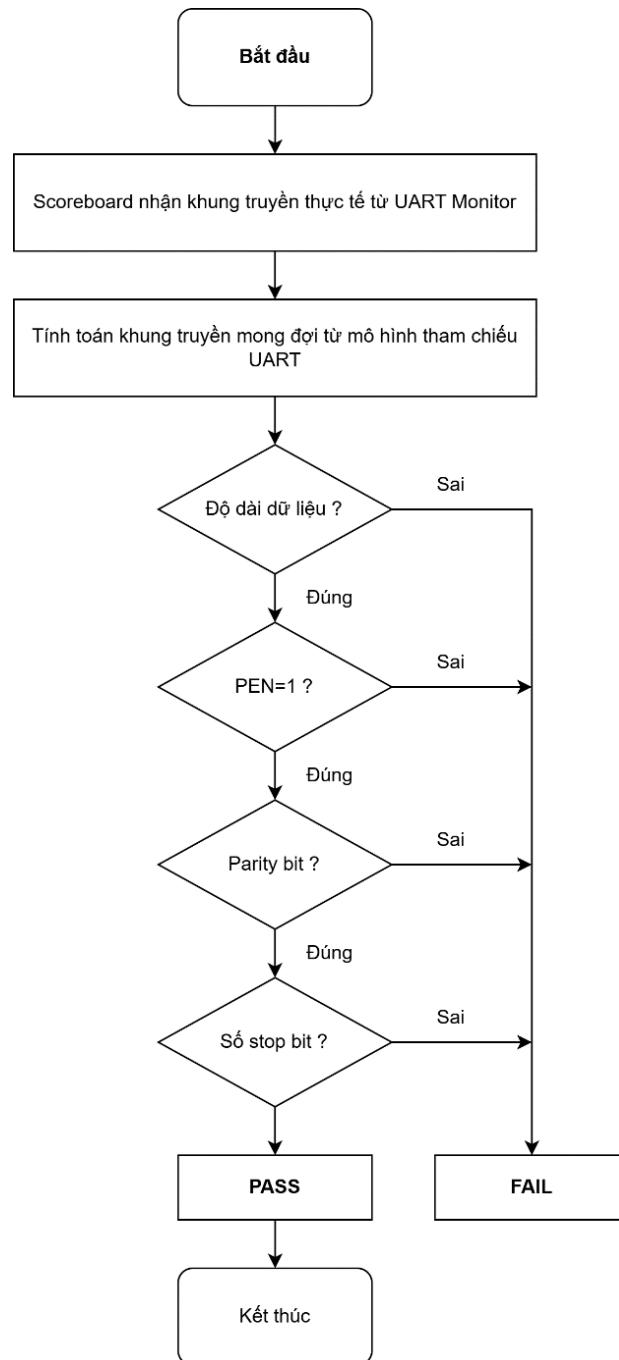
Scoreboard được thiết kế theo kiến trúc kiểm tra tập trung, bao gồm một mô hình tham chiếu UART và bốn khối kiểm tra chức năng độc lập. Nhiệm vụ của scoreboard là xây dựng kết quả mong đợi dựa trên cấu hình hiện hành của DUT, sau đó đối chiếu với dữ liệu thực tế thu thập được trong quá trình mô phỏng nhằm đánh giá tính chính xác của các chức năng truyền và nhận dữ liệu.



Hình 3.42: Kiến trúc kiến trúc của UART Scoreboard

Hình 3.42 trình bày kiến trúc của UART Scoreboard được sử dụng trong môi trường kiểm chứng. Thành phần trung tâm của scoreboard là mô hình tham chiếu UART, có nhiệm vụ theo dõi các giao dịch cấu hình được gửi tới DUT thông qua AHB Monitor. Các giá trị ghi vào các thanh ghi cấu hình như MDR, DLL, DLH và LCR được lưu lại để xác định tốc độ baud, độ dài dữ liệu, chế độ parity và số stop bit hiện hành. Dựa trên các thông số này, mô hình tham chiếu thực hiện tính toán khung truyền UART mong đợi cho cả chiều phát (TX) và chiều nhận (RX).

Kết quả được tạo ra từ mô hình tham chiếu sau đó được chuyển tới bộ kiểm tra khung truyền TX và bộ kiểm tra khung truyền RX. Đồng thời, scoreboard cũng nhận các khung truyền thực tế do UART Monitor thu thập được từ DUT để thực hiện so sánh. Thông qua quá trình đối chiếu này, scoreboard có thể kiểm tra các thuộc tính của khung truyền như độ dài dữ liệu, bit parity và số lượng stop bit theo đúng cấu hình đã được thiết lập.



Hình 3.43: Lưu đồ giải thuật kiểm tra khung truyền UART.

Lưu đồ giải thuật kiểm tra khung truyền UART được trình bày trong Hình 3.43. Trước tiên, scoreboard nhận khung truyền thực tế từ UART Monitor và sử dụng các tham số cấu hình hiện hành của UART để xây dựng khung truyền mong đợi. Tiếp theo, các thuộc tính của khung truyền được kiểm tra tuần tự. Đầu tiên là độ dài dữ liệu, nhằm xác nhận số lượng bit dữ liệu trong khung truyền phù hợp với cấu hình đã thiết lập. Sau đó, nếu chế độ parity được kích hoạt, giá trị parity bit sẽ

được kiểm tra để bảo đảm tính nhất quán giữa dữ liệu truyền và cơ chế kiểm tra lỗi. Cuối cùng, số lượng stop bit của khung truyền được đối chiếu với cấu hình hiện hành của DUT.

Khung truyền chỉ được đánh giá là hợp lệ khi tất cả các điều kiện kiểm tra đều thỏa mãn. Nếu bất kỳ thuộc tính nào không khớp với giá trị mong đợi, scoreboard sẽ ghi nhận lỗi tương ứng và đánh dấu khung truyền không đạt yêu cầu. Cơ chế này giúp phát hiện các lỗi liên quan đến định dạng khung truyền, đồng thời hỗ trợ quá trình phân tích và xác định nguyên nhân gây lỗi trong quá trình kiểm chứng.

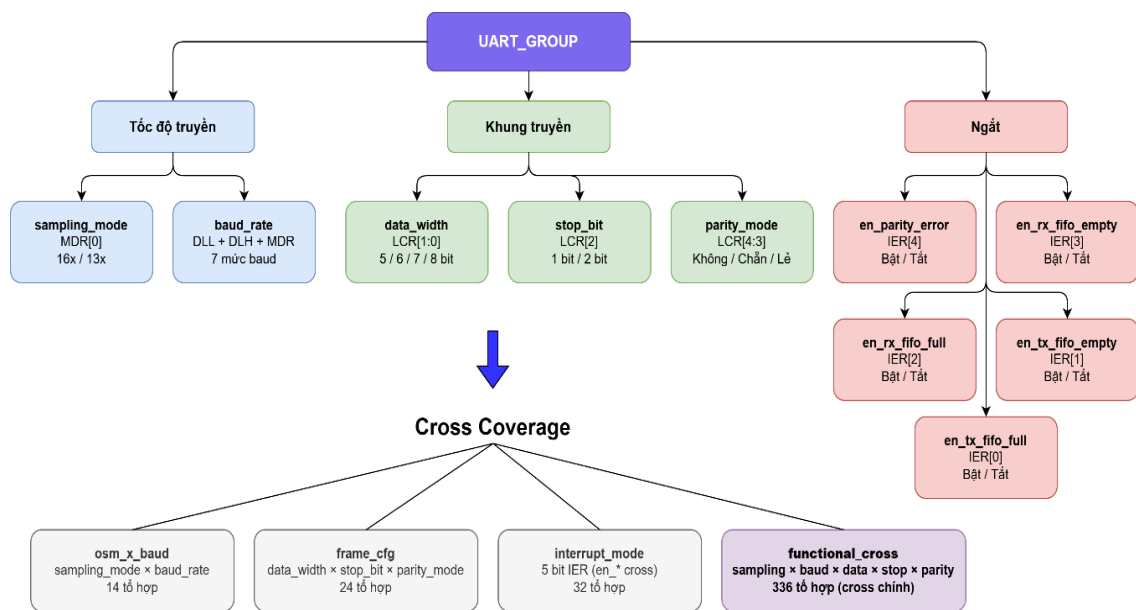
Bên cạnh việc kiểm tra cấu trúc khung truyền, scoreboard còn kiểm tra tính chính xác của dữ liệu thông qua bộ kiểm tra dữ liệu TX và RX. Với chiều phát, dữ liệu mong đợi được lấy từ các giao dịch ghi vào thanh ghi TBR do AHB Monitor thu thập và được so sánh với dữ liệu thực tế từ UART Monitor tại chân TX của DUT. Với chiều nhận, dữ liệu mong đợi được lấy từ các giao dịch do UART Driver gửi tới DUT và được đối chiếu với dữ liệu thực tế do UART Monitor thu thập. Kết quả so sánh giúp đánh giá tính đúng đắn của chức năng truyền nhận dữ liệu của AHB Lite - UART IP.

3.3.5 Thiết kế mô hình độ bao phủ chức năng

Dựa trên nền tảng lý thuyết về độ bao phủ chức năng đã trình bày ở Mục 2.6.2, mô hình độ bao phủ chức năng cho UART IP được xây dựng thông qua nhóm bao phủ UART_GROUP nhằm đánh giá mức độ các tổ hợp cấu hình quan trọng đã được kiểm tra trong quá trình mô phỏng.

UART_GROUP được tích hợp trực tiếp vào Scoreboard và chỉ được kích hoạt ghi nhận khi toàn bộ cấu hình UART đã xác lập xong - cụ thể là khi bit BGE trong thanh ghi LCR bằng 1, báo hiệu bộ tạo baud rate đã được kích hoạt. Cách tiếp cận này đảm bảo chỉ các trạng thái cấu hình hợp lệ mới được tính vào kết quả bao phủ.

UART_GROUP được thiết kế với các coverpoint phản ánh trực tiếp các thông số cấu hình của AHB Lite - UART IP theo đặc tả, bao gồm cấu hình tốc độ truyền, cấu hình khung truyền và cấu hình ngắt. Hình 3.44 minh họa cấu trúc tổng thể của UART_GROUP và mối quan hệ giữa các coverpoint với các thanh ghi cấu hình tương ứng.



Hình 3.44: Cấu trúc tổng thể của UART_GROUP

Về tốc độ truyền, chế độ oversampling (16x hoặc 13x) và tốc độ baud (7 mức từ 2400 đến 115200) được ghi nhận riêng biệt. Đáng chú ý là tốc độ baud không lưu trực tiếp trong một thanh ghi mà được suy ra từ tổ hợp DLL, DLH và MDR. Do đó một bước mã hóa trung gian được thực hiện trước khi ghi nhận bao phủ. Nếu tổ hợp không hợp lệ, trường hợp đó được đánh dấu lỗi ngay trong quá trình mô phỏng.

Về cấu hình khung truyền, ba thông số được theo dõi gồm độ rộng dữ liệu (5, 6, 7 hoặc 8 bit), số stop bit (1 hoặc 2) và chế độ parity (không có, chẵn hoặc lẻ), tất cả được lấy từ thanh ghi LCR.

Về cấu hình ngắt, 5 bit enable trong thanh ghi IER được theo dõi độc lập, mỗi bit ghi nhận cả hai trạng thái bật và tắt.

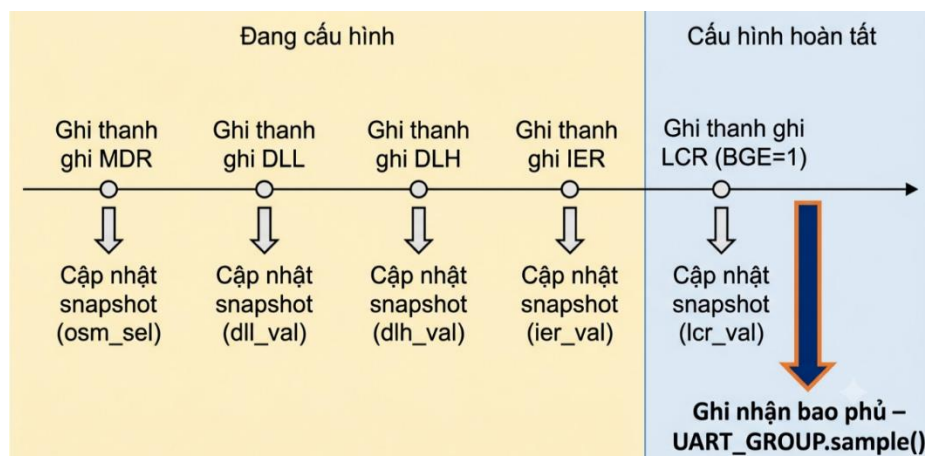
Bảng 3.17: Tổng hợp toàn bộ các điểm bao phủ và điểm bao phủ giao chéo trong mô hình.

STT	Tên	Loại	Thanh ghi nguồn	Mô tả	Số bins
1	sampling_mode	Điểm bao phủ	MDR[0]	Chế độ oversampling	2
2	baud_rate	Điểm bao phủ	DLL+DLH+MDR	Tốc độ baud	7
3	data_width	Điểm bao phủ	LCR[1:0]	Độ rộng dữ liệu	4
4	stop_bit	Điểm bao phủ	LCR[2]	Số stop bit	2
5	parity_mode	Điểm bao phủ	LCR[4:3]	Chế độ parity	3
6	en_parity_error	Điểm bao phủ	IER[4]	Ngắt lỗi parity	2
7	en_rx_fifo_empty	Điểm bao phủ	IER[3]	Ngắt RX FIFO rỗng	2
8	en_rx_fifo_full	Điểm bao phủ	IER[2]	Ngắt RX FIFO đầy	2
9	en_tx_fifo_empty	Điểm bao phủ	IER[1]	Ngắt TX FIFO rỗng	2
10	en_tx_fifo_full	Điểm bao phủ	IER[0]	Ngắt TX FIFO đầy	2
11	osm_x_baud	Điểm bao phủ chéo	—	Oversampling × Baud rate	14
12	frame_cfg	Điểm bao phủ chéo	—	Độ rộng × Stop bit × Parity	24
13	interrupt_mode	Điểm bao phủ chéo	—	Tổ hợp 5 bit ngắt IER	32
14	functional_cross	Điểm bao phủ chéo (Chính)	—	Oversampling × Baud × Khung truyền	336

Bên cạnh các coverpoint đơn lẻ, UART_GROUP định nghĩa thêm các bao phủ giao chéo nhằm kiểm tra sự kết hợp giữa các thông số cấu hình, vì trong thực tế một lỗi thiết kế thường chỉ xuất hiện ở một tổ hợp cấu hình cụ thể chứ không phải ở từng thông số riêng lẻ.

Osm_x_baud là giao chéo giữa chế độ oversampling và tốc độ baud tạo ra 14 tổ hợp hợp lệ. Frame_cfg giao chéo giữa độ rộng dữ liệu, số stop bit và chế độ parity tạo ra 24 tổ hợp. Functional_cross là điểm bao phủ giao chéo chính, giao chéo tổng hợp 5 thông số cấu hình tạo ra 336 tổ hợp phản ánh toàn bộ không gian cấu hình chức năng của AHB Lite - UART IP. Interrupt_mode được giữ như một cross riêng biệt với 32 tổ hợp, không đưa vào functional_cross vì sẽ tạo ra 10.752 tổ hợp, quá lớn để đạt được trong thực tế.

Một thách thức đặc thù của AHB Lite - UART IP là tốc độ baud được xác định bởi tổ hợp ba thanh ghi DLL, DLH và MDR được ghi ở các thời điểm khác nhau, nên không thể ghi nhận bao phủ ngay tại thời điểm ghi từng thanh ghi riêng lẻ. Để giải quyết, cơ chế snapshot được áp dụng: Sau mỗi lần ghi thanh ghi cấu hình, trạng thái toàn bộ hệ thống được cập nhật vào bộ nhớ tạm. Tuy nhiên, điểm bao phủ chỉ được ghi nhận tại một thời điểm duy nhất khi bit BGE trong thanh ghi LCR được đặt bằng 1, báo hiệu toàn bộ cấu hình đã hoàn chỉnh. Hình 3.45 minh họa cơ chế này.



Hình 3.45: Cơ chế ghi nhận điểm bao phủ dựa trên snapshot cấu hình thanh ghi UART

Toàn bộ mô hình độ bao phủ chức năng được thiết kế bám sát đặc tả của AHB Lite - UART IP, tập trung vào không gian cấu hình mà hệ thống hỗ trợ. Với 336 tổ hợp trong functional_cross và 32 tổ hợp trong interrupt_mode, mô hình đảm bảo rằng các tình huống hoạt động quan trọng của UART đều được kiểm tra một cách có hệ thống, đồng thời tránh tạo ra số lượng tổ hợp quá lớn không thể đạt được trong thực tế.

3.3.6 Xây dựng kế hoạch kiểm chứng

Kế hoạch kiểm chứng được xây dựng nhằm xác định đầy đủ các mục tiêu kiểm tra cần đạt được trước khi kết thúc quá trình kiểm chứng AHB Lite - UART IP, được tổ chức thành 6 nhóm kiểm tra phản ánh các chức năng chính của hệ thống. Bảng 3.2 tổng hợp các nhóm kiểm tra và số lượng testcase tương ứng.

Bảng 3.18: Tổng hợp các nhóm kiểm tra trong kế hoạch kiểm chứng

STT	Nhóm kiểm tra	Số testcase	Phương pháp
1	Kiểm tra thanh ghi	2	Trực tiếp
2	Kiểm tra tốc độ baud và khung UART	23	Trực tiếp
3	Kiểm tra TX/RX FIFO và cơ chế ngắt	2	Trực tiếp
4	Kiểm tra xử lý lỗi	2	Trực tiếp
5	Kiểm tra ngẫu nhiên có ràng buộc	15	Ngẫu nhiên hóa
Tổng		47	—

Nhóm kiểm tra thanh ghi gồm 2 testcase xác minh giá trị reset mặc định và tính nhất quán đọc/ghi trên từng bit. Nhóm kiểm tra tốc độ baud và khung UART gồm 23 testcase, bao phủ 7 tốc độ baud ở cả hai chế độ oversampling 16x và 13x, kết hợp với các tổ hợp độ rộng dữ liệu, stop bit và parity. Nhóm kiểm tra FIFO và cơ chế ngắt gồm 2 testcase xác minh tín hiệu ngắt khi FIFO rỗng, FIFO đầy và hành vi khi ghi vượt dung lượng. Nhóm kiểm tra xử lý lỗi gồm 2 testcase kiểm tra các tình huống ngoại lệ gồm truy cập địa chỉ không hợp lệ và ngắt khi nhận dữ liệu có lỗi parity.

Ngoài kiểm tra định hướng, 15 testcase ngẫu nhiên có ràng buộc được xây dựng với cấu hình khung truyền được ngẫu nhiên hóa trên từng mức baud rate. Mỗi testcase chạy lặp lại 100 lần, tổng cộng 1500 lần chạy. Bổ sung thêm 5 testcase thiết kế có chủ đích nhằm kích hoạt các tổ hợp cấu hình cụ thể chưa được bao phủ, đảm bảo functional_cross của độ bao phủ chức năng đạt mức bao phủ cao nhất. Kết quả thực hiện theo kế hoạch này sẽ được trình bày chi tiết trong Chương 4.

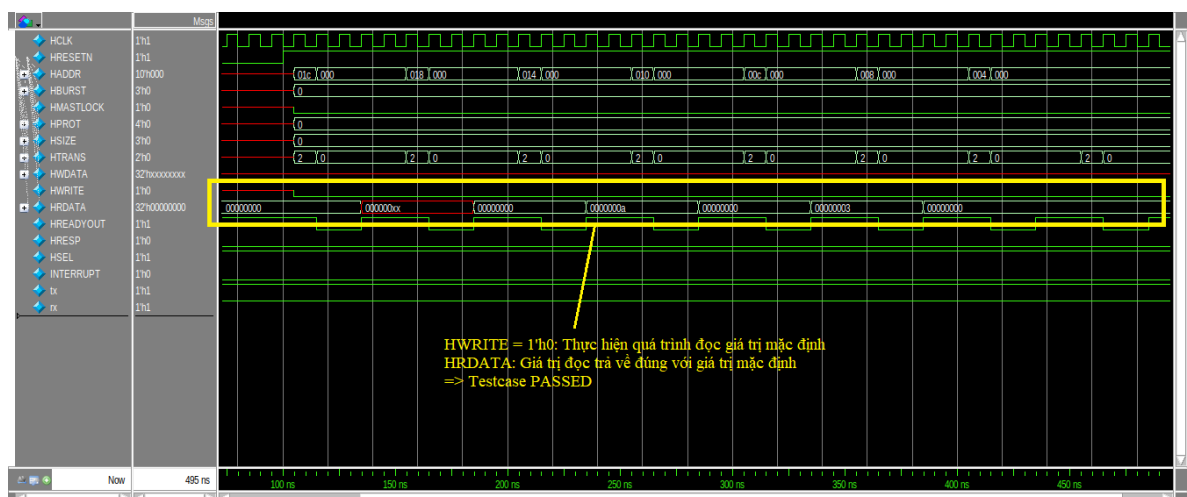
CHƯƠNG 4 KẾT QUẢ THỰC HIỆN

4.1 Kết quả kiểm thử chức năng toàn hệ thống

4.1.1 Kiểm thử truy cập thanh ghi UART

Nhóm kiểm thử truy cập thanh ghi UART được thực hiện nhằm xác minh tính đúng đắn của cơ chế đọc/ghi thanh ghi thông qua lớp trừu tượng hóa thanh ghi UVM RAL. Hai testcase được xây dựng dựa trên hai sequence có sẵn của UVM RAL là `uvm_reg_hw_reset_seq` và `uvm_reg_bit_bash_seq`.

Testcase `default_value_test` sử dụng `uvm_reg_hw_reset_seq` để đọc lần lượt toàn bộ thanh ghi UART và so sánh giá trị đọc được với giá trị reset được định nghĩa trong mô hình thanh ghi (Mục 3.3.1). Hình 4.1 thể hiện waveform của testcase này, trong đó tín hiệu `HADDR` thay đổi tuần tự qua các địa chỉ thanh ghi (0x01C, 0x018, 0x014, 0x010, 0x00C, 0x008, 0x004, 0x000), tín hiệu `HWRITE` duy trì mức thấp xác nhận đây là các giao dịch đọc, và giá trị `HRDATA` trả về tương ứng với từng địa chỉ. Kết quả testcase `PASSED`, xác nhận giá trị reset của toàn bộ thanh ghi khớp với đặc tả trong register model.

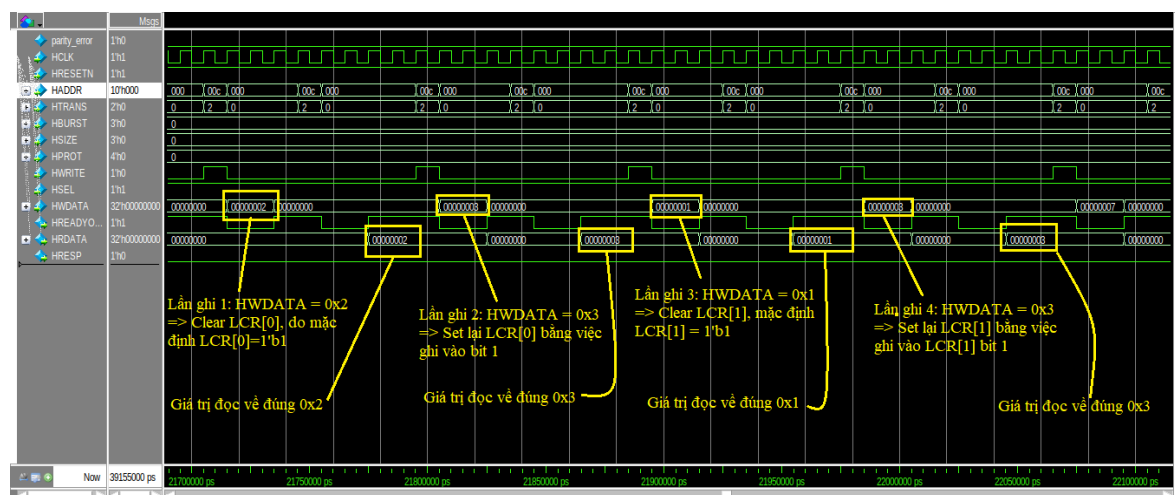


Hình 4.1: Waveform testcase `default_value_test`

Testcase `rw_value_test` sử dụng `uvm_reg_bit_bash_seq` — một sequence có sẵn trong lớp trừu tượng hóa thanh ghi của UVM — để tuần tự ghi bit 1 và bit 0

vào từng bit của mỗi thanh ghi, sau đó đọc lại và so sánh với giá trị đã ghi. Đối với các thanh ghi chỉ đọc (RO), thao tác ghi được thực hiện nhưng không tác động đến giá trị thanh ghi.

Hình 4.2 minh họa hành vi trên thanh ghi LCR (0x00C) là thanh ghi RW đại diện với giá trị mặc định là 0x3 (WLS = 2'b11). Do bit 0 đã ở mức 1 sẵn, sequence bắt đầu bằng cách clear bit 0 trước với giá trị ghi đầu tiên là HWDATA = 0x2, sau đó lần lượt nhận các giá trị 0x3, 0x1, 0x3, 0x7 tương ứng với việc set/clear từng bit. Tín hiệu HWRITE xen kẽ giữa mức cao (ghi) và mức thấp (đọc), và HRDATA đọc lại sau mỗi lần ghi đều khớp với giá trị HWDATA tương ứng.



Hình 4.2: Waveform rw_value_test – thanh ghi LCR (RW)

Quá trình kiểm tra được thực hiện tương tự trên toàn bộ các thanh ghi còn lại trong register map. Do waveform của toàn bộ quá trình có độ dài lớn, hình 4.2 chỉ minh họa một phần của thanh ghi đại diện. Kết quả cả hai testcase đều PASSED, xác nhận cơ chế đọc/ghi thanh ghi hoạt động đúng theo đặc tả.

4.1.2 Kiểm thử tốc độ baud

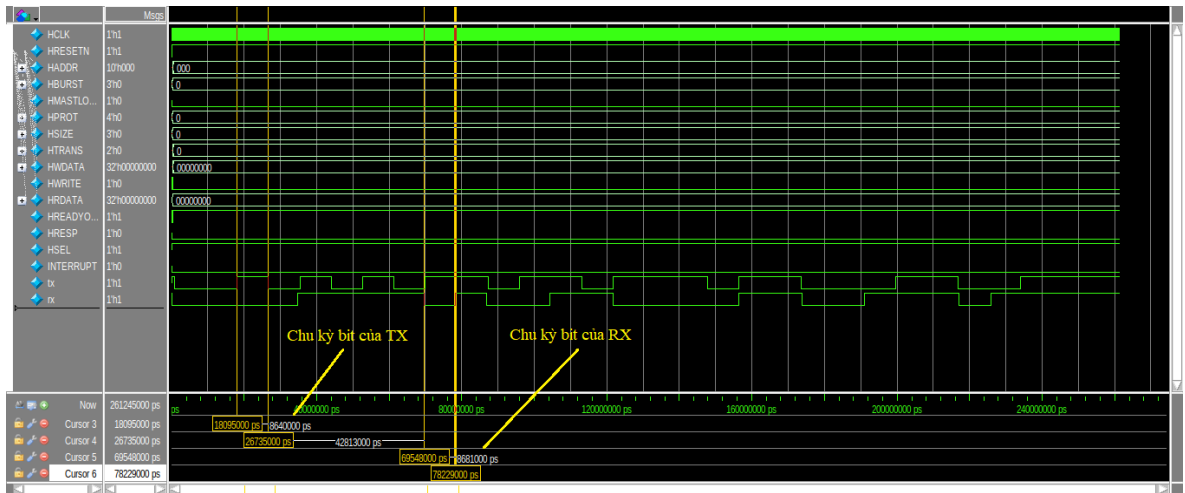
Nhóm kiểm tra tốc độ baud gồm 14 testcase kiểm tra 7 mức baud rate từ 2400 đến 115200 baud ở cả hai chế độ oversampling 16x và 13x. Mỗi testcase cấu hình khung UART cố định gồm 8 bit dữ liệu, 1 stop bit, không có parity, và truyền lần lượt 3 byte dữ liệu 0xAA, 0xBB, 0xCC đồng thời theo cả hai chiều TX và RX.

Tiêu chí kiểm tra là chu kỳ bit thực tế đo được trên đường uart_txd và uart_rxd phải khớp với giá trị lý thuyết theo đặc tả. Bảng 4.1 tổng hợp kết quả 14 testcase.

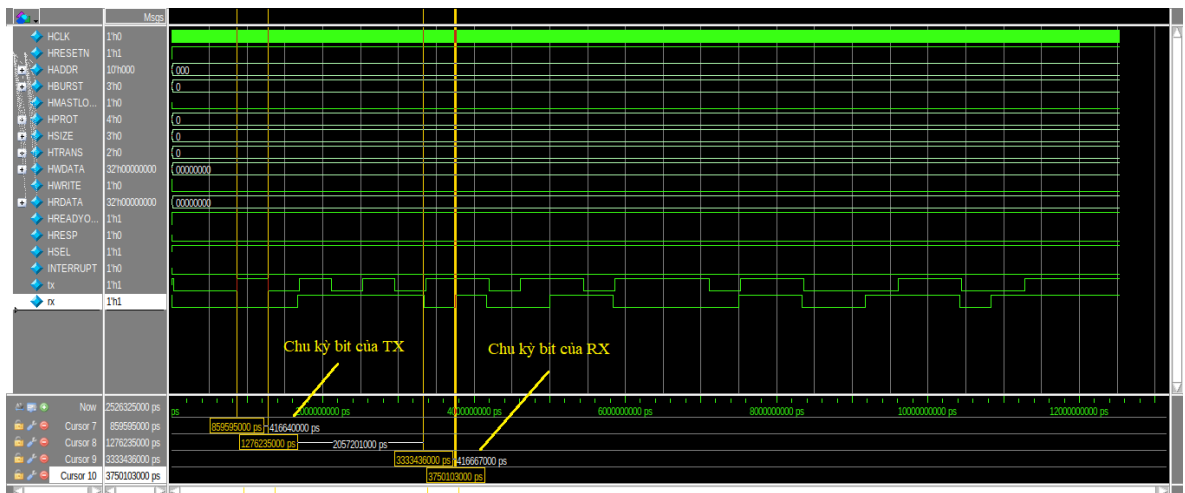
Bảng 4.1: Kết quả kiểm tra tốc độ baud

Testcase	Baud rate	Oversampling	Chu kỳ bit TX (μ s)	Chu kỳ bit RX (μ s)	Lý thuyết (μ s)	Sai số TX (%)	Kết quả
baud2400_16x_test	2400	16x	416.640	416.667	416.667	0.006%	PASSED
baud4800_16x_test	4800	16x	208.608	208.333	208.333	0.132%	PASSED
baud9600_16x_test	9600	16x	104.160	104.167	104.167	0.007%	PASSED
baud19200_16x_test	19200	16x	52.000	52.083	52.083	0.159%	PASSED
baud38400_16x_test	38400	16x	26.080	26.042	26.042	0.146%	PASSED
baud76800_16x_test	76800	16x	12.960	13.021	13.021	0.468%	PASSED
baud115200_16x_test	115200	16x	8.640	8.681	8.681	0.472%	PASSED
baud2400_13x_test	2400	13x	416.650	416.667	416.667	0.004%	PASSED
baud4800_13x_test	4800	13x	208.260	208.333	208.333	0.035%	PASSED
baud9600_13x_test	9600	13x	104.130	104.167	104.167	0.036%	PASSED
baud19200_13x_test	19200	13x	52.083	52.083	52.083	0%	PASSED
baud38400_13x_test	38400	13x	26.000	26.042	26.042	0.161%	PASSED
baud76800_13x_test	76800	13x	13.000	13.021	13.021	0.160%	PASSED
baud115200_13x_test	115200	13x	8.710	8.681	8.681	0.334%	PASSED

Hình 4.3 và Hình 4.4 minh họa waveform của hai testcase đại diện ở hai đầu dải tốc độ. Với baud115200_16x_test, chu kỳ bit TX đo được là 8.640 μ s và RX là 8.681 μ s, sai lệch so với lý thuyết (8.681 μ s) lần lượt là 0.47% và 0% — nằm trong ngưỡng cho phép. Với baud2400_16x_test, chu kỳ bit TX đo được là 416.640 μ s và RX là 416.667 μ s, khớp với giá trị lý thuyết (416.667 μ s) với sai lệch không đáng kể. Kết quả toàn bộ 14 testcase đều PASSED.



Hình 4.3: Waveform baud115200_16x_test



Hình 4.4: Waveform baud2400_16x_test

4.1.3 Kiểm thử cấu hình khung truyền UART

Nhóm kiểm thử cấu hình khung truyền UART được thực hiện nhằm xác minh khả năng hỗ trợ các cấu hình độ rộng dữ liệu, chế độ parity và số lượng stop bit của AHB - UART IP. Các testcase được xây dựng để kiểm tra dữ liệu có độ dài từ 5 đến 8 bit, ba chế độ parity, cùng hai cấu hình 1 stop bit và 2 stop bit.

Hình 4.5 minh họa testcase cấu hình 5 bit dữ liệu, even parity và 1 stop bit với dữ liệu truyền là 0x1A. Waveform cho thấy khung truyền gồm 1 bit start, 5 bit dữ liệu, 1 bit parity và 1 bit stop.



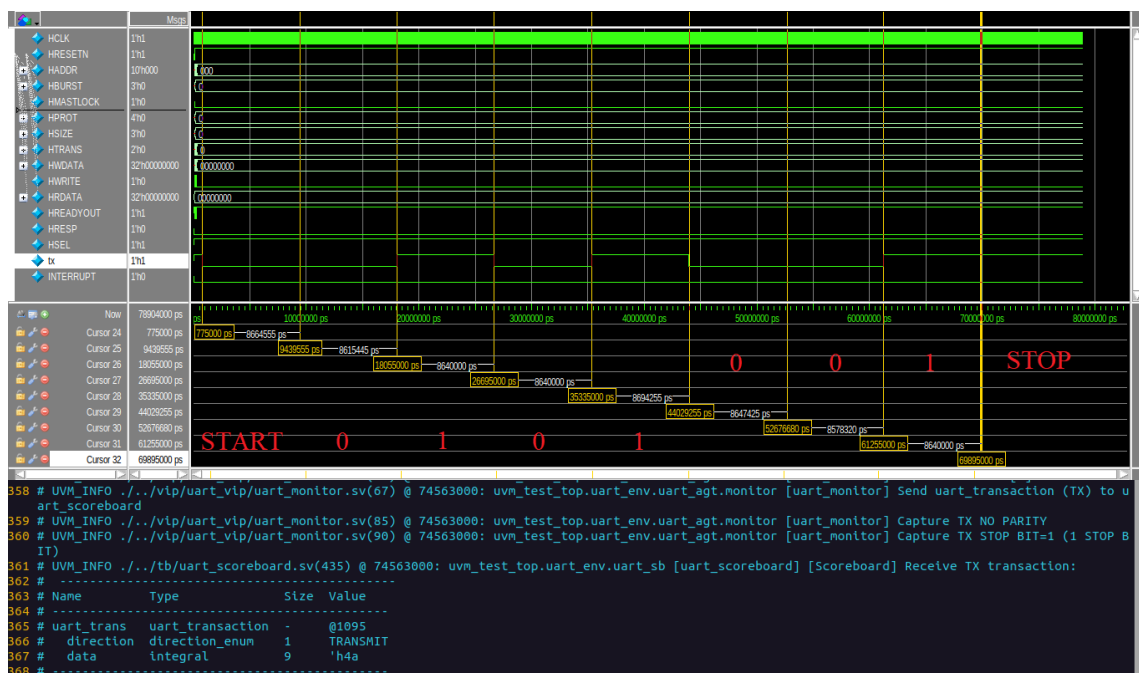
Hình 4.5: Waveform cấu hình 5 bit dữ liệu, parity chẵn và 1 stop bit

Hình 4.6 minh họa testcase cấu hình 6 bit dữ liệu, odd parity và 1 stop bit với dữ liệu truyền là 0x2A. Waveform thể hiện đầy đủ 6 bit dữ liệu, bit parity lẻ và 1 bit stop theo đúng cấu hình thiết lập.



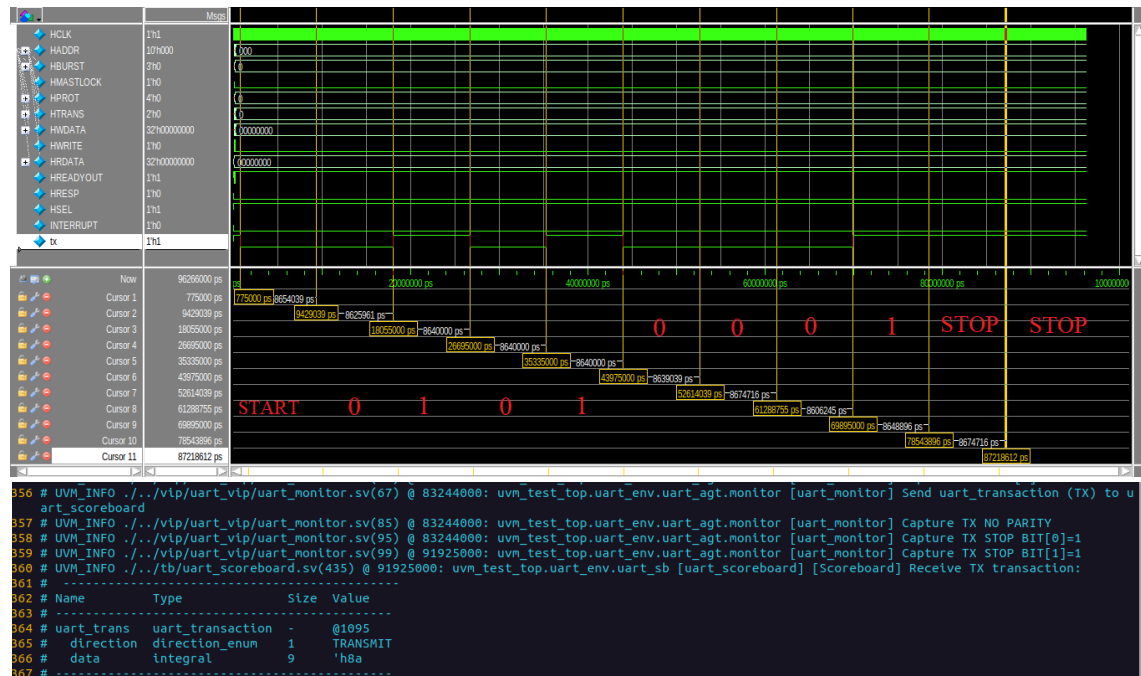
Hình 4.6: Waveform cấu hình 6 bit dữ liệu, parity lẻ và 1 stop bit

Hình 4.7 thể hiện testcase cấu hình 7 bit dữ liệu, không parity và 1 stop bit với dữ liệu truyền là 0x4A. Sau khi truyền đủ 7 bit dữ liệu, khung truyền chuyển trực tiếp sang stop bit mà không xuất hiện parity bit.



Hình 4.7: Waveform cấu hình 7 bit dữ liệu, không parity và 1 stop bit.

Hình 4.8 minh họa testcase cấu hình 8 bit dữ liệu, không parity và 2 stop bit với dữ liệu truyền là 0x8A. Waveform cho thấy sau 8 bit dữ liệu xuất hiện liên tiếp hai stop bit ở mức logic cao. Kết quả testcase PASSED.



Hình 4.8: Waveform cấu hình 8 bit dữ liệu, không parity và 2 stop bit.

Kết quả cho thấy AHB - UART IP hỗ trợ đầy đủ các cấu hình độ dài dữ liệu, parity và stop bit theo đặc tả thiết kế.

Bảng 4.2: Kết quả kiểm thử cấu hình khung truyền UART

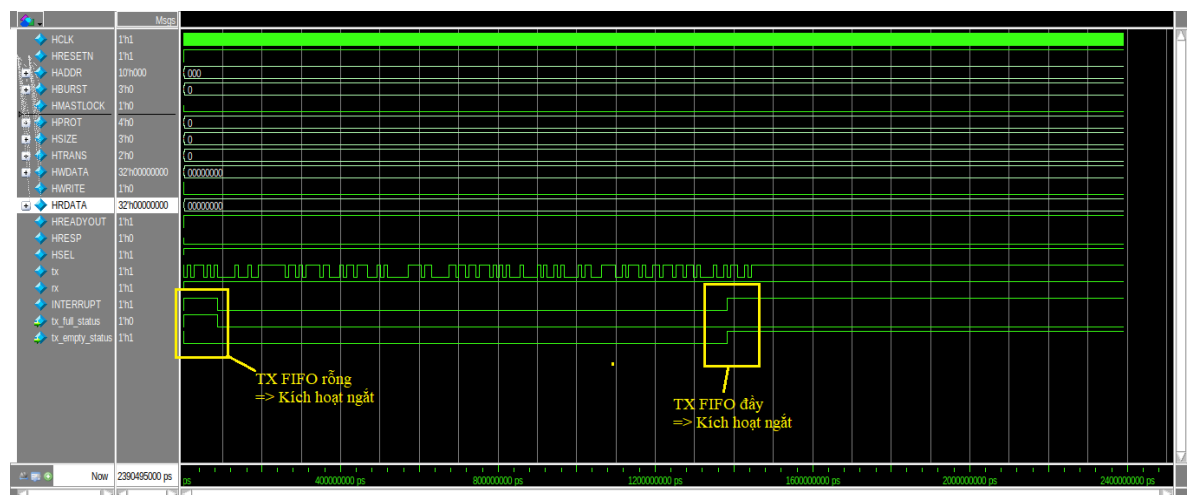
Hạng mục kiểm tra	Giá trị kiểm tra	Kết quả
Độ dài dữ liệu	5, 6, 7, 8 bit	PASSED
Chế độ parity	Không, chẵn, lẻ	PASSED
Số stop bit	1, 2	PASSED

4.1.4 Kiểm thử FIFO và cơ chế ngắt

Nhóm kiểm thử FIFO và cơ chế ngắt gồm hai testcase tx_interrupt_test và rx_interrupt_test, được thực hiện nhằm xác minh tín hiệu ngắt của TX FIFO và RX FIFO khi ở trạng thái rỗng hoặc đầy, đồng thời kiểm tra hành vi của hệ thống khi ghi dữ liệu vượt quá dung lượng FIFO.

Testcase tx_interrupt_test kiểm tra cơ chế ngắt TX FIFO qua ba giai đoạn. Trước tiên, hai bit ngắt IER.en_tx_fifo_full và IER.en_tx_fifo_empty được kích hoạt thông qua thao tác ghi vào thanh ghi IER. Tiếp theo, hệ thống được cấu hình với chế độ oversampling 16x (MDR.OSM_SEL = 0), tốc độ baud 115200 (DLL = 0x36), khung truyền 8 bit dữ liệu, 1 stop bit, không parity và kích hoạt bộ tạo baud rate thông qua LCR.BGE = 1'b1.

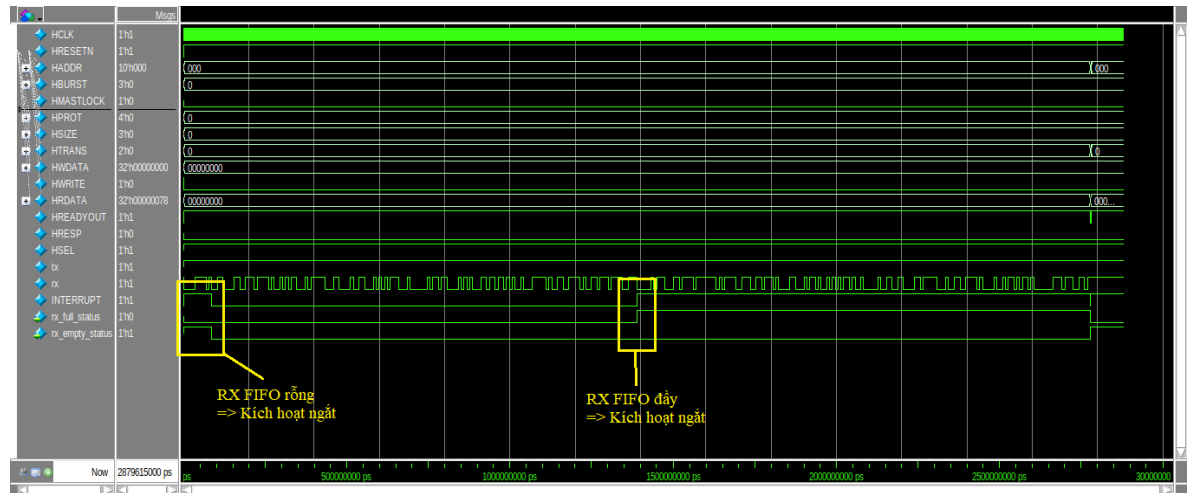
Hình 4.5 thể hiện waveform của testcase tx_interrupt_test. Ở giai đoạn đầu, khi TX FIFO ở trạng thái rỗng, tín hiệu tx_empty_status được kích hoạt ở mức cao trong khi tx_full_status ở mức thấp, tín hiệu INTERRUPT được kích hoạt tương ứng. Tiếp theo, 32 byte dữ liệu ngẫu nhiên được ghi liên tiếp vào TX FIFO. Khi 16 byte đầu tiên lấp đầy TX FIFO, tx_full_status chuyển lên mức cao và tx_empty_status bị hủy kích hoạt, tín hiệu INTERRUPT tiếp tục duy trì ở mức cao phản ánh trạng thái FIFO đầy. Kết quả testcase PASSED.



Hình 4.5: Waveform testcase tx_interrupt_test

Tương tự, testcase rx_interrupt_test kiểm tra cơ chế ngắt RX FIFO với cấu hình tương đương. Hai bit ngắt IER.en_rx_fifo_full và IER.en_rx_fifo_empty được kích hoạt trước khi bắt đầu quá trình nhận dữ liệu. Hình 4.6 thể hiện waveform của testcase rx_interrupt_test. Ban đầu khi RX FIFO rỗng, rx_empty_status ở mức cao và tín hiệu INTERRUPT được kích hoạt tương ứng. Khi dữ liệu được nhận liên tiếp đến khi RX FIFO đầy, rx_full_status chuyển lên

mức cao, rx_empty_status bị hủy kích hoạt và INTERRUPT tiếp tục duy trì ở mức cao. Các thao tác nhận dữ liệu vượt dung lượng chỉ trả về 16 giá trị ban đầu trong FIFO mà không ghi đè dữ liệu hiện có. Kết quả testcase PASSED.



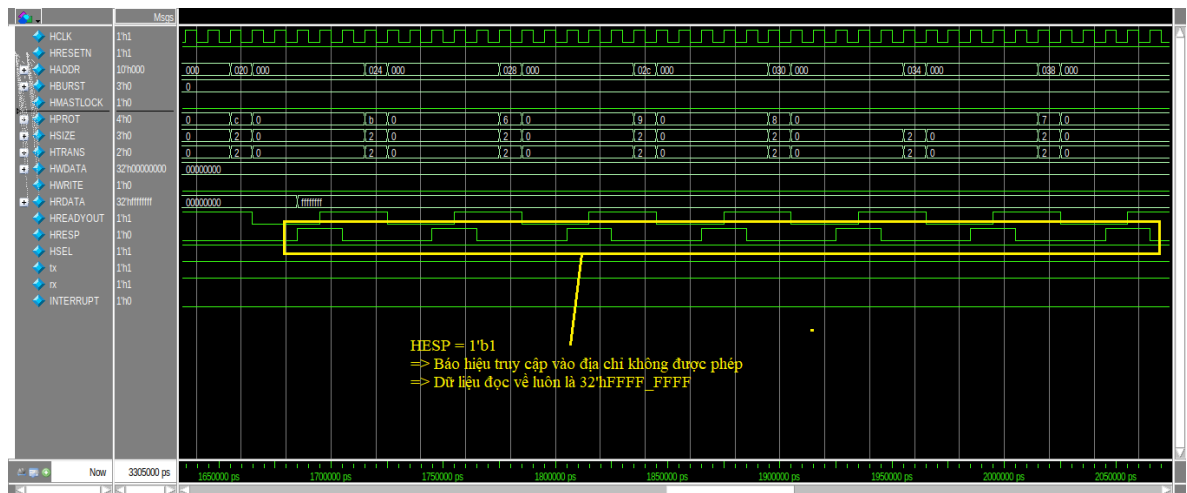
Hình 4.6: Waveform testcase rx_interrupt_test

4.1.5 Kiểm thử xử lý lỗi

Nhóm kiểm thử xử lý lỗi gồm 2 testcase được thực hiện nhằm xác minh hành vi của hệ thống trong các tình huống ngoại lệ, bao gồm truy cập địa chỉ không hợp lệ, cấu hình parity không đúng và cơ chế ngắt khi nhận dữ liệu có lỗi parity.

Testcase rsvd_test xác minh hành vi của hệ thống khi thực hiện các giao dịch đọc và ghi vào vùng địa chỉ không hợp lệ nằm ngoài register map của AHB Lite - UART IP.

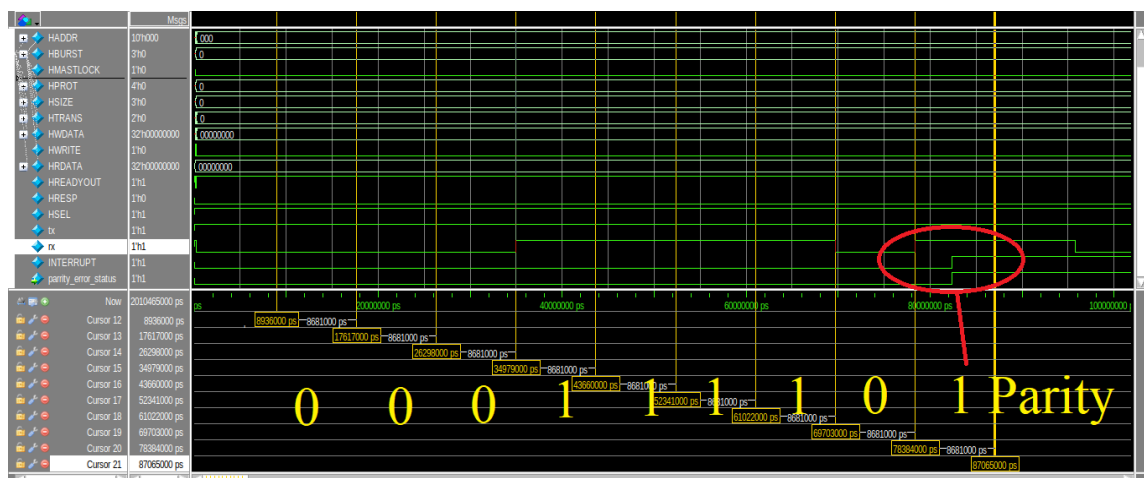
Hình 4.7 thể hiện waveform của testcase rsvd_test. Tín hiệu HADDR thay đổi tuần tự qua các địa chỉ reserved (0x020, 0x024, 0x028, 0x02C, 0x030, 0x034, 0x038). Với mỗi giao dịch truy cập, tín hiệu HRESP được kích hoạt lên mức cao báo hiệu truy cập không hợp lệ, đồng thời HRDATA trả về giá trị 0xFFFFFFFF. Kết quả testcase PASSED.



Hình 4.7: Waveform testcase rsvd_test

Testcase parity_error_test xác minh ngắt lỗi parity khi RX nhận dữ liệu sai cấu hình. Trước khi nhận, bit ngắt IER.parity_error_status được kích hoạt. Hệ thống cấu hình theo chế độ: oversampling 16x, baudrate 115200, 8 bit dữ liệu, 1 stop bit và even parity.

Hình 4.8 thể hiện waveform của testcase parity_error_test. Dữ liệu được truyền đi là 0x78 (0111 1000), với tổng số bit 1 là 4 (số chẵn), bit parity đúng theo chế độ even parity phải là 0. Tuy nhiên, UART driver cố tình truyền bit parity là 1, khiến RX phát hiện lỗi parity ngay khi nhận được khung truyền và lập tức kích hoạt tín hiệu INTERRUPT. Kết quả testcase PASSED.



Hình 4.8: Waveform testcase parity_error_test

4.1.6 Kiểm thử ngẫu nhiên có ràng buộc

Nhóm kiểm thử ngẫu nhiên có ràng buộc gồm 15 testcase được xây dựng nhằm mở rộng không gian kiểm tra bằng cách ngẫu nhiên hóa cấu hình khung truyền UART trên từng mức tốc độ baud.

Với mỗi testcase, các thông số khung truyền bao gồm độ rộng dữ liệu, chế độ parity và số stop bit được ngẫu nhiên hóa có ràng buộc, trong khi tốc độ baud được cố định ở từng mức tương ứng. Mỗi testcase được chạy lặp lại 100 lần, tổng cộng 1500 lần chạy nhằm tăng khả năng phát hiện lỗi tiềm ẩn.

Kết quả kiểm thử hồi quy (regression) được tổng hợp và lưu lại trong file log mô phỏng, minh họa tại Hình 4.9.

```
3983 # UVM_INFO ./.../regmodel/uart_reg2ahb_adapter.sv(43) @ 1737295000: reporter [uart_reg2ahb_adapter] bus2reg: address=0x1c, data=0xf7, kind=UVM_READ, status = UVM_IS_OK
3984 # UVM_INFO @ 1737295000: reporter [RegModel] Read register via map regmodel.ahb_map: regmodel.RBR=f7
3985 # UVM_INFO ./.../testcases/rand_frame_13x_115200_test.sv(75) @ 1737295000: uvm_test_top [rand_frame_13x_115200_test] Read data from RBR[19]: rdata=f7
3986 # UVM_INFO /home/dt/Questasin/questasin/verilog_src/uvm-1.2/src/base/uvm_object.tcl(1270) @ 1894255000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
3987 # =====
3988 # REGRESSION REPORT
3989 # =====
3990 # STT Testcase Runs Result
3991 # -----
3992 # 1 rand_frame_16x_2400_test 100 PASSED
3993 # 2 rand_frame_16x_4800_test 100 PASSED
3994 # 3 rand_frame_16x_9600_test 100 PASSED
3995 # 4 rand_frame_16x_19200_test 100 PASSED
3996 # 5 rand_frame_16x_38400_test 100 PASSED
3997 # 6 rand_frame_16x_76800_test 100 PASSED
3998 # 7 rand_frame_16x_115200_test 100 PASSED
3999 # 8 rand_frame_13x_2400_test 100 PASSED
4000 # 9 rand_frame_13x_4800_test 100 PASSED
4001 # 10 rand_frame_13x_9600_test 100 PASSED
4002 # 11 rand_frame_13x_19200_test 100 PASSED
4003 # 12 rand_frame_13x_38400_test 100 PASSED
4004 # 13 rand_frame_13x_76800_test 100 PASSED
4005 # 14 rand_frame_13x_115200_test 100 PASSED
4006 # 15 rand_disparity_error_test 100 PASSED
4007 # -----
4008 # Total: 1500 runs - 15/15 PASSED
4009 # =====
4010 # -----
4011 # TEST PASSED
4012 # -----
4013 # UVM_INFO /home/dt/Questasin/questasin/verilog_src/uvm-1.2/src/base/uvm_report_server.sv(847) @ 1894255000: reporter [UVM/REPORT/SERVER]
4014 #
```

Hình 4.9: Báo cáo kiểm thử hồi quy của kiểm thử ngẫu nhiên ràng buộc

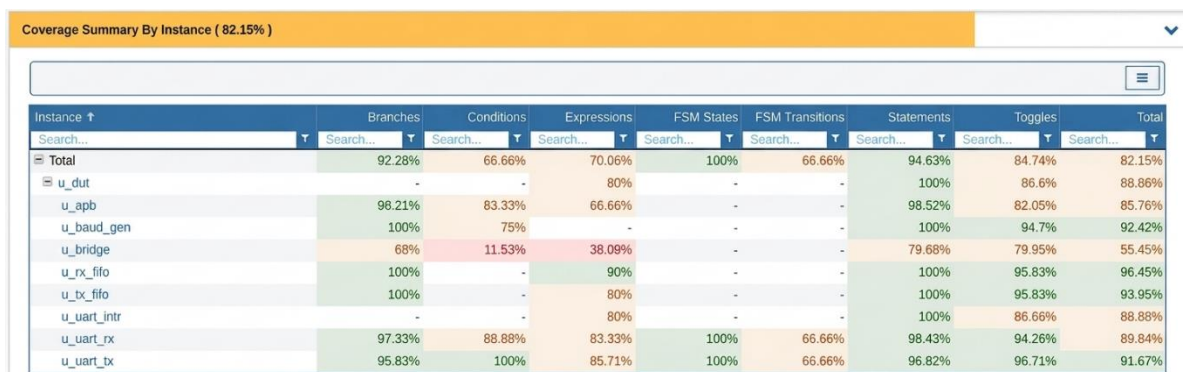
Toàn bộ 15 testcase đều PASSED sau 1500 lần chạy, xác nhận hệ thống hoạt động ổn định trên toàn bộ các tổ hợp cấu hình khung truyền được ngẫu nhiên hóa ở cả hai chế độ oversampling 16x và 13x. Kết quả này đồng thời góp phần tăng độ bao phủ chức năng cho các tổ hợp cấu hình chưa được kiểm tra trong các nhóm kiểm thử định hướng trước đó.

4.2 Kết quả độ bao phủ

4.2.1 Độ bao phủ mã nguồn

Hình 4.10 thể hiện kết quả độ bao phủ mã nguồn thu được sau khi hoàn thành toàn bộ các testcase. Tổng độ bao phủ toàn hệ thống đạt 82.15%, trong đó độ bao phủ câu lệnh đạt 94.63%, độ bao phủ nhánh đạt 92.28% và độ bao phủ toggle đạt 84.74%.

Xét theo từng khối chức năng, u_baud_gen đạt độ bao phủ cao nhất với 92.42%, tiếp theo là u_rx_fifo và u_tx_fifo lần lượt đạt 96.45% và 93.95%. Khối u_bridge có độ bao phủ thấp nhất với 55.45%, chủ yếu do độ bao phủ điều kiện chỉ đạt 11.53% và độ bao phủ biểu thức đạt 38.09%, phản ánh một số nhánh xử lý đặc thù của cầu nối AHB-APB chưa được kích hoạt trong quá trình kiểm thử.



Instance	Branches	Conditions	Expressions	FSM States	FSM Transitions	Statements	Toggles	Total
Total	92.28%	66.66%	70.06%	100%	66.66%	94.63%	84.74%	82.15%
u_dut	-	-	80%	-	-	100%	86.6%	88.86%
u_apb	98.21%	83.33%	66.66%	-	-	98.52%	82.05%	85.76%
u_baud_gen	100%	75%	-	-	-	100%	94.7%	92.42%
u_bridge	68%	11.53%	38.09%	-	-	79.68%	79.95%	55.45%
u_rx_fifo	100%	-	90%	-	-	100%	95.83%	96.45%
u_tx_fifo	100%	-	80%	-	-	100%	95.83%	93.95%
u_uart_intr	-	-	80%	-	-	100%	86.66%	88.88%
u_uart_rx	97.33%	88.88%	83.33%	100%	66.66%	98.43%	94.26%	89.84%
u_uart_tx	95.83%	100%	85.71%	100%	66.66%	96.82%	96.71%	91.67%

Hình 4.10: Kết quả độ bao phủ mã nguồn

4.2.2 Độ bao phủ chức năng

Mô hình độ bao phủ chức năng được xây dựng theo thiết kế đã trình bày ở Mục 3.3.5, bao gồm 10 coverpoint đơn lẻ và 4 điểm bao phủ giao chéo trong nhóm bao phủ UART_GROUP. Hình 4.11 thể hiện kết quả độ bao phủ chức năng thu được sau khi hoàn thành toàn bộ các testcase. Toàn bộ các coverpoint và cross coverage đều đạt 100%, bao gồm các coverpoint đơn lẻ như sampling_mode, baud_rate, data_width, stop_bit, parity_mode và 5 bit ngắt trong thanh ghi IER, cùng với các điểm bao phủ giao chéo osm_x_baud, frame_cfg, interrupt_mode và functional_cross với 336 tổ hợp cấu hình. Kết quả này xác nhận toàn bộ không

gian cấu hình chức năng của AHB Lite - UART IP đã được kiểm tra đầy đủ theo kế hoạch kiểm chứng đề ra.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
renu_gkguart_scoreboard		100.00%	100	100.00%	✓				
TYPE UART_GROUP		100.00%	100	100.00%	✓				auto()
CVP UART_GROUP::sampling_mode		100.00%	100	100.00%	✓				
CVP UART_GROUP::baud_rate		100.00%	100	100.00%	✓				
CVP UART_GROUP::data_width		100.00%	100	100.00%	✓				
CVP UART_GROUP::stop_bit		100.00%	100	100.00%	✓				
CVP UART_GROUP::parity_mode		100.00%	100	100.00%	✓				
CVP UART_GROUP::en_parity_error		100.00%	100	100.00%	✓				
CVP UART_GROUP::en_rx_fifo_empty		100.00%	100	100.00%	✓				
CVP UART_GROUP::en_tx_fifo_full		100.00%	100	100.00%	✓				
CVP UART_GROUP::en_tx_fifo_empty		100.00%	100	100.00%	✓				
CVP UART_GROUP::en_tx_fifo_full		100.00%	100	100.00%	✓				
CROSS UART_GROUP::osm_x_baud		100.00%	100	100.00%	✓				
CROSS UART_GROUP::frame_cfg		100.00%	100	100.00%	✓				
CROSS UART_GROUP::interrupt_mode		100.00%	100	100.00%	✓				
CROSS UART_GROUP::functional_cross		100.00%	100	100.00%	✓				
bin <oversampling_16x.baud_76800.data_...		2	1	100.00%	✓				
bin <oversampling_16x.baud_2400.data_...		2	1	100.00%	✓				
bin <oversampling_13x.baud_4800.data_...		2	1	100.00%	✓				
bin <oversampling_13x.baud_19200.data_...		2	1	100.00%	✓				
bin <oversampling_13x.baud_115200.data_...		6	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		3	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		6	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		9	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		6	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		9	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		24	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		9	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		9	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		18	1	100.00%	✓				
bin <oversampling_16x.baud_9600.data_...		6	1	100.00%	✓				

Hình 4.11: Kết quả độ bao phủ chức năng

5. Kết luận và phương hướng phát triển

5.1. Kết luận

Sau khi thực hiện đề tài "Thiết kế và kiểm chứng lõi AHB Lite - UART IP theo phương pháp UVM", nhóm đã đưa ra những kết luận sau:

Về ưu điểm:

- Đề tài đã thiết kế thành công lõi AHB Lite - UART IP hỗ trợ đầy đủ các tính năng truyền nhận dữ liệu nối tiếp theo đặc tả.
- Môi trường kiểm chứng UVM được xây dựng theo kiến trúc phân cấp, có khả năng tái sử dụng và mở rộng cao.
- Toàn bộ 52 testcase đều PASSED, xác nhận hệ thống hoạt động đúng theo đặc tả.
- Độ bao phủ chức năng đạt 100% trên toàn bộ không gian cấu hình, độ bao phủ mã nguồn đạt 82.15%.
- Lớp trừu tượng hóa thanh ghi UVM RAL giúp đơn giản hóa việc truy cập và kiểm tra các thanh ghi cấu hình.

Về nhược điểm:

- Độ bao phủ điều kiện của khối cầu nối AHB-APB còn thấp do một số tham số thiết kế được cố định.
- Chưa kiểm thử và đánh giá hệ thống trên nền tảng FPGA thực tế.
- Chưa đánh giá các chỉ số timing, công suất tiêu thụ của thiết kế.

5.2. Hướng phát triển

Dựa trên kết quả đạt được, đề tài có thể được phát triển theo các hướng sau:

- Mở rộng môi trường kiểm chứng để hỗ trợ kiểm thử đa luồng và các kịch bản truyền nhận phức tạp hơn.
- Tích hợp kiểm chứng hình thức nhằm nâng cao độ tin cậy của thiết kế, đặc biệt đối với các trạng thái chưa được kích hoạt trong quá trình mô phỏng.

- Đánh giá và tối ưu hóa thiết kế về mặt timing và công suất tiêu thụ thông qua quá trình tổng hợp logic.
- Triển khai và kiểm thử lõi AHB Lite - UART IP trên nền tảng FPGA thực tế nhằm đánh giá khả năng hoạt động trong các ứng dụng nhúng.
- Tích hợp lõi AHB Lite - UART IP vào hệ thống SoC hoàn chỉnh và kiểm chứng ở mức hệ thống

TÀI LIỆU THAM KHẢO

- [1] H. Chen, “Design and Implementation of a Synchronous AHB to APB Bridge in System-on-Chip Applications,” *Dean Francis Press*, Oct. 2024.
- [2] M. Roopa, R. M. Vani, and P. V. Hunagund, “UART Controller as AMBA APB Slave,” in *Proc. National Conf. on Challenges in Research & Technology in the Coming Decades (CRT 2013)*, IET, 2014.
- [3] Y. Liu, L. Zhang, S. Xu, J. Wang, C. N. Wang, and L. Zhou, “Design and Verification of UART Circuit of SoC Based on AMBA Bus,” in *Proc. IEEE Conf.*, 2021.
- [4] R. Yamini and M. V. Ramya, “Design and Verification of UART using System Verilog,” *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 9, no. 5, Jun. 2020.
- [5] S. S. K. K. Yadav, P. Satyanarayana, B. V. Krishna, and V. G. Krishnan, “Design and Verification of APB Protocol,” in *Proc. IEEE Conf.*, 2024.
- [6] A. Srinivasan, “SystemVerilog Verification of Wishbone-Compliant Serial Peripheral Interface,” M.S. thesis, Rochester Institute of Technology, Dec. 2017.
- [7] S. Balaraju, “UVM Verification of an I2C Master Core,” M.S. thesis, Rochester Institute of Technology, May 2019.
- [8] P. T. Ngoc, H. N. Bao, D. D. Tan, and T. Q. Phuc, “Thiết kế và thi công bộ truyền nhận UART đa kênh dựa trên công nghệ FPGA,” *Trường Đại học Sư phạm Kỹ thuật TP.HCM*, Sep. 2020.
- [9] Siemens Digital Industries Software, *UVM Cookbook*, Siemens EDA.
- [10] C. Spear, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, Synopsys, Inc., Marlboro, MA.

- [11] ChipVerify, “ChipVerify — Learn Verilog, SystemVerilog and UVM,” *chipverify.com*. [Online]. Available: <https://www.chipverify.com>. [Accessed: Jun. 2026].
- [12] ARM Limited, *AMBA AHB-Lite Protocol Specification*, ARM IHI0033A, 2010.
- [13] ARM Limited, *AMBA APB Protocol Specification*, ARM IHI0024C, 2010.
- [14] ARM Limited, "AMBA Specification (Rev 2.0)," ARM IHI 0011A, ARM Limited, Cambridge, UK, 1999.
- [15] E. Peña and M. G. Legaspi, “UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter,” *Analog Dialogue*, vol. 54, no. 4, pp. 1-5, Dec. 2020.